

Associate Architect 양성과정 · Architecture 설계

Day 1 · Architecture 스타일 (1)

과정 소개 · Data Flow / Data Centered / Implicit Invocation / Interaction-Oriented / Hierarchical / Distributed(1)

- 0. Course Introduction · 1.1 Introduction to Architecture Style
- 1.2 Data Flow · 1.3 Data Centered · 1.4 Implicit Invocation Architecture
- 1.5 Interaction-Oriented · 1.6 Hierarchical Architecture
- 1.7 Distributed Architecture (1)

Architectural Style 주제: 스타일 분류

■ 7개 category, 23개 style

Category	Style	Sub-style
Data Flow	Batch Sequential / Pipe and Filter / Process Control	
Data Centered	Shared Repository / Blackboard	
Implicit Invocation	Event Based / Message Based	
Interaction-Oriented	Model-View-Controller / Presentation-Abstraction-Control	
Hierarchical	Main-Subroutine / Master-Slave / Layered	Virtual Machine / Microkernel
Distributed	Multi-tier / Broker / Service-Oriented	Client-Server / Dispatcher / Microservice
Deployment	Container-Based / Cloud-Based / Edge-Based	

1.1

Introduction to Architecture Style

건축 양식의 비유로 이해하는 Architectural Style 과 Quality Attribute, 그리고 Pattern 과의 관계

- 건축 양식 비유 (Gothic / Byzantine)
- Architectural Style 의 정의와 구성 요소
- Quality Attributes
- Architectural Style 카탈로그
- Pattern 과 Style

건축 양식(Architectural Styles): Gothic Style

■ Gothic 양식의 특징

- 뾰족한 첨탑
- 스테인드 글라스
- 높고 둥근 아치형 천장



뾰족한 첨탑 (밀라노 두오모)



스테인드 글라스



높고 둥근 아치형 천장

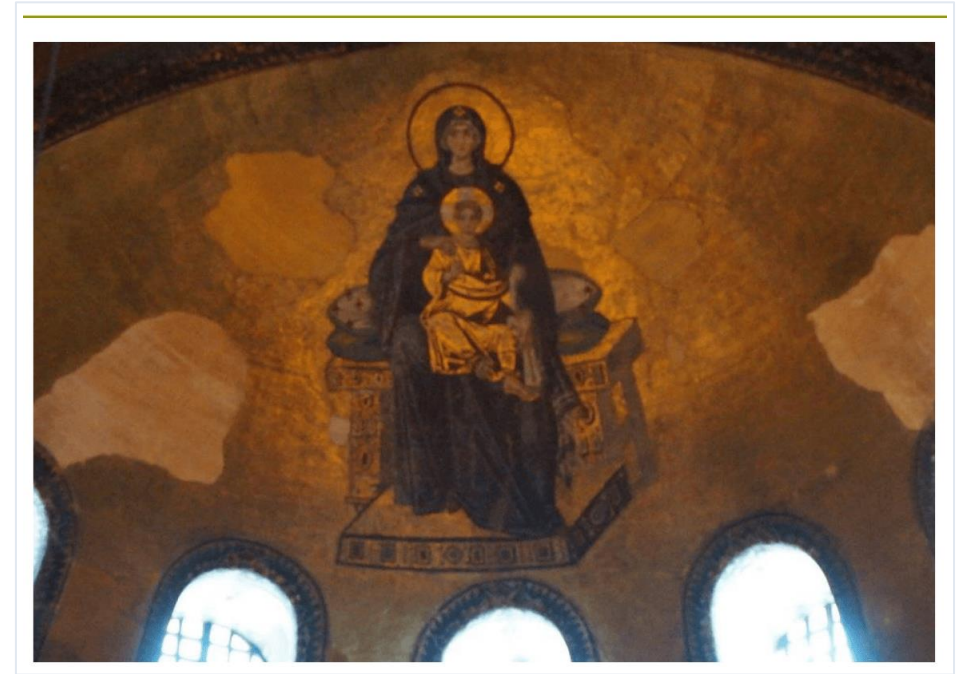
건축 양식(Architectural Styles): Byzantine Style

■ Byzantine 양식의 특징

- 정사각형 벽 위에 둥근 돔 설치
- 모자이크 내부 장식



정사각형 벽 위의 둥근 돔 (성 소피아 성당)



모자이크 내부 장식

성당 건축으로 본 두 가지 Style

■ Gothic Style

- 뾰족한 탑
- 높고 둥근 아치형 천장
- 스테인드 글라스

■ Byzantine Style

- 정사각형 벽 위에 둥근 돔 설치
- 모자이크 내부 장식



세인트 패트릭스
대성당, 뉴욕



밀라노 두오모, 밀라노

Gothic: 세인트 패트릭 대성당(뉴욕), 밀라노 두오모(밀라노)



성 마르크 성당, 베니스



성 소피아 성당, 이스탄불

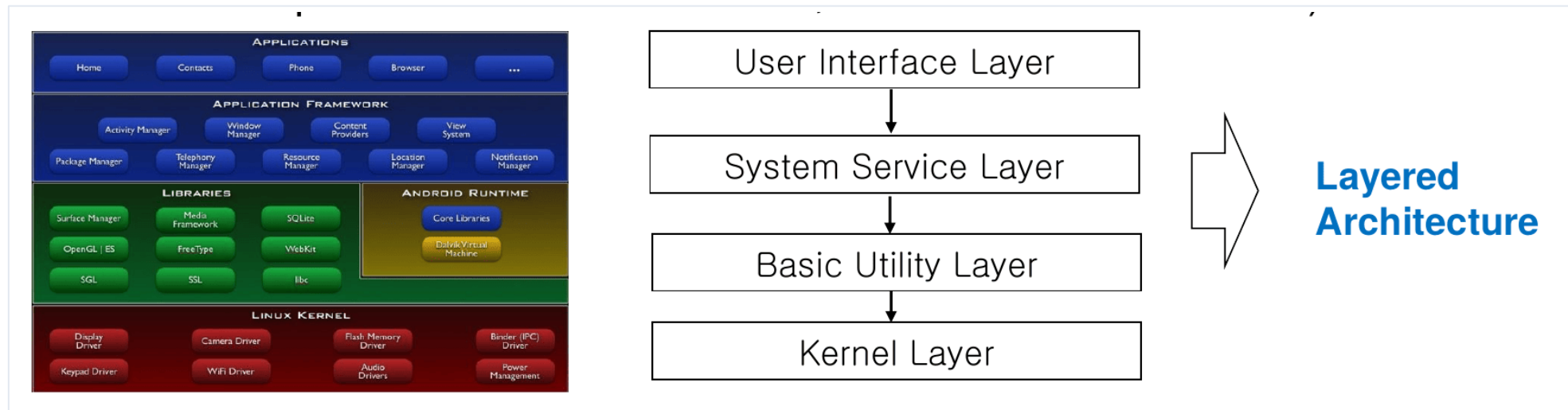
Byzantine: 성 마르크 성당(베니스), 성 소피아 성당(이스탄불)

핵심 같은 양식의 건물은 반복되는 공통 특징을 가진다 — 소프트웨어의 Architectural Style 도 마찬가지다

소프트웨어의 Architectural Style 이란

■ Architectural Style 은

- 실무에서 반복적으로 발견되는 Software Architecture 의 한 부류(class)를 기술한다
- 소프트웨어 시스템의 구조적 특징(structural features)을 추상화한 것이다
- 이름이 붙은 설계 결정(design decisions)의 모음이다
 - 예: 요소의 유형(element types), 요소 간의 관계, 여러 가지 제약(constraints)



Android 플랫폼 구조를 추상화한 Layered Architecture

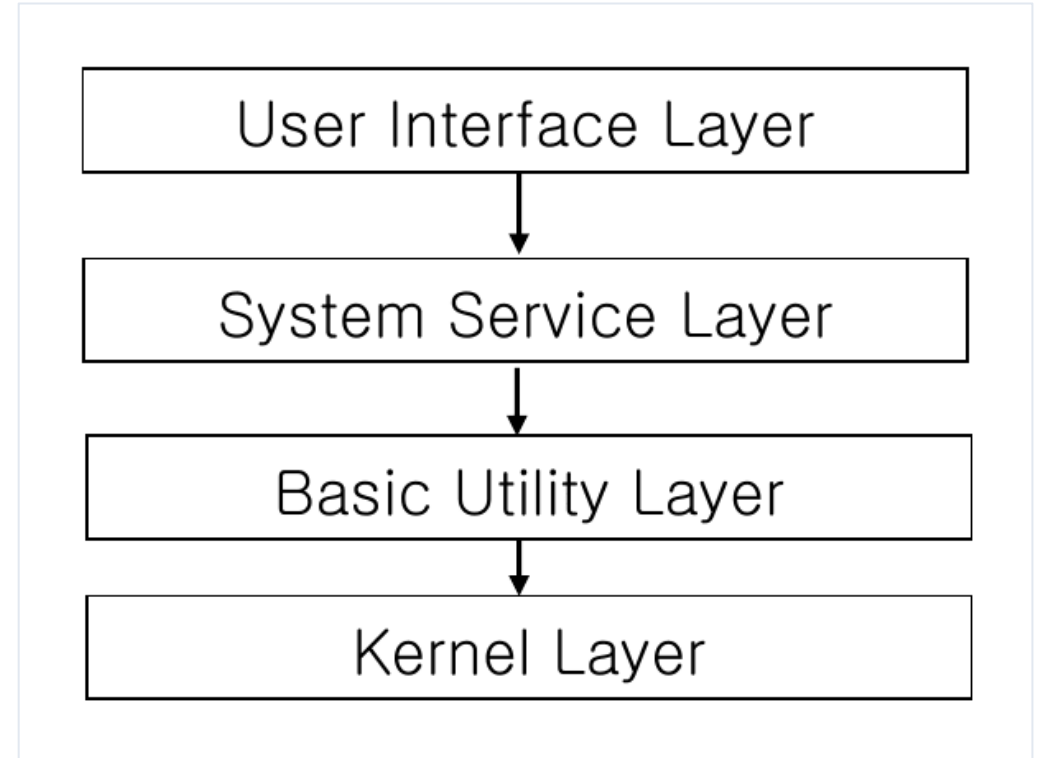
Style 을 결정하는 세 요소와 Layered Style 예

■ Architectural Style 을 결정하는 것

- 요소 유형(element types)의 집합
- 요소 사이의 관계(relationships)의 집합
- 토폴로지와 행위에 대한 제약(constraints)의 집합

■ 예: Layered Style

- Elements: Layer (module 의 한 종류)
- Relations: allowed-to-use (일반적인 depends-on 관계를 특수화한 것)
- Constraints: allowed-to-use 관계는 순환(circular)이 되면 안 된다



Layered Style: 위 layer 가 아래 layer 를 사용한다

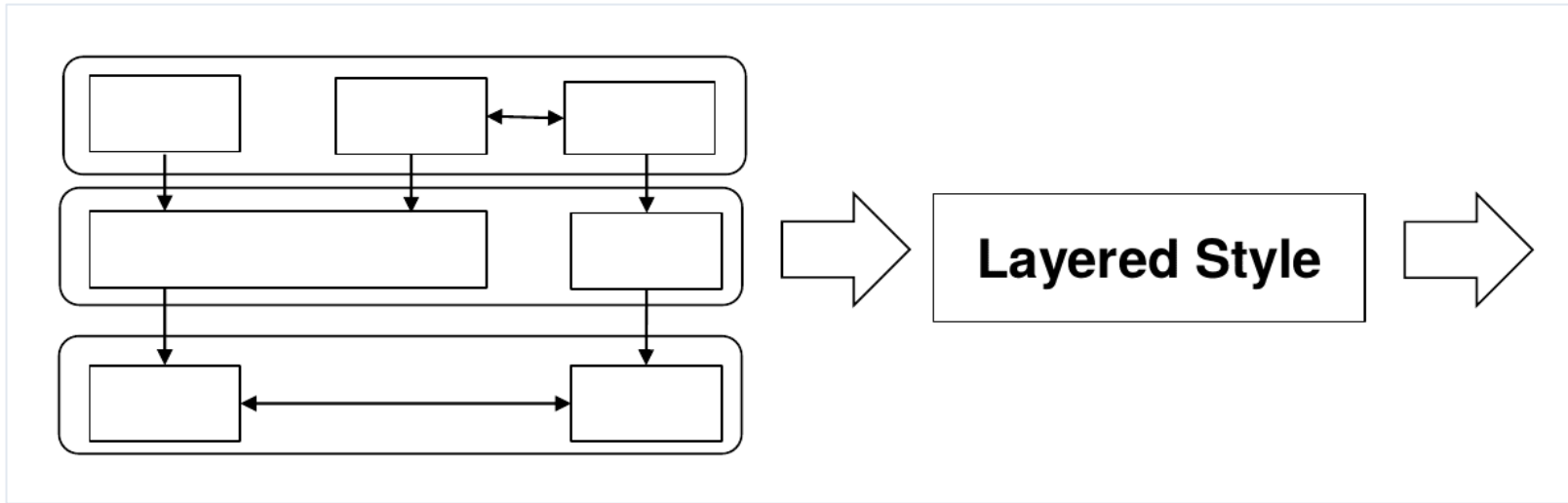
Architectural Style이 중요한 이유

■ Architectural Style 은 유사한 Software Architecture 들의 추상화이다

- 구체적인 시스템들에서 공통 구조를 뽑아내 이름을 붙인 것

■ Style 은 Quality Attribute 를 추론(reason)하는 데 도움이 된다

- Style 을 알면 그 구조가 어떤 품질 속성에 유리하고 불리한지 미리 판단할 수 있다
- 예: Layered Style → Modifiability(+), Reusability(+), Runtime Performance(-)



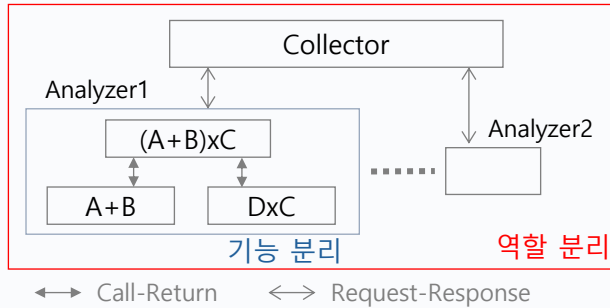
유사한 구조들 → Layered Style → 품질 속성에 대한 판단

SW 설계의 4가지 원칙 (Design Principles)

■ 설계(Design)는 4가지 원칙에 대한 선택의 조합이며, 각 선택은 QA 에 서로 다른 영향(trade-off)을 미친다

Architectural Style 은 이 원칙들을 특정한 방식으로 조합해 놓은 것 → Style 을 고르는 일은 곧 QA trade-off 를 고르는 일

1. 기능 / 역할의 분리 Separation of Function / Role

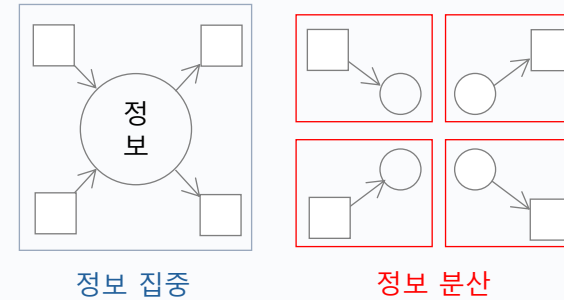


기능(Function) : 런타임에 단독으로 드러나지 않는 모듈 단위 (class, library 등) — 코드 수준의 분리

역할(Role) : 런타임에 단독으로 드러나는 컴포넌트 단위 (process, service 등) — 실행 수준의 분리

핵심 : 변경·Test 범위의 국소화, 장애 격리 ↔ 통합 복잡도 증가

2. 정보의 집중 / 분산 Centralization / Distribution of Information

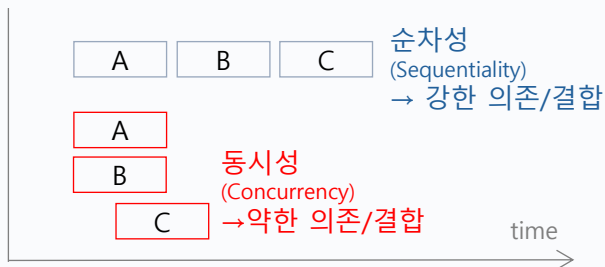


집중 : 하나의 저장소·소유자에 데이터를 모음 (DB, Shared Repository)

분산 : 소유자별로 데이터를 분할·복제 (Service 별 DB, Cache, Edge)

핵심 : 접근 용이성·일관성 ↔ 단일 장애점(SPOF)·공격 포인트 집중

3. 순차성 / 동시성 Sequentiality / Concurrency



순차 : 하나의 흐름이 차례로 처리 — 결정적(deterministic), 재현 용이

동시 : 다수의 흐름(thread, process)이 동시에 처리 — 자원 활용·응답성

핵심 : 처리량·응답성 ↔ 동기화·race·deadlock 등 복잡도 증가

4. 강한 / 약한 결합 Tight / Loose Coupling



강한 결합 : 직접 호출·공유 메모리 등 상대의 구현을 아는 의존

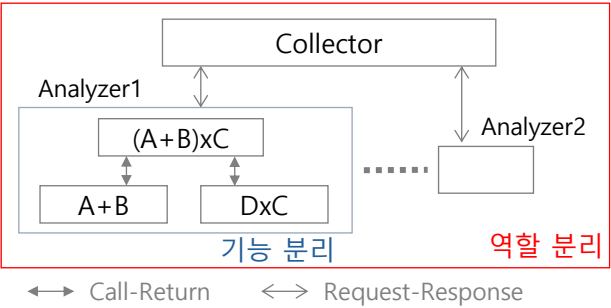
약한 결합 : Interface·Service·Mediator 를 통한 간접 의존

핵심 : 직접성(성능·단순함) ↔ 독립성(변경·교체·격리)

※ 각 원칙은 '어느 쪽이 옳다'가 아니라 시스템의 QA 우선순위에 따라 선택하는 것 → 다음 4장에서 원칙별 QA 영향의 원인을, 이어서 전체 매트릭스를 정리

원칙 1. 기능 / 역할의 분리

Separation of Function / Role



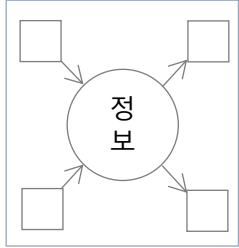
기능(Function) : 런타임에 단독으로 드러나지 않는 모듈 단위(class, library, package). 같은 프로세스 안에서 코드 수준으로 책임을 나눈다 → Call-Return 관계

역할(Role) : 런타임에 단독으로 드러나는 컴포넌트 단위(process, service, container). 실행 수준으로 책임을 나눈다 → Request-Response 관계

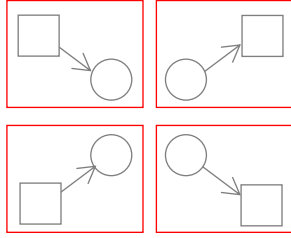
공통 효과 : 변경·결함·Test 의 범위를 '단위' 안으로 국소화. 대신 단위 수가 늘어 통합(조립) 복잡도가 증가

Quality Attribute	기능(Function) 분리 — 영향과 원인		역할(Role) 분리 — 영향과 원인	
Availability (가용성)	+	결함이 모듈에 국소화되어 원인 파악·복구가 빠름 (런타임 격리는 아님)	+	장애가 컴포넌트 경계에서 격리 → 개별 재시작·이중화 단위
Modifiability (변경 용이성)	+	변경 범위가 모듈 내부로 한정, 영향 분석 용이	+	컴포넌트 단위 독립 변경·교체, 팀별 병행 개발
Performance (성능)	0	같은 프로세스 내 호출이라 O/H 무시 가능	±	통신·직렬화 O/H 로 지연 ↑ / 독립 확장으로 처리량 ↑
Security (보안)	+	책임 분리로 권한 최소화·검토 범위 축소 (런타임 경계 없어 제한적)	+	경계에서 인증·접근 통제 가능 / 노출 인터페이스(공격 표면) 증가
Safety (안전성)	+	안전 관련 기능을 분리해 검증 범위 축소	+	안전 기능을 별도 런타임으로 격리, watchdog·감시자 배치 가능
Testability (시험 용이성)	+	모듈 단위 단위 Test 용이 (통합 Test 복잡도는 ↑)	±	컴포넌트 Test-mock 용이 / 통합 Test 는 환경·순서 의존으로 복잡
Deployability (배포 용이성)	+	모듈 교체 용이하나 배포 단위는 컴포넌트에 묶임	+	독립 배포·rollback 단위 확보 (Deployability 의 핵심)
Usability (사용성)	0	직접 영향 없음	0	직접 영향 없음 (통신 지연 시 응답성에 간접 영향)
Energy Eff. (에너지 효율)	0	추가 자원 소비 거의 없음	-	프로세스/컨테이너별 기본 자원 + 통신 비용
Integrability (통합 용이성)	+	명확한 인터페이스·책임 → 결합 지점이 분명	+	런타임 인터페이스(API) 표준화로 외부 통합 용이

원칙 2. 정보의 집중 / 분산 Centralization / Distribution of Information



정보 집중



정보 분산

집중 : 데이터를 하나의 저장소·소유자에 모은다 (중앙 DB, Shared Repository, Blackboard). 모든 Client 가 같은 곳을 본다

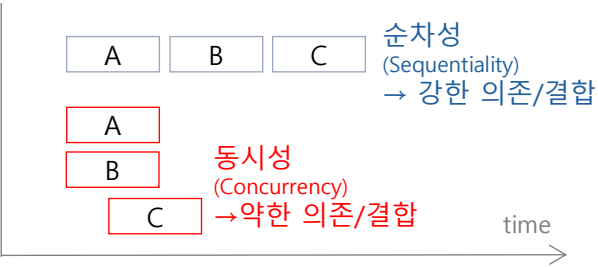
분산 : 데이터를 소유자별로 분할하거나 복제한다 (Service 별 DB, Cache, Edge, Replica). 동기화가 필요해진다

핵심 trade-off : 접근 용이성·일관성(Single Source of Truth) ↔ 단일 장애점(SPOF)·병목·공격 포인트 집중

Quality Attribute	정보 집중 — 영향과 원인	정보 분산 — 영향과 원인
Availability (가용성)	— 저장소가 단일 장애점(SPOF) → 이중화 없이는 전체 정지	+ 부분 장애 허용, 복제본으로 지속 운영
Modifiability (변경 용이성)	± Client 간 의존은 약함 / Data Schema 변경은 전체에 영향	+ 소유자별로 스키마·저장 방식 독립 변경
Performance (성능)	+ 접근 경로 단순, 조인·캐시·일관성 유지 비용 낮음 (병목 주의)	± 지역성(locality)·병렬 처리 ↑ / 조인·일관성 유지 통신 ↓
Security (보안)	— 명확한 공격 포인트, 침해 시 피해 범위 큼 (통제·감사는 용이)	± 피해 범위 분산 / 보호·감사해야 할 지점 다수
Safety (안전성)	± 단일 진실 원천으로 상태 불일치 방지 / 단일 장애가 전체 안전에 영향	± 장애 전파 억제 / 복제본 간 상태 불일치 위험
Testability (시험 용이성)	+ 상태 관찰·조작 지점이 하나 → 시나리오 재현 용이	— 분산된 상태 조합의 재현·검증 어려움
Deployability (배포 용이성)	— 스키마 변경·마이그레이션이 전체 배포와 결합	+ 소유자 단위 독립 마이그레이션·배포
Usability (사용성)	+ 어디서나 같은 정보, 일관된 사용 경험	— 동기화 지연(eventual consistency) 으로 불일치가 사용자에게 노출
Energy Eff. (에너지 효율)	+ 중복 저장·동기화 통신 없음	— 복제·동기화 통신, 중복 저장
Integrability (통합 용이성)	+ 공통 데이터 모델로 통합 (스키마 결합은 주의)	— 데이터 모델 정합성 유지 부담, 변환 계층 필요

원칙 3. 순차성 / 동시성

Sequentiality / Concurrency



순차 : 하나의 제어 흐름이 작업을 차례로 처리한다. 앞 작업이 끝나야 다음이 시작 → 결정적(deterministic), 강한 결합 경향

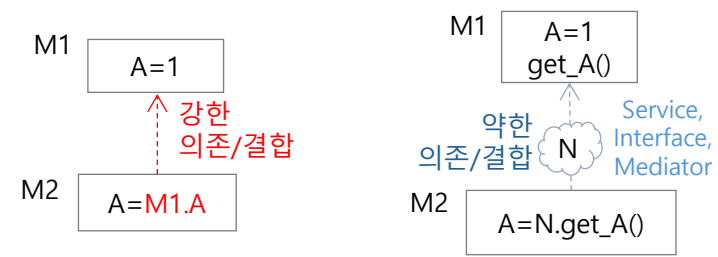
동시 : 여러 흐름(thread, process, async task)이 동시에 처리한다. 흐름 간 동기화가 필요 → 약한 결합 경향

핵심 trade-off : 처리량·응답성 ↔ 동기화·race condition·deadlock 등 복잡도 (Test·분석 난이도)

Quality Attribute	순차성 (Sequential) — 영향과 원인		동시성 (Concurrent) — 영향과 원인	
Availability (가용성)	0	구조 단순 / 하나가 막히면(blocking) 전체 정지	+	blocking 격리, 일부 장애에도 나머지 지속 (deadlock 주의)
Modifiability (변경 용이성)	0	제어 흐름 단순, 이해·변경 용이	-	동기화 코드 산재, 변경 영향 예측 어려움
Performance (성능)	-	자원 유휴, 대기 시간 누적	+	처리량·자원 활용 ↑ (Context Switching O/H 는 존재)
Security (보안)	+	race / TOCTOU 문제 없음	-	race condition 기반 취약점 발생 가능
Safety (안전성)	+	결정적 동작 → 분석·인증(certification) 용이	-	비결정성, race-deadlock 위험
Testability (시험 용이성)	+	재현성 높음	-	타이밍 의존 결함, 재현·검증 어려움
Deployability (배포 용이성)	0	직접 영향 없음	0	직접 영향 없음 (자원 설정에 의존)
Usability (사용성)	-	긴 작업 중 UI 응답 정지	+	비동기 처리로 응답성 유지
Energy Eff. (에너지 효율)	0	직접 영향 없음	±	빨리 끝내고 idle(+) / spin-switching O/H(-)
Integrability (통합 용이성)	0	동기 호출 인터페이스 단순	-	비동기 순서·재시도·시간 초과 등 인터페이스 복잡

원칙 4. 강한 / 약한 결합

Tight / Loose Coupling



강한 결합 : 직접 호출·공유 메모리·구체 타입 참조 등 상대의 구현을 알고 의존한다 ($A = M1.A$)

약한 결합 : Interface·Service·Mediator·Message 등 매개체를 통해 간접적으로 의존한다 ($A = N.get_A()$)

핵심 trade-off : 직접성(성능·단순함) ↔ 독립성(변경·교체·격리). 매개체 자체가 새로운 실패 지점·O/H 가 된다

Quality Attribute	강한 결합 (Tight) — 영향과 원인		약한 결합 (Loose) — 영향과 원인	
Availability (가용성)	—	장애·예외가 직접 전파	+	격리·대체 가능 (매개체가 SPOF 가 될 수 있음)
Modifiability (변경 용이성)	—	상대 구현 변경이 직접 영향	+	인터페이스만 유지하면 독립 변경
Performance (성능)	+	직접 호출, 최단 경로	—	간접 호출·직렬화·중재자 O/H
Security (보안)	—	침해가 그대로 전파, 경계 통제 없음	+	매개체에서 인증·인가·감사 집중
Safety (안전성)	—	오류 전파 차단 어려움	+	경계에서 격리·차단 (매개체의 실패 모드는 별도 고려)
Testability (시험 용이성)	±	통합 Test 단순 / 단위 Test 는 mock 어려움	±	단위 Test mock 용이 / 통합 Test 복잡
Deployability (배포 용이성)	—	함께 빌드·배포해야 함	+	독립 배포·교체
Usability (사용성)	0	직접 영향 없음	0	직접 영향 없음
Energy Eff. (에너지 효율)	+	통신 O/H 없음	—	간접 호출·직렬화 비용
Integrability (통합 용이성)	—	교체·외부 연동 어려움	+	표준 인터페이스로 상호운용(Interoperability)

설계 원칙이 Quality Attribute 에 미치는 영향

설계 원칙	선택	Av	Mo	Pe	Se	Sa	Te	De	Us	En	In	근거 / 주요 trade-off
1. 기능/역할의 분리	기능(Function) 분리	+	+	0	+	+	+	+	0	0	+	변경·단위 Test 범위를 모듈 단위로 국소화. 런타임 단위가 아니므로 가용성·성능 영향은 간접적(결함 국소화). 모듈 수 증가로 통합 Test 복잡도 ↑
	역할(Role) 분리	+	+	±	+	+	±	+	0	-	+	장애 격리·독립 배포·독립 확장의 단위(Av, De ↑). 대신 통신 O/H 로 지연 ↑(처리량은 ↑), 노출 인터페이스와 운영 자원(process, container) 증가
2. 정보의 집중/분산	정보 집중	-	±	+	-	±	+	-	+	+	+	접근 용이·일관성(Single Source of Truth). 단 단일 장애점(SPOF)·병목, 명확한 공격 포인트. Client 간 의존은 약하나 Data Schema 에 강하게 의존
	정보 분산	+	+	±	±	±	-	+	-	-	-	부분 장애 허용, 소유자별 독립 변경·배포. 대신 일관성 유지를 위한 동기화 통신 O/H, 분산 상태의 재현·Test 가 어렵고 보호 지점이 다수
3. 순차성/동시성	순차성 (Sequential)	0	0	-	+	+	+	0	-	0	0	결정적(deterministic) 동작으로 분석·재현 용이, race 없음. 대신 자원이 유휴 상태로 남고 하나가 막히면(blocking) 전체 응답성 저하
	동시성 (Concurrent)	+	-	+	-	-	-	0	+	±	-	처리량·응답성 개선, blocking 격리. 대신 동기화·race condition·deadlock 등 복잡도 ↑ → 수정·Test·안전 분석 난이도 ↑, Context Switching O/H
4. 강한/약한 결합	강한 결합 (Tight)	-	-	+	-	-	±	-	0	+	-	직접 호출로 빠르고 단순, 통합 Test 는 오히려 단순. 대신 변경·장애·예외·침해가 그대로 전파되고, 단위 Test(mock)·교체·독립 배포가 어려움
	약한 결합 (Loose)	+	+	-	+	+	±	+	0	-	+	Interface/Mediator 로 독립성·교체 가능성 확보, 접근 통제 지점 확보. 대신 간접 호출·직렬화 O/H, 매개체 자체가 SPOF·통합 Test 복잡도 요인

+ 유리 - 불리 ± 양면적(설계·구현에 따라 다름) 0 직접 영향 적음 ※ 값은 '경향'이며 Tactic 적용에 따라 완화·강화됨

Av=Availability · Mo=Modifiability · Pe=Performance · Se=Security · Sa=Safety · Te=Testability · De=Deployability · Us=Usability · En=Energy Efficiency · In=Integrability

Architectural Style = 설계 원칙의 조합

Style \ 원칙	기능 분리	역할 분리	순차성	동시성	정보 집중	정보 분산	약한 결합	강한 결합
Batch Sequence	●		●		●			●
Pipe-Filter	○	●		●		●	●	
Process Control	●			●				●
Shared Repository		●		●	●		●	
Blackboard		●		●	●		○	○
Buffered Message Based		●		●	○		●	
Non-Buffered Event Based	●	○	●		○			●
MVC	●				●		○	○
PAC	●			●		●	●	
Main-Subroutine	●		●		●			●
Master-Slave		●		●		●	○	
Layered	○	○	○	○	○	○	○	○
Virtual Machine		●		●	●		●	
Micro-Kernel	●	●		●	●		●	
Multi-Tier		●		●	●	○	●	
Client-Server		●		●	●		●	
Broker		●		●	○		●	
Dispatcher		●		●				●
Microservice		●		●		●	●	
Edge-Based		●		●		●	○	○

Style 은 원칙의 '조합'이다

- 각 Style 은 4가지 원칙에 대한 특정한 선택을 미리 묶어 놓은 것
- 따라서 Style 의 QA 특성은 적용된 원칙들의 QA 영향을 합성하면 대부분 설명된다
- 예: Microservice = 역할 분리 + 동시성 + 정보 분산 + 약한 결합 → Mo, De, Av ↑ / Te, Pe(자연), En ↓
- 예: Batch Sequence = 기능 분리 + 순차성 + 정보 집중 + 강한 결합 → 단순·재현 용이 / Av, Mo, Pe ↓

읽는 법

- ● 해당 원칙이 Style 의 핵심 특성
- ○ 부분적·선택적으로 적용되거나 구현에 따라 달라짐

설계에서의 활용

- 요구되는 QA 우선순위 → 유리한 원칙 선택 → 그 원칙 조합에 가까운 Style 후보 도출 → Tactic 으로 약점 보완

Quality Attributes (품질 속성)

Quality Attribute	정의
Availability (가용성)	임무(mission)가 요구되는 어느 시점에서든, 임무 시작 시 시스템이 정해진 동작 가능 상태에 있는 정도
Modifiability (변경 용이성)	시스템에 변경을 얼마나 쉽게 가할 수 있는가의 정도
Performance (성능)	이벤트가 발생했을 때 시스템이 특정 동작을 수행하는 데 걸리는 시간
Security (보안)	악의적인 행위의 가능성과 정보의 도난·유실 가능성을 줄이는 시스템의 능력
Testability (시험 용이성)	테스트를 통해 소프트웨어의 결함을 얼마나 쉽게 드러낼 수 있는가
Usability (사용성)	사용자가 원하는 작업을 얼마나 쉽게 완수할 수 있는가
Safety (안전성)	시스템이 사람·재산·환경에 피해를 주는 위험 상태(hazard)에 빠지지 않도록 하고, 빠졌을 때 피해를 억제·복구하는 능력
Deployability (배포 용이성)	실행 가능한 소프트웨어를 예측 가능한 시간과 노력으로 운영 환경에 배치(또는 rollback)할 수 있는 정도
Energy Efficiency (에너지 효율성)	요구된 기능을 수행하면서 에너지·계산 자원(CPU, memory, network)의 소비를 최소화하는 능력 — Resource Utilization
Integrability (통합 용이성)	별도로 개발된 컴포넌트를 결합하는 데 드는 비용과 위험의 정도. 외부 시스템과 정보를 교환·활용하는 능력(Interoperability)을 포함

Quality Attributes: Availability · Modifiability

■ Availability

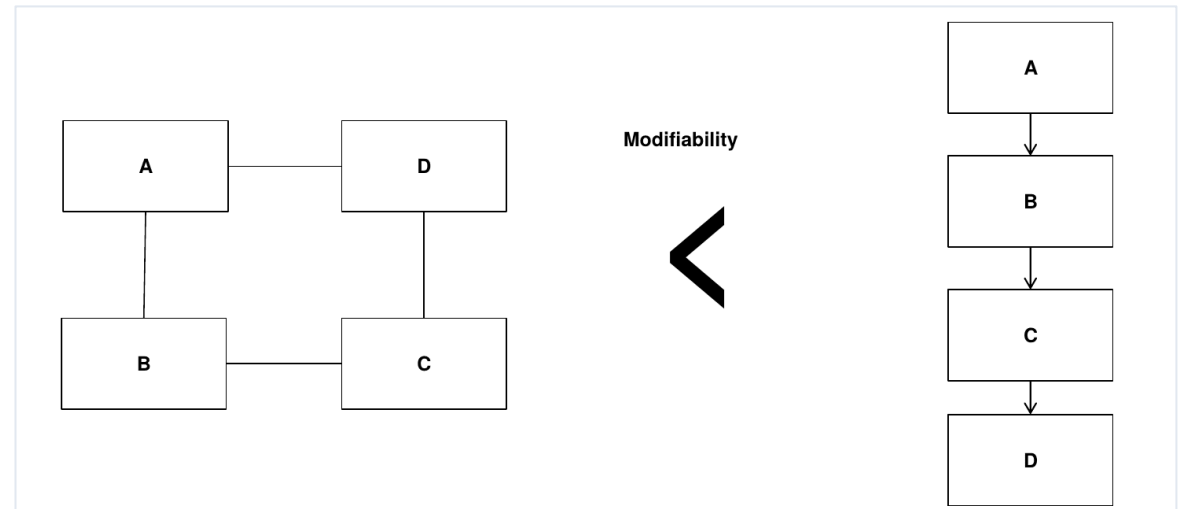
- 임무가 요구되는 어느 시점에서든 시스템이 정해진 동작 가능 상태에 있는 정도
- Server 하나보다 Main/Backup Server 를 두는 구조가 더 높은 Availability 를 가진다

■ Modifiability

- 시스템에 변경을 얼마나 쉽게 가할 수 있는가의 정도
- 요소가 서로 얽힌 구조보다 한 방향으로만 의존하는 구조가 변경하기 쉽다



Availability: 단일 Server < Main + Backup Server



Modifiability: 순환 의존 구조 < 단방향 의존 구조

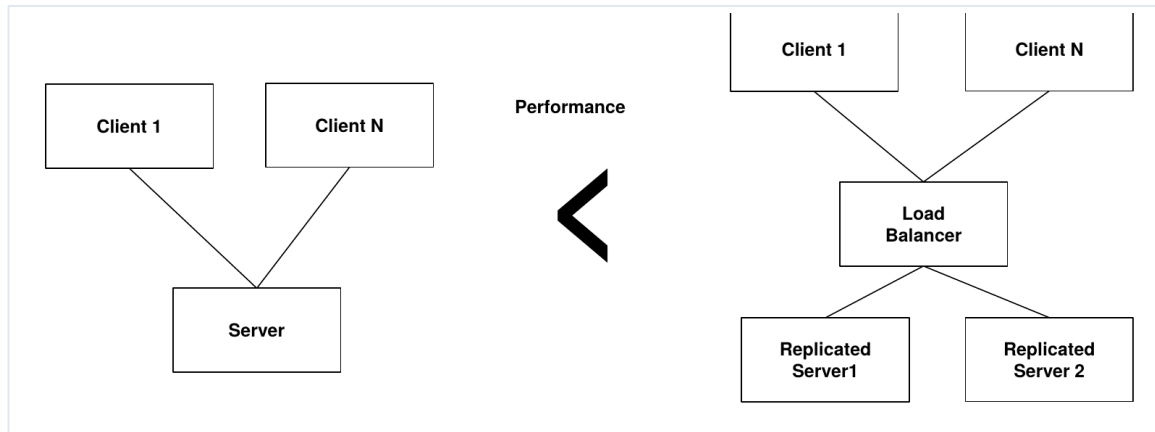
Quality Attributes: Performance · Security

■ Performance

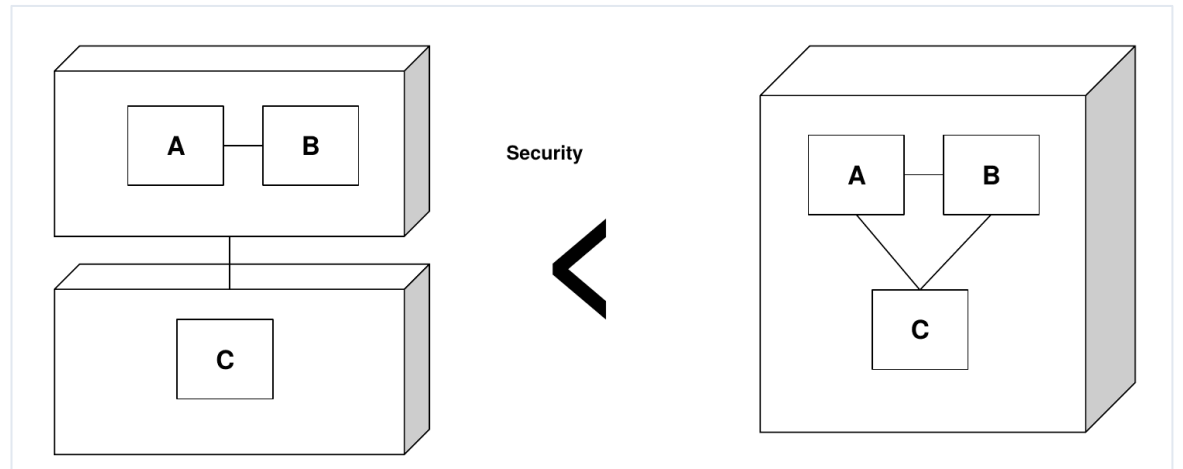
- 이벤트가 발생했을 때 시스템이 특정 동작을 수행하는 데 걸리는 시간
- Load Balancer 로 복제된 Server 에 요청을 나누면 단일 Server 보다 성능이 좋아진다

■ Security

- 악의적인 행위의 가능성과 정보의 도난·유실 가능성을 줄이는 시스템의 능력
- 요소를 분리된 경계에 두는 것보다 하나의 보호 경계 안에 두는 구조가 더 안전하다



Performance: 단일 Server < Load Balancer + Replicated Server



Security: 분리된 경계 < 단일 보호 경계

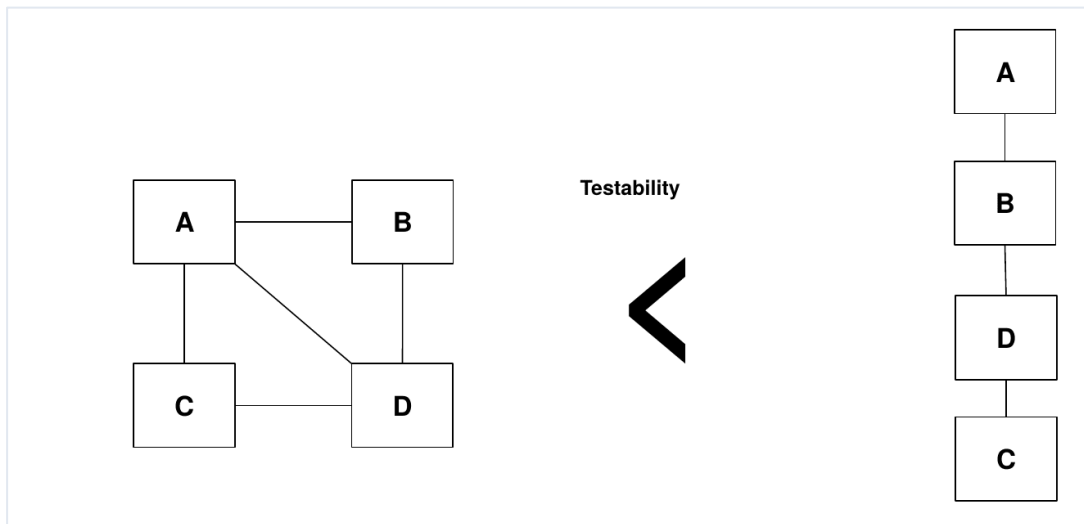
Quality Attributes: Testability · Usability

■ Testability

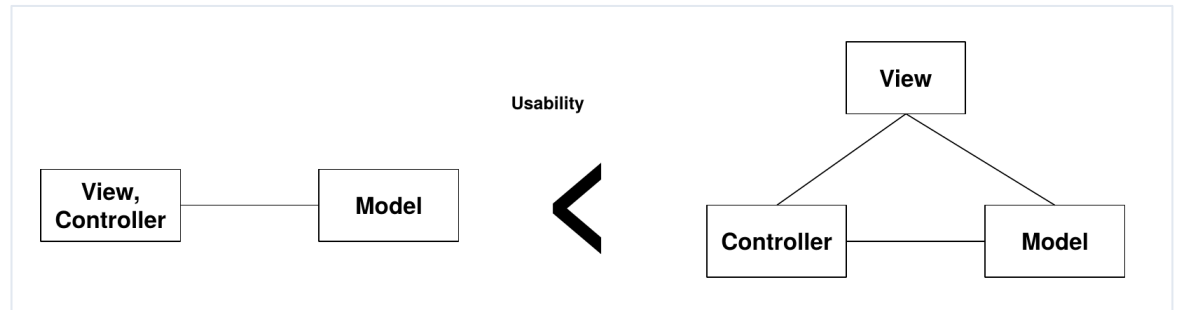
- 테스트를 통해 소프트웨어의 결함을 얼마나 쉽게 드러낼 수 있는가
- 요소가 서로 얽힌 구조보다 순차적으로 연결된 구조가 테스트하기 쉽다

■ Usability

- 사용자가 원하는 작업을 얼마나 쉽게 완수할 수 있는가
- View 와 Controller 를 분리하면 사용자 인터페이스를 다루기가 더 쉬워진다



Testability: 얽힌 구조 < 순차 구조



Usability: View+Controller 결합 < View / Controller / Model 분리

Architectural Style 카탈로그의 역사

■ Garlan 과 Shaw 의 카탈로그 (1995)

- Architectural Style 의 카탈로그를 처음 정리했다
- 이후 다른 연구자들이 이 목록을 확장했다

■ 카탈로그의 한계

- 완전하고 유일하며 서로 겹치지 않는 Style 목록은 존재하지 않는다
- 실제 시스템은 여러 Style 을 동시에 보인다
- 순수한(pure) Architecture 는 사소하지 않은 시스템에서는 거의 찾아보기 어렵다

분류	Style
Data flow	batch sequential, pipes and filters, process control
Call-return	main program-subroutine, object-oriented, component-based, peer-to-peer, service-oriented, N-tiered
Event-based	asynchronous messaging, publish-subscribe, implicit invocation, data-triggered
Data-centered	repository, blackboard

이 과정의 Architectural Style 카탈로그

■ 7개 category, 23개 style

- 각 style 은 configuration, example, strength, weakness 관점에서 설명한다

Category	Style	Sub-style
Data Flow	Batch Sequential / Pipe and Filter / Process Control	
Data Centered	Shared Repository / Blackboard	
Implicit Invocation	Event Based / Message Based	
Interaction-Oriented	Model-View-Controller / Presentation-Abstraction-Control	
Hierarchical	Main-Subroutine / Master-Slave / Layered	Virtual Machine / Microkernel
Distributed	Multi-tier / Broker / Service-Oriented	Client-Server / Dispatcher / Microservice
Deployment	Container-Based / Cloud-Based / Edge-Based	

Pattern이란 무엇인가?

■ Pattern의 정의

- Pattern은 반복되는 문제(recurring problem)에 대한 반복되는 해법(recurring solution)이다
- 해법의 틀(solution schema)은 다음으로 기술한다
 - (Structure) 구성 Component들과 그 책임, 그리고 관계
 - (Dynamics) Component들이 협력하는 방식

■ Pattern의 분류

- Architectural pattern: Architecture 수준
- Design pattern: Component 수준
- Idiom: 구현(implementation) 수준

Pattern 을 구성하는 것

■ Context (맥락)

- 설계 문제를 일으키는 설계 상황

■ Problem (문제)

- 그 맥락에서 반복적으로 나타나는 힘(forces)의 집합
- Forces: 요구사항, 제약, 바람직한 속성

■ Solution (해법)

- 힘들 사이의 균형을 잡는 구성(configuration)
- Component 와 관계로 이루어진 구조(Structure)
- 실행 시간 행위(Run-time behavior)

핵심 Pattern Name 아래에 Context – Problem – Solution 을 기술한다

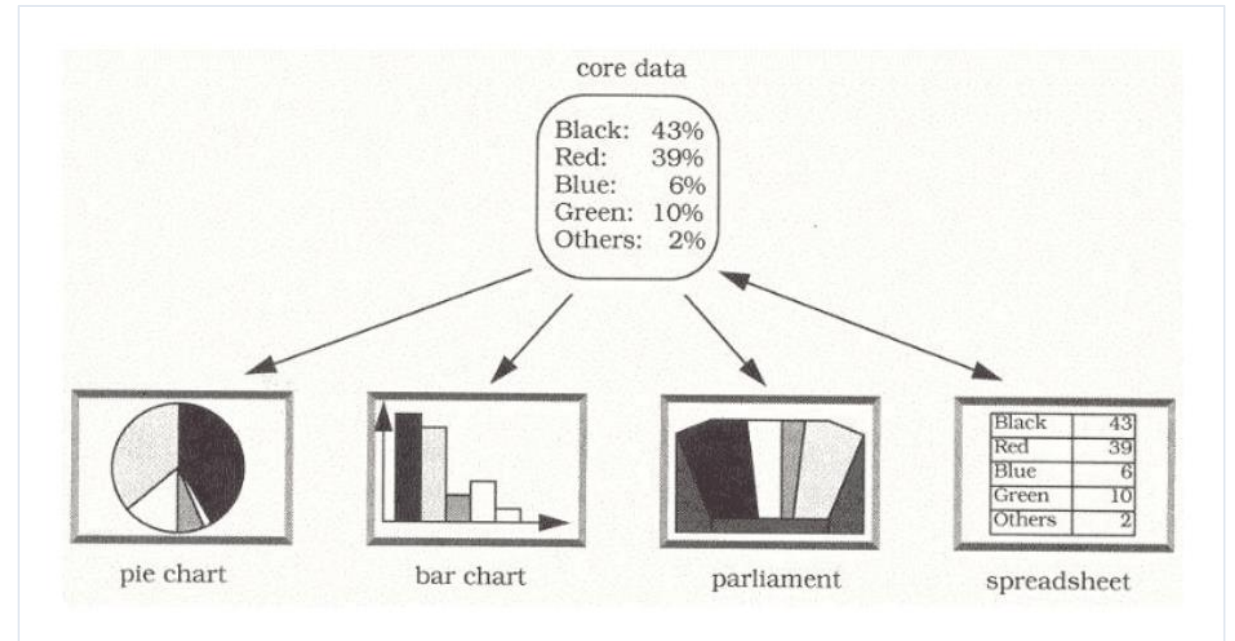
Pattern 의 예(MVC): Context 와 Problem

■ Context

- 오래 사용되는 대화형(interactive) 시스템의 사용자 인터페이스가 핵심 데이터(core data)를 여러 방식으로 보여 준다
 - 같은 데이터를 pie chart, bar chart, 표 등으로 표시

■ Problem

- 사용자 인터페이스 변경이 쉬워야 하고, 실행 중 (run time)에도 가능해야 한다
- 사용자 인터페이스를 수정하거나 이식(porting)해도 응용의 기능 핵심(functional core) 코드에 영향을 주면 안 된다

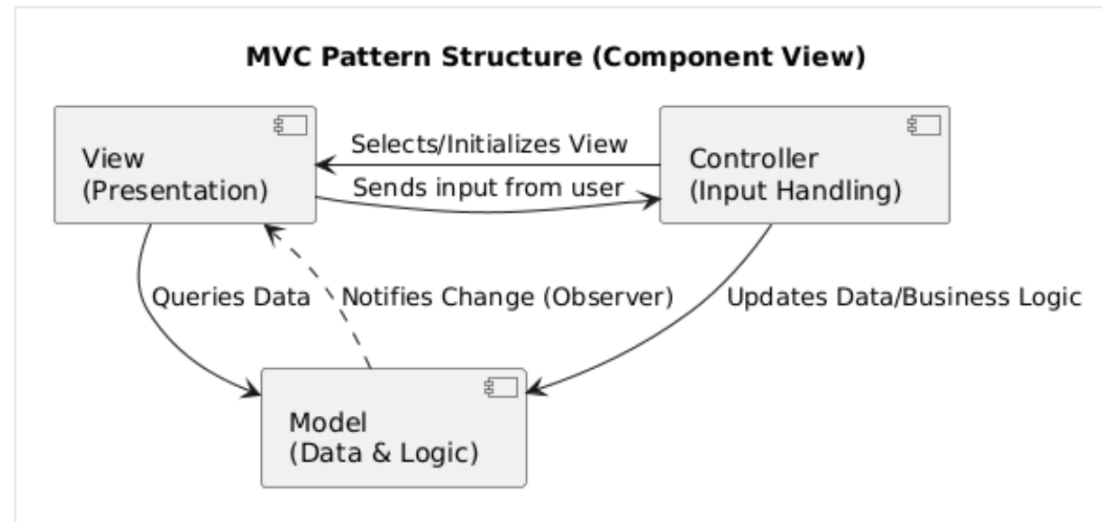


하나의 core data 를 여러 view 로 표시

Pattern 의 예(MVC): Solution – Structure

■ MVC 의 구조 (Component View)

- Model (Data & Logic): 데이터와 비즈니스 로직을 담당한다
- View (Presentation): Model 의 데이터를 조회(query)해 화면에 표시하고, Model 의 변경을 Observer 로 통보받는다
- Controller (Input Handling): 사용자 입력을 받아 Model 을 갱신하고 View 를 선택·초기화한다

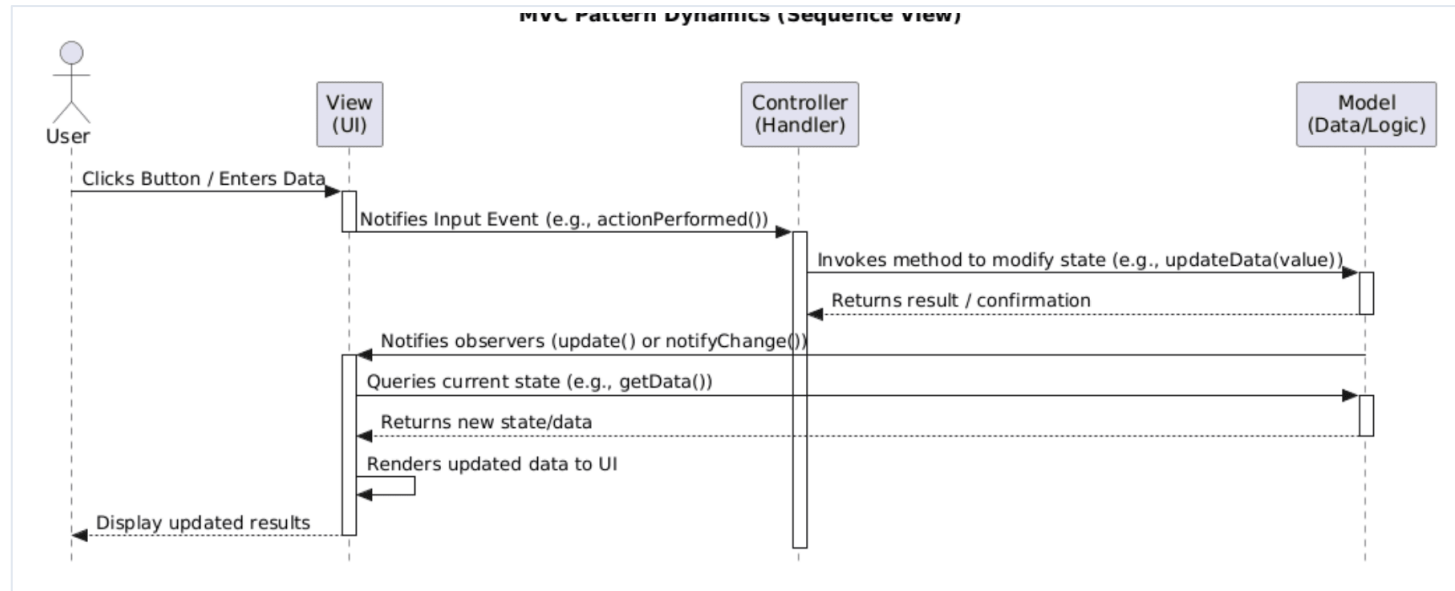


MVC Pattern Structure (Component View)

Pattern 의 예(MVC): Solution – Dynamics

■ 초기화 과정 (Sequence View)

- User 가 버튼을 누르거나 데이터를 입력하면 View 가 입력 이벤트를 Controller 에 알린다
- Controller 가 Model 의 상태를 바꾸는 메서드를 호출하고 결과를 돌려받는다
- Model 이 Observer(View)에 변경을 통보하면 View 가 현재 상태를 조회해 갱신된 데이터를 화면에 그린다



MVC Pattern Dynamics (Sequence View)

Architectural Pattern vs. Architectural Style

■ 초점의 차이

- Architectural Pattern 은 문제 지향적(problem-oriented)인 Architecture 해법에 초점을 둔다
- Architectural Style 은 관련된 Software Architecture 들의 구조적·행위적 특징에 초점을 둔다

■ 그러나 실제로는

- Architectural Style 의 개념은 Pattern 의 개념과 매우 유사하다
- 따라서 Architecture 분야에서는 "style" 과 "pattern" 이라는 용어를 같은 뜻으로 써도 된다

핵심 Pattern 은 '문제→해법'을, Style 은 '구조의 특징'을 강조하지만 사실상 같은 개념으로 쓰인다

Architecture 설계에 쓰이는 솔루션 용어의 구분

■ 모두 '특정 설계에 적용 가능한 솔루션'을 일컫는 용어 — 추상화 수준과 적용 범위가 다르다

1. Architectural Concept 아키텍처 개념

시스템 전체의 기본 형태에 대한 아이디어·설계 지침 (Guide). 이후 설계의 방향·형태·규모를 결정하며 시스템의 정체성과 주요 목적이 여기서 드러난다.

예 다수 사용자가 문서를 실시간으로 공유하고 동시에 편집하는 방식

2. Architectural Style 아키텍처 스타일

시스템 구조(형태)의 추상 모델 = 요소 유형·관계·제약의 집합. 구체화하여 전체 구조에 적용하며, 둘 이상을 혼합하는 경우가 많다.

예 Client-Server + Publish-Subscribe → Realtime Web

3. Reference Architecture 참조 아키텍처

특정 도메인에서 검증된 구조 템플릿. 도메인 어휘로 요소·책임·관계를 미리 정해 두고, 목적에 맞게 수정(tailoring)하여 사용한다.

예 실시간 협업 편집 RA(OT/CRDT 동기화 서버 + 클라이언트) — Google Docs-Figma 류가 이를 구현. 도메인 예: AUTOSAR, ROS 2, 3-Tier Web

4. Architectural Pattern 아키텍처 패턴

반복되는 설계 문제에 대한 검증된 해법을 문제-맥락-해결로 기술한 것. Style 보다 구체적·부분적이며, 여러 Tactic 을 묶어 놓은 형태가 많다.

예 Asynchronous Messaging, Data/Message Push, Circuit Breaker, CQRS

5. Architectural Tactic 아키텍처 태틱

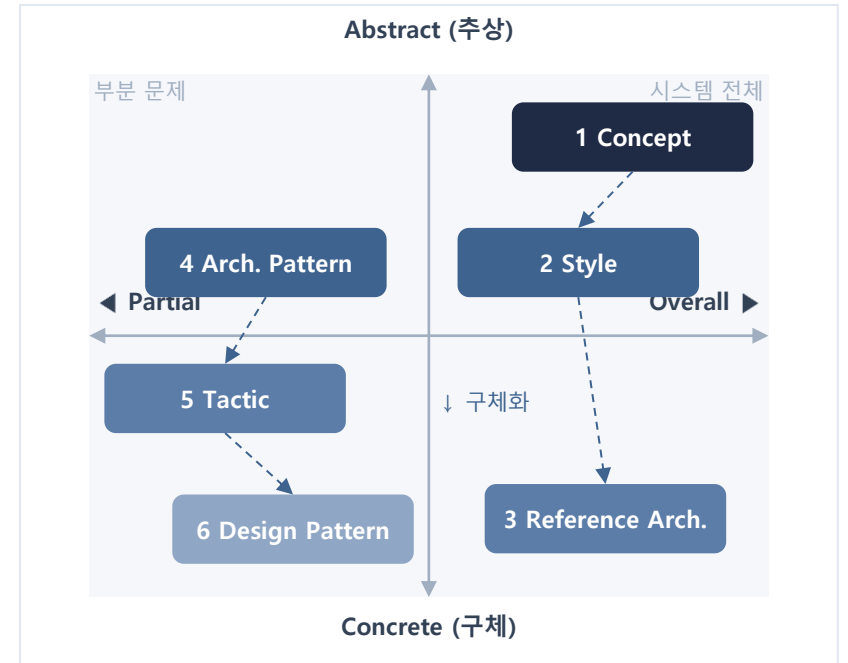
하나의 Quality Attribute 응답을 제어하기 위한 단일 설계 결정·지침. 지침에 맞추어 설계를 작성한다. Pattern 의 구성 요소.

예 편집 내용 동기화 0.1초 이내(Performance) → Scheduling Policy, Data Push / Ping-Echo, Redundancy(Availability)

6. Design Pattern 디자인 패턴

Class-Object 수준의 구조와 협력에 대한 해법(GoF). 컴포넌트 내부 구현에 적용되어 아키텍처 결정을 코드로 실현한다.

예 Observer(→ Pub-Sub 구현), Strategy, Factory, Proxy(→ Broker 구현)



Architectural Decision = 위 솔루션들의 선택·조합·구체화

- 자기 자신의 지식과 경험 활용
- 기존 Best Practice(Style, RA, Pattern, Tactic) 활용
- 다른 사람의 지식과 경험 활용 (Brainstorming, Review)

※ 이외 Deployment Pattern(배포 구조), Idiom(언어 수준) 등이 있다. Style 과 Architectural Pattern 은 문헌에 따라 같은 뜻으로 쓰이기도 한다 (SEI: Pattern = 문제·맥락·해결, Style = 요소·관계·제약)

1.2

Data Flow Architecture

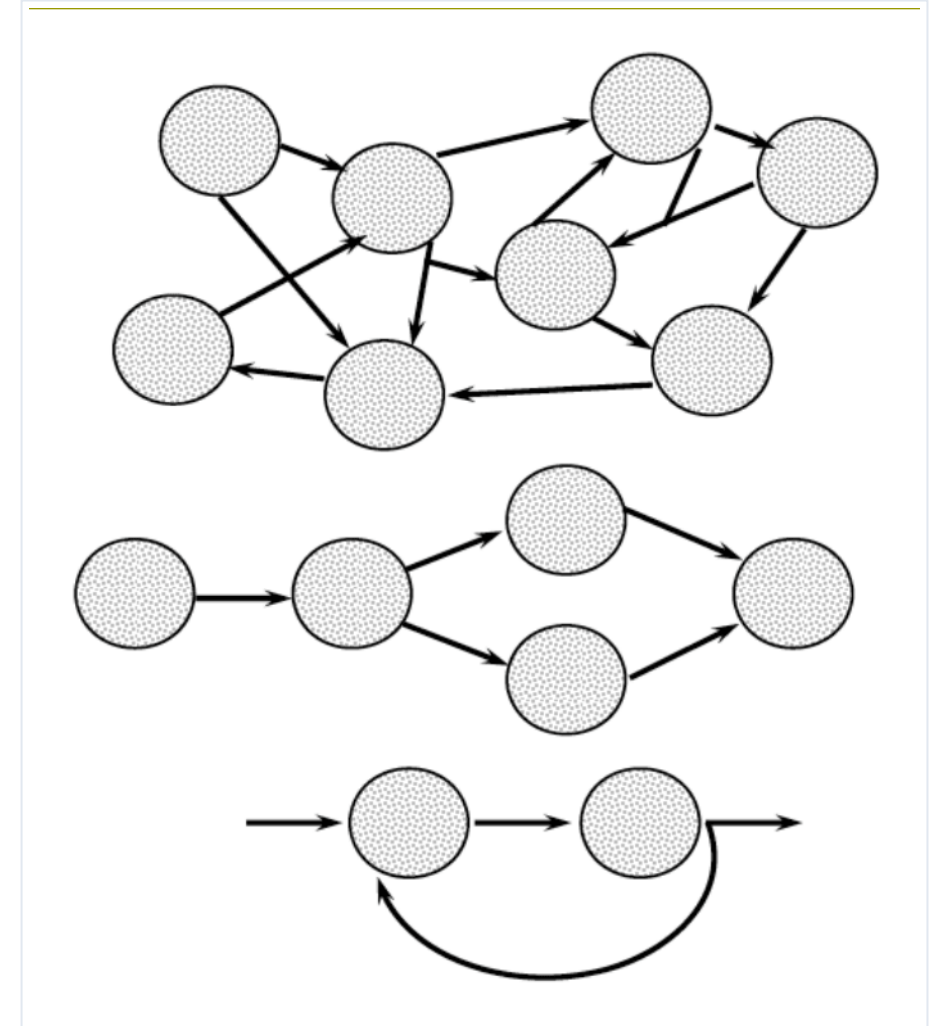
입력 데이터가 연속된 변환(transformation)을 거쳐 흘러가는 구조로 시스템을 구성하는 스타일

- Data Flow 시스템의 특징과 종류
- Batch Sequential Style
- Pipe and Filter Style
- Process Control Style
- Data Flow 스타일 비교

Data Flow 시스템의 종류

■ 데이터 흐름의 세 가지 형태

- 일반적으로 데이터는 임의의 pattern 으로 흐를 수 있다 (그림 위)
 - 그러나 임의의 구조는 이해와 분석이 어렵다
- 실제로는 거의 선형(nearly linear)인 Data Flow 시스템에 주로 관심을 둔다 (그림 가운데)
- 또는 매우 단순하고 제약이 강한 순환(cyclic) 구조를 다룬다 (그림 아래)
 - 예: 출력이 다시 입력으로 되돌아오는 제어 loop

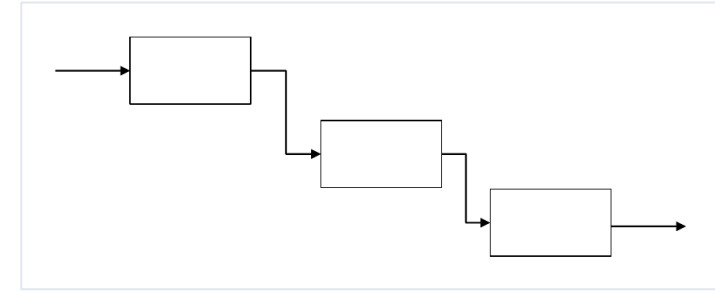


임의 흐름 / 거의 선형 흐름 / 단순 순환 구조

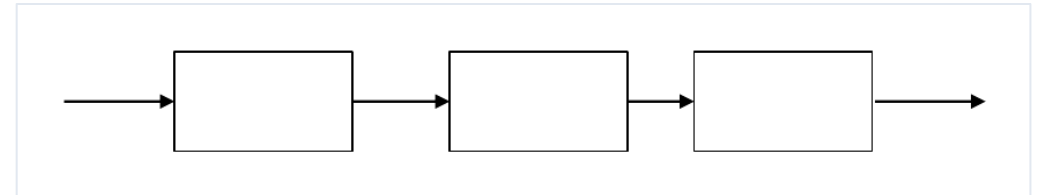
Data Flow Architecture 의 3가지 Style

■ 세 가지 Style

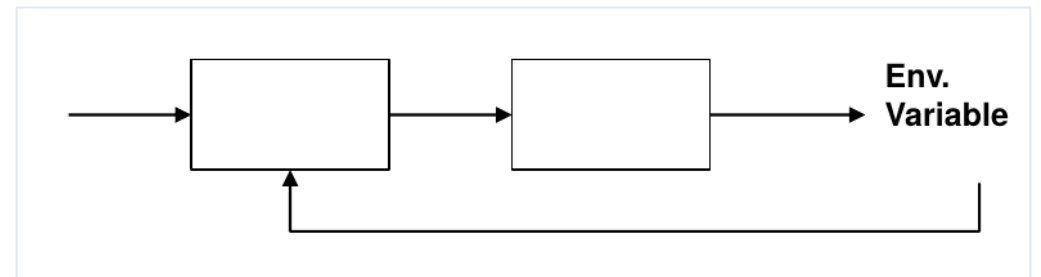
- Batch Sequential: 순차적인 처리 단계(step)로 구성 — 각 단계가 완전히 끝난(run to completion) 뒤 다음 단계로 넘어간다
- Pipe and Filter: 데이터 stream 을 점진적(incremental)으로 변환 — 앞 단계가 끝나기 전에 다음 단계가 처리를 시작할 수 있다
- Process Control: 환경 변수(environment variable)를 제어하기 위한 loop 구조 — 출력값을 다시 입력으로 되돌려 목표값을 유지한다



Batch Sequential



Pipe and Filter

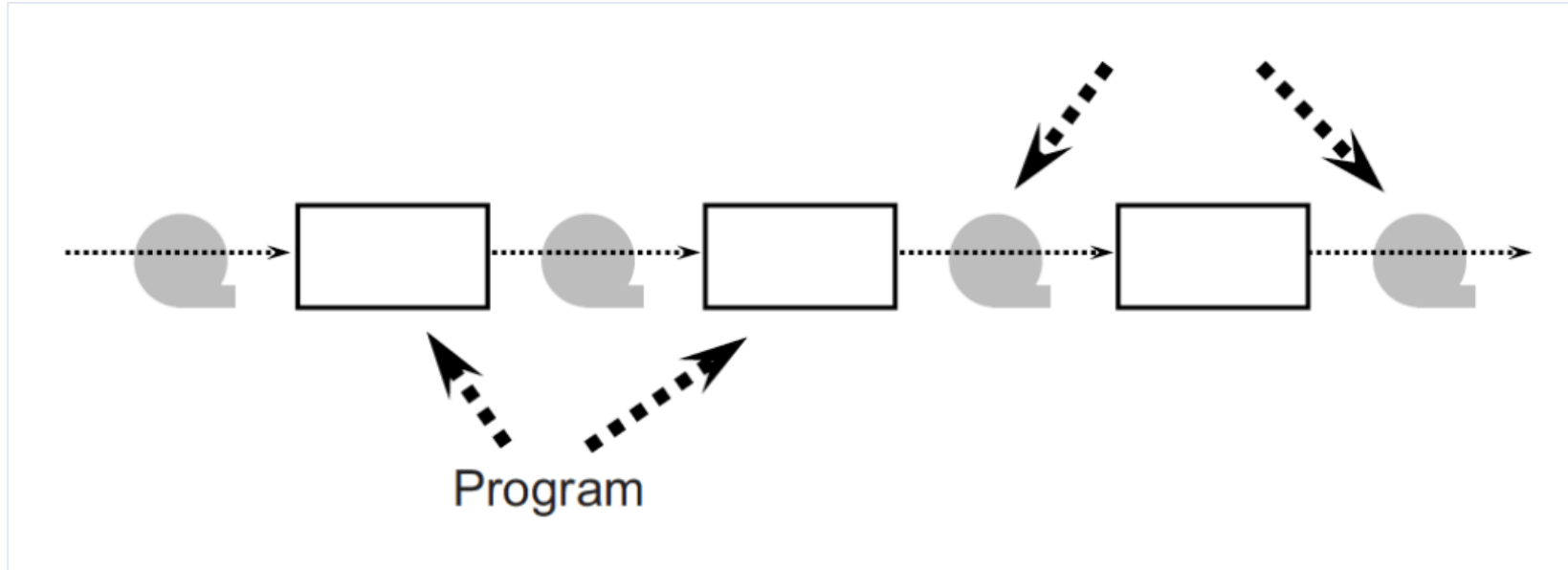


Process Control

Batch Sequential Style — 개요

■ 주요 특징

- 독립된 program 들이 정해진 순서대로 차례로 실행된다
- 데이터는 한 program 에서 다음 program 으로 덩어리(aggregate) 단위로 전달된다
 - 즉 앞 program 이 출력 전체를 만들어 낸 뒤에야 다음 program 이 그 데이터를 읽는다



Program 사이를 데이터 덩어리가 순서대로 이동한다

Batch Sequential Style — 구성 요소

■ Elements

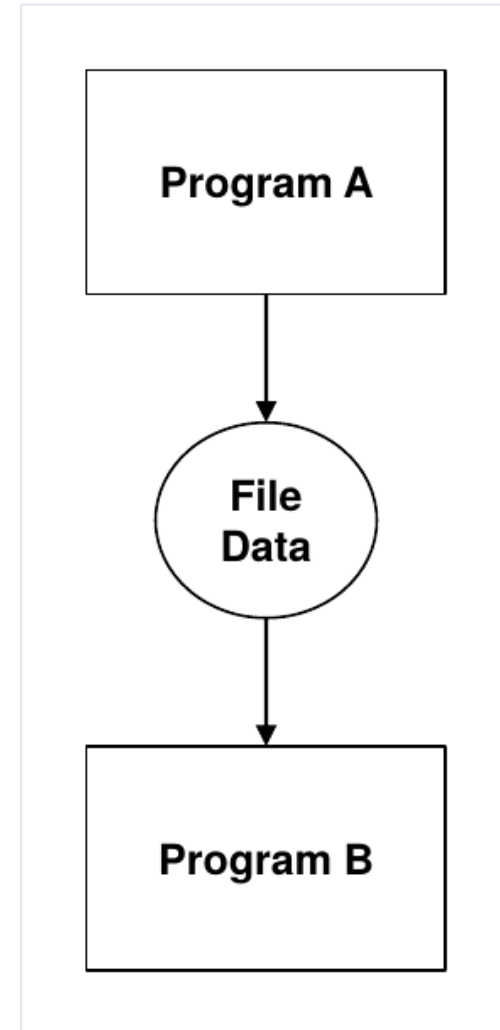
- Components: 독립적인 program 들
- Connectors: file
 - 데이터는 program 사이에서 하나의 덩어리로, 대개 file 형태로 전달된다

■ Relations

- 한 program 이 만든 file 이 다음 program 의 입력이 된다

■ Constraint

- 각 program 은 다음 program 이 시작하기 전에 완전히 끝나야 한다

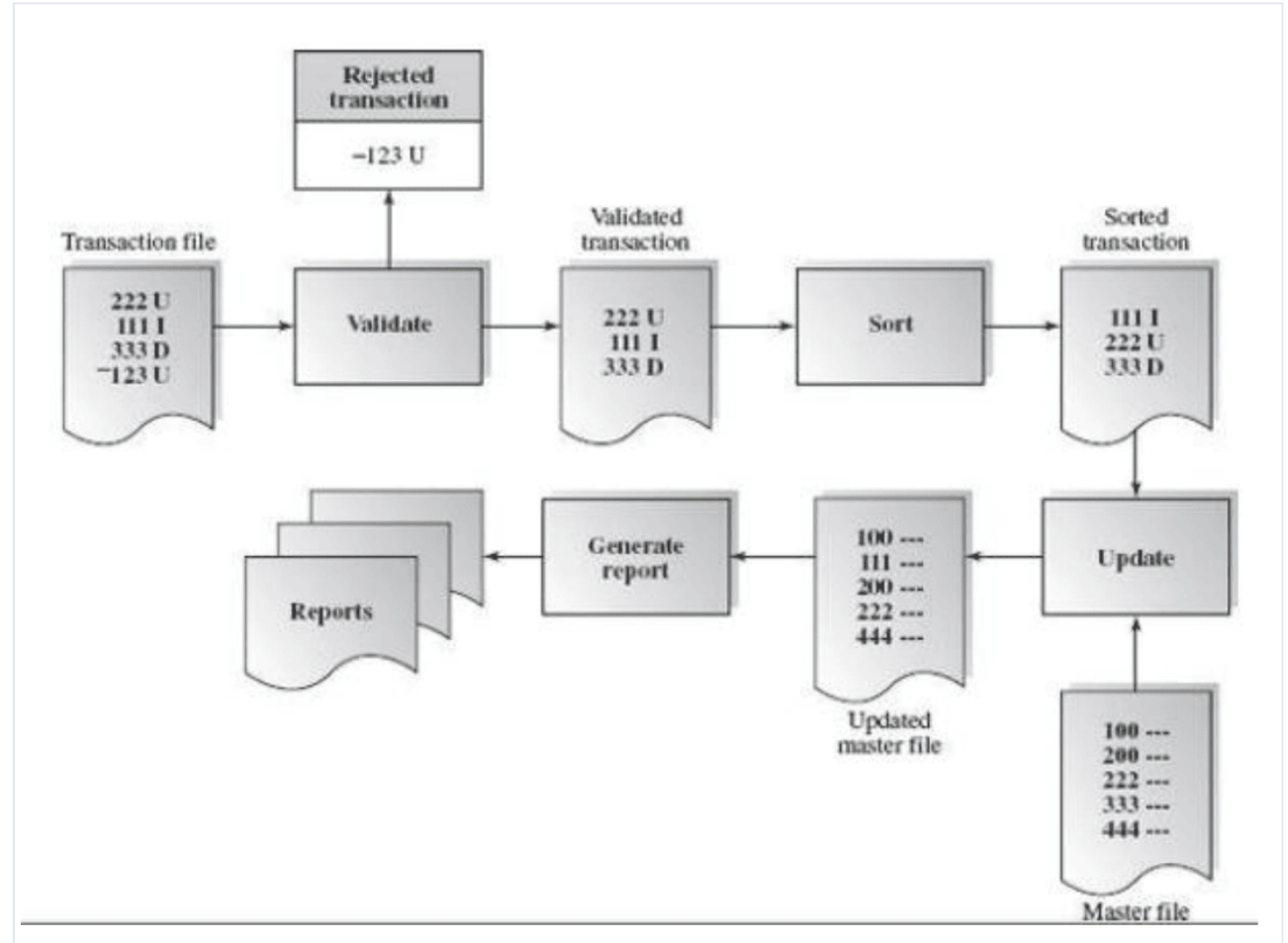


Program A → File Data → Program B

예제 – Information Processing Architecture

■ 전형적인 정보 처리 batch 구조

- Transaction file 을 Validate → Sort → Update 순으로 처리한다
- 각 단계는 file 을 입력받아 새 file(validated / sorted transaction)을 만든다
- Update 단계가 master file 을 갱신하고, Generate report 가 updated master file 로 보고서를 만든다
- 잘못된 transaction 은 rejected transaction file 로 따로 분리된다



Transaction file 기반 정보 처리 batch 흐름

예제 – Programming Level

■ 코드 수준에서 본 Batch Sequential

- main() 이 searching → counting → sorting 을 순서대로 호출한다
- 각 method 는 입력 file 을 읽어 출력 file 을 만들고, 그 file 이 다음 method 의 입력이 된다
 - inputFile → matchedFile → countedFile → reportFile 로 데이터가 file 단위로 전달된다

```
public class batch_sequential {  
    ...  
    public static void main() {  
        searching(kwd, inputFile, matchedFile);  
        counting(matchedFile, countedFile);  
        sorting (countedFile, reportFile);  
    }  
    public static void searching(String kwd, String inFile, String outFile) {...}  
  
    public static void counting(String inFile, String outFile)  
    {...}  
  
    public static void sorting(String inFile, String outFile)  
    {...} }
```

file 을 주고받는 세 단계의 순차 호출

예제 – Compiler Architecture

■ Compiler 를 Batch Sequential 로 보면

- Text → Lex(어휘 분석) → Syn(구문 분석) → Sem(의미 분석) → Opt(최적화) → Code(코드 생성) 단계가 순서대로 실행된다
- 각 단계의 출력 전체가 다음 단계의 입력이 된다

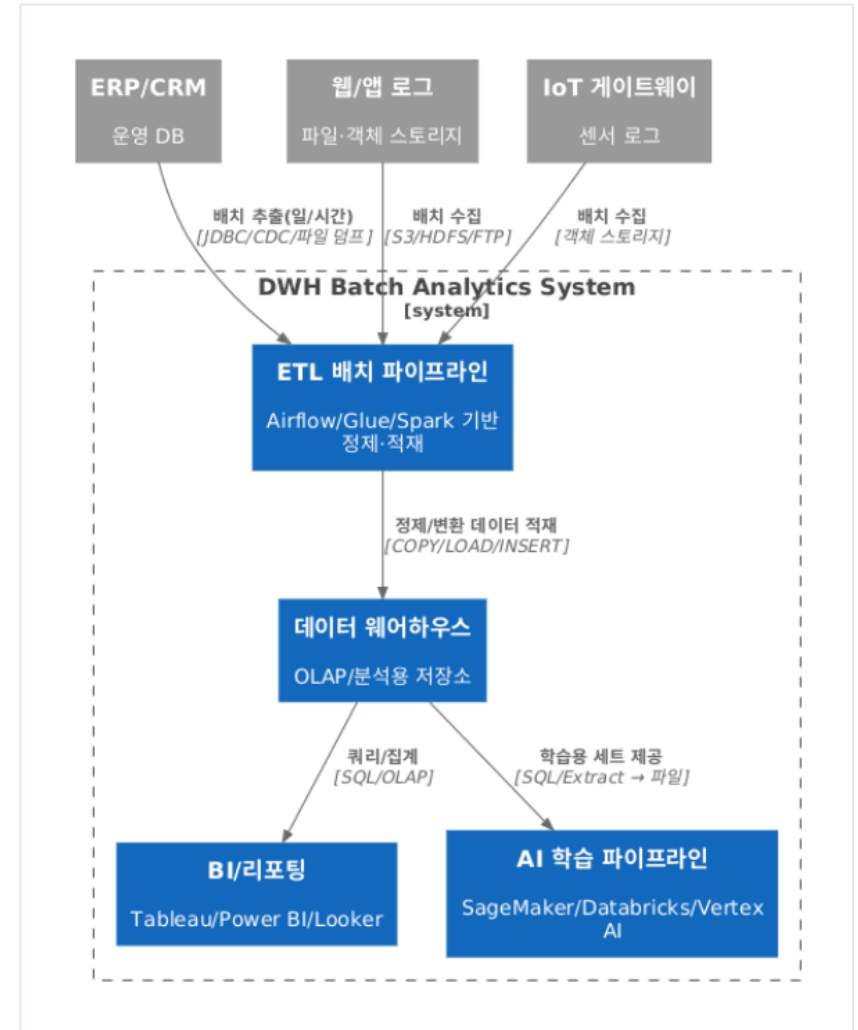


Compiler Architecture

예제 – 데이터웨어하우스 기반 배치 처리 Architecture

■ DWH Batch Analytics System

- ERP/CRM 운영 DB, 웹/앱 로그, IoT 게이트웨이 센서 로그가 배치 추출(시간)·배치 수집으로 들어온다
- ETL 배치 파이프라인(Airflow/Glue/Spark 기반)이 정제·적재하여 데이터 웨어하우스(OLAP/분석용 저장소)에 저장한다
- BI/리포팅(Tableau/Power BI/Looker)과 AI 학습 파이프라인(SageMaker/Databricks/Vertex AI)이 웨어하우스 데이터를 사용한다
- 각 단계가 끝나야 다음 단계가 시작되는 전형적인 Batch Sequential 구조



데이터웨어하우스 기반 배치 처리

Consequences: 장점과 약점

■ Benefits (장점)

- 구성이 단순하다
- 대규모 데이터 처리에 효율적이다
- 처리 단계(processing step)를 재사용할 수 있다
- 입출력 데이터 형식만 맞으면 처리 단계를 쉽게 교체할 수 있다

■ Weaknesses (약점)

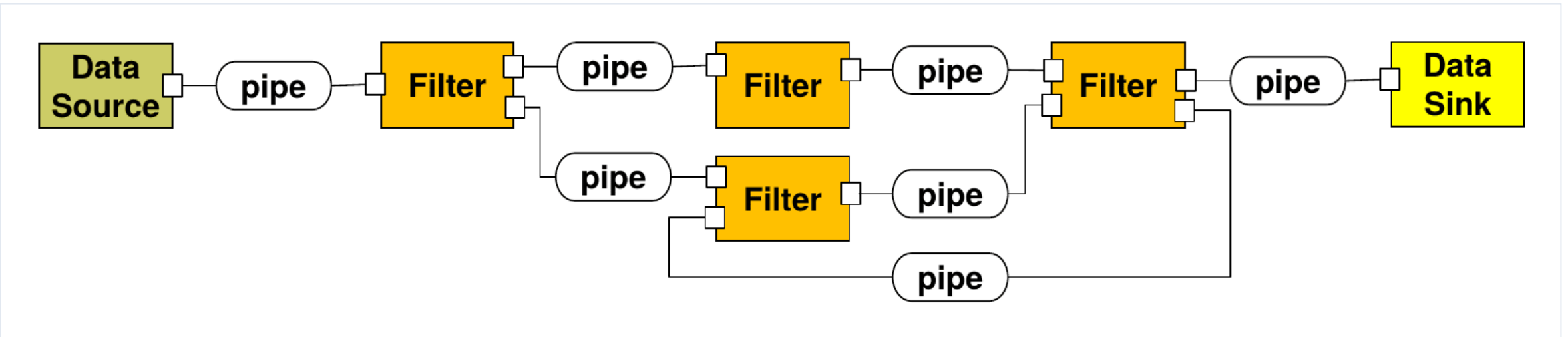
- 대화형(interactive) 처리가 불가능하다
- 동시성(concurrency)이 없다
- 지연(latency)이 높고 처리량(throughput)이 낮다
- 가용성(availability)이 제한된다

핵심 단계가 끝나야 다음 단계가 시작되므로 단순하지만, 동시성과 응답성은 포기해야 한다

Pipe and Filter Style — 개요

■ 주요 특징

- 동시적(concurrent)이고 점진적(incremental)인 실행이 핵심이다
 - Filter 는 입력 전체를 기다리지 않고 들어오는 데이터를 조금씩 변환해 내보낸다
- Loop(순환 구조)도 허용된다
- Data Source 에서 나온 데이터가 여러 pipe 와 Filter 를 거쳐 Data Sink 로 흘러간다



Data Source → pipe → Filter → ... → Data Sink (분기와 loop 가능)

Pipe and Filter Style — 구성 요소

■ Elements

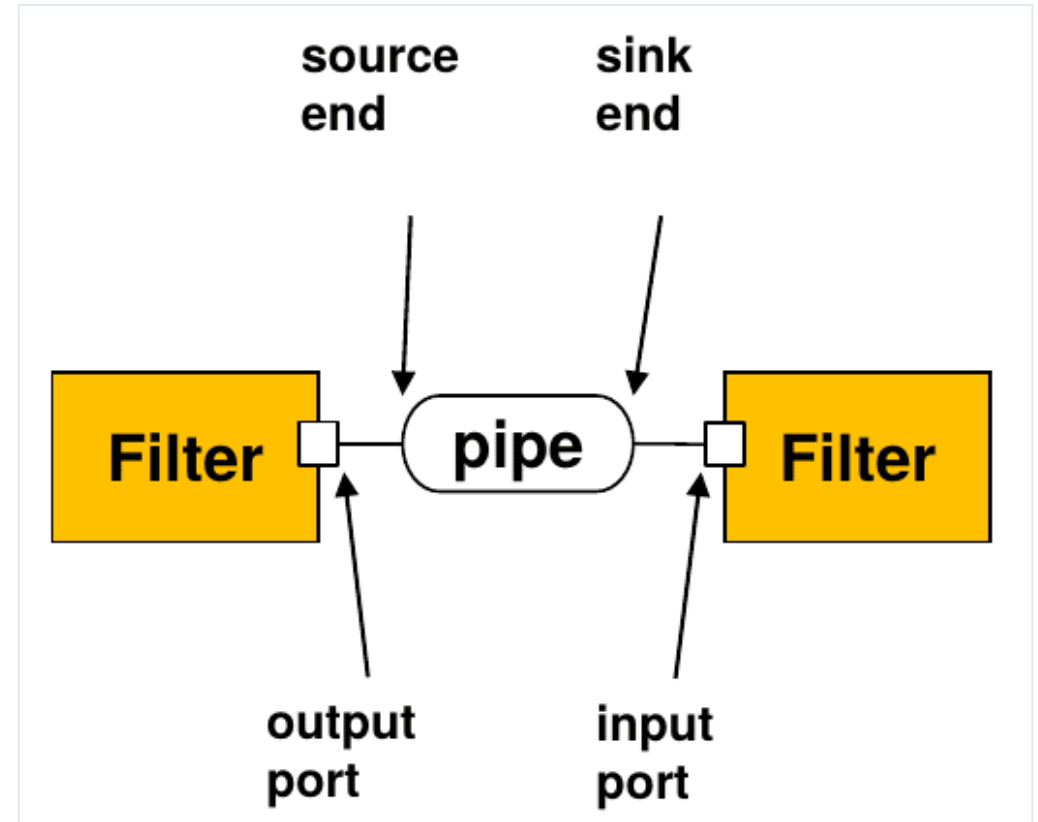
- Component: Filter — source 데이터의 일부를 점진적으로 sink 데이터로 변환한다
- Connector: Pipe — 단방향 흐름, 순서 보존(order-preserving), 데이터 보존(data-preserving)

■ Relations

- Pipe 의 source end 는 Filter 의 output port 에 연결된다
- Pipe 의 sink end 는 Filter 의 input port 에 연결된다

■ Constraints

- Filter 는 서로 독립적인 개체(independent entity)여야 한다



Filter 의 port 와 pipe 의 end 연결

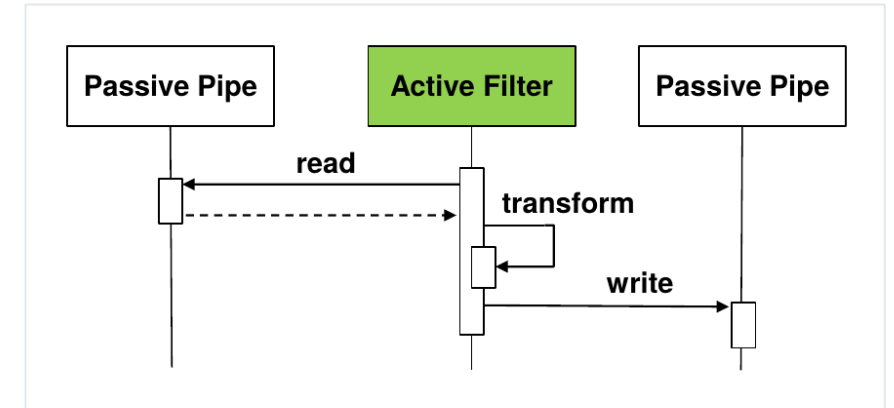
두 종류의 Filter: Active 와 Passive

■ Active Filter

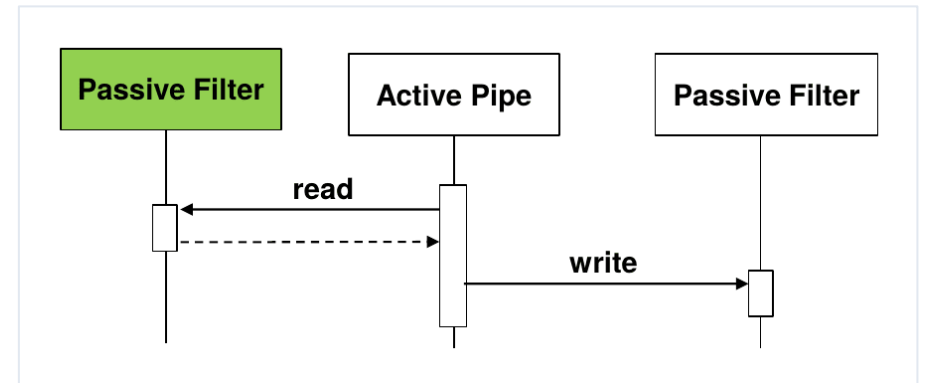
- 스스로 데이터를 끌어와(pull) 변환한 뒤 밀어낸다(push)
- read/write 기능을 제공하는 Passive Pipe 와 함께 동작한다

■ Passive Filter

- Active Pipe 와 함께 동작한다
 - Active Pipe 가 앞 Filter 에서 데이터를 꺼내(pull) 다음 Filter 로 밀어 넣는다(push)



Active Filter: Filter 가 read/transform/write 를 주도



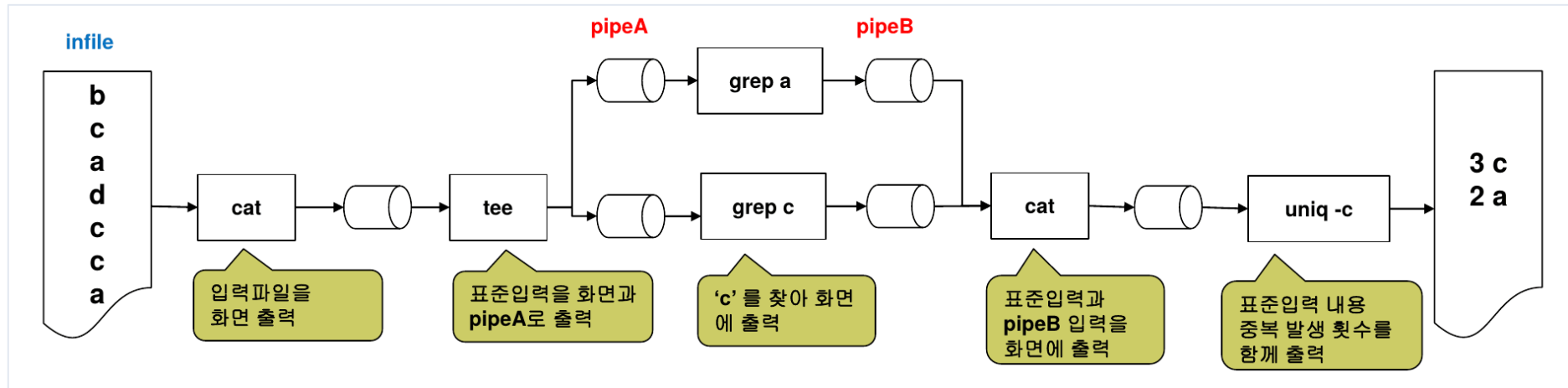
Passive Filter: Pipe 가 read/write 를 주도

핵심 누가 데이터 이동을 주도하는가(Filter 인가 Pipe 인가)에 따라 Active/Passive 가 나뉜다

예제 – Unix 의 Pipes and Filters

■ Named pipe 로 앞뒤 process 를 연결

- mkfifo 로 named pipe(pipeA, pipeB)를 만들면 앞뒤 process 사이에 데이터를 주고받을 수 있다
- cat, tee, grep, uniq 같은 각 명령이 Filter, 그 사이를 잇는 | 와 named pipe 가 Pipe 역할을 한다
 - 표준 입력을 읽어 변환한 뒤 표준 출력으로 내보내는 구조가 반복된다



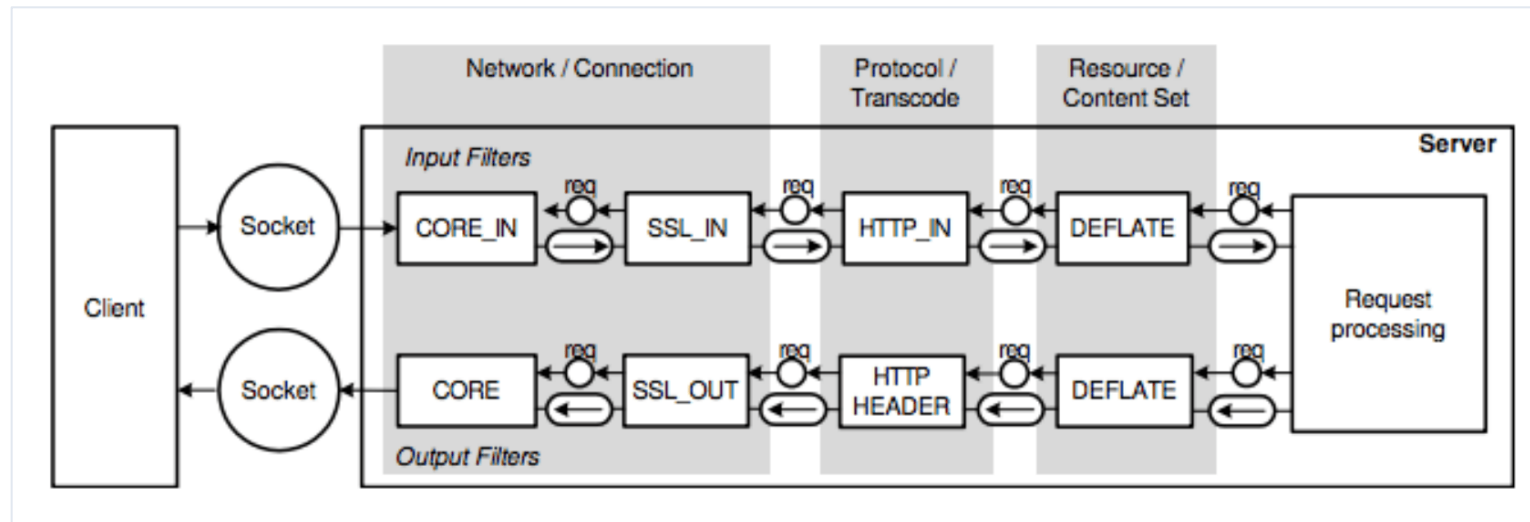
`$ cat infile | tee pipeA | grep c | cat - pipeB | uniq -c` (grep a < pipeA > pipeB 가 뒤에서 동작)

핵심 Unix 명령어 조합은 Pipe and Filter 스타일의 가장 대표적인 예이다

예제 – Apache Filters

■ Apache 웹 서버의 요청/응답 처리

- Client 요청은 Socket → Input Filters(CORE_IN → SSL_IN → HTTP_IN → DEFLATE) 를 거쳐 Request processing 에 도달한다
- 응답은 Output Filters(DEFLATE → HTTP_HEADER → SSL_OUT → CORE) 를 역순으로 거쳐 Socket 으로 나간다
- 각 Filter 는 Network/Connection, Protocol/Transcode, Resource/Content Set 계층에 속한다

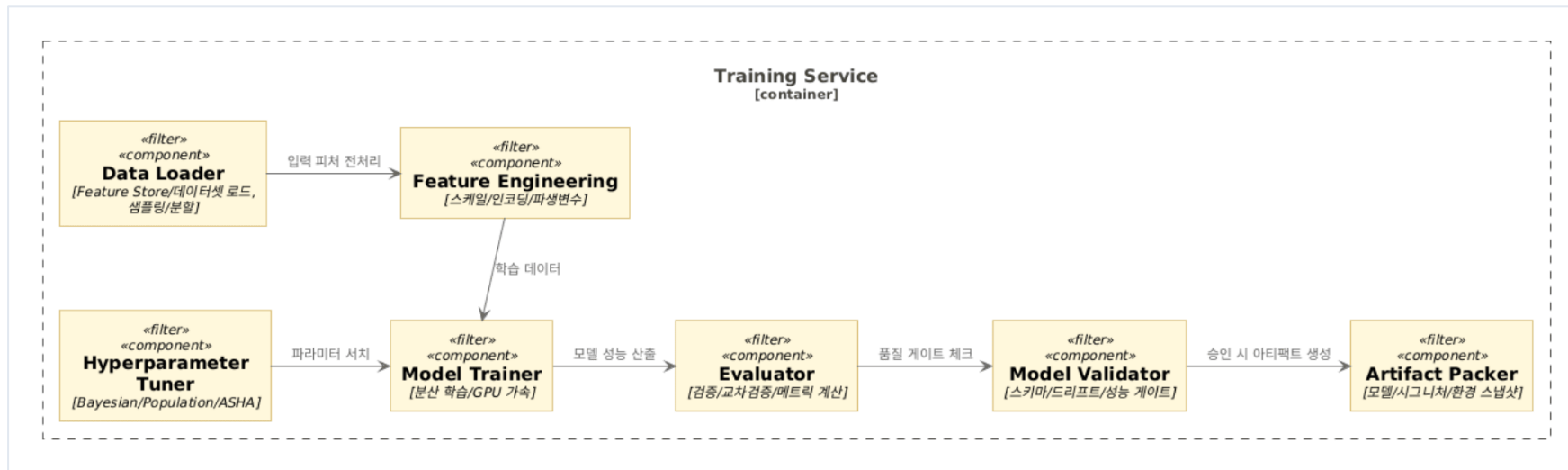


Apache Filter 구조 (출처: The Apache Modelling Project, fmc-modeling.org)

예제 – AI Training Pipeline Architecture

■ AI 학습 파이프라인

- Data Loader → Feature Engineering → Model Trainer → Evaluator → Model Validator → Artifact Packer 의 component 들이 순서대로 연결된다
- Hyperparameter Tuner 가 Model Trainer 에 튜닝 결과를 전달한다
- 각 component 가 Filter, component 사이의 데이터 전달이 Pipe 에 해당한다



Training Service 내부의 Pipe and Filter 구조

Consequences: 장점과 약점

■ Benefits (장점)

- 구성이 단순하다
- Filter 를 재사용할 수 있다
- 시스템을 유지보수하고 확장하기 쉽다
- 독립적인 Filter 를 분산·병렬 처리해 성능을 높일 수 있다

■ Weaknesses (약점)

- 대화형(interactive) I/O 처리에 적합하지 않다
- 독립 Filter 가 많아지면 계산 overhead 가 크게 늘어난다
- 오래 실행되는 계산(long-running computation)에는 적합하지 않을 수 있다
- fault tolerance 가 낮다

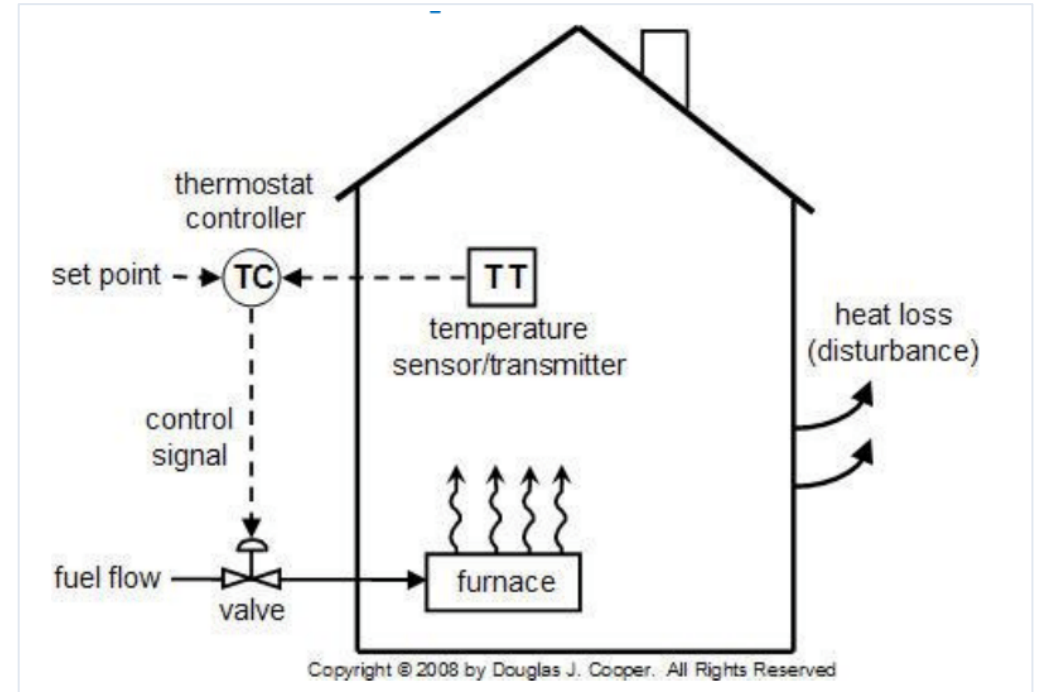
Process Control Style — 개요

■ 정의

- Process Control Architecture 는 Data Flow Architecture 의 한 종류
- 일련의 variable 로부터 데이터가 흘러나오고, 이 variable 들이 process 의 실행을 제어한다
- 목적은 원하는 process 결과(desired outcome)를 유지하는 것

■ 예: 난방 온도 제어

- set point(목표 온도)와 센서(TT)의 측정값을 비교해 thermostat controller(TC)가 valve 를 조절한다
- 외부 열 손실(disturbance)이 있어도 실내 온도를 목표값에 맞춘다



난방 시스템의 제어 loop

Process Control Style — 구성 요소

■ Elements

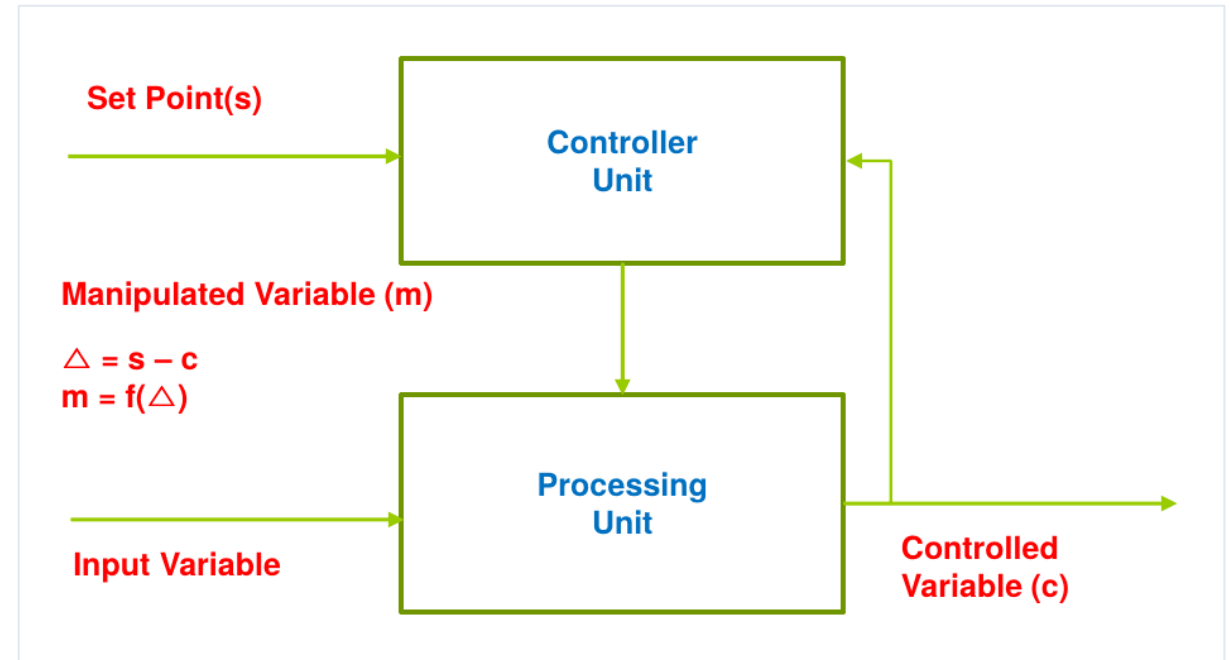
- Components: Control Unit, Processing Unit
- Connectors: variable 의 집합 — Set Point, Manipulated Variable, Input Variable, Controlled Variable

■ Relations

- Controller 가 Set Point(s)와 Controlled Variable(c)의 차이 $\Delta = s - c$ 로 Manipulated Variable $m = f(\Delta)$ 을 계산해 Processing Unit 에 보낸다
- Processing Unit 의 출력인 Controlled Variable 이 다시 Controller 로 되돌아온다

■ Constraints

- Controlled Variable 의 값이 Set Point 값에 최대한 가깝게 유지되어야 한다

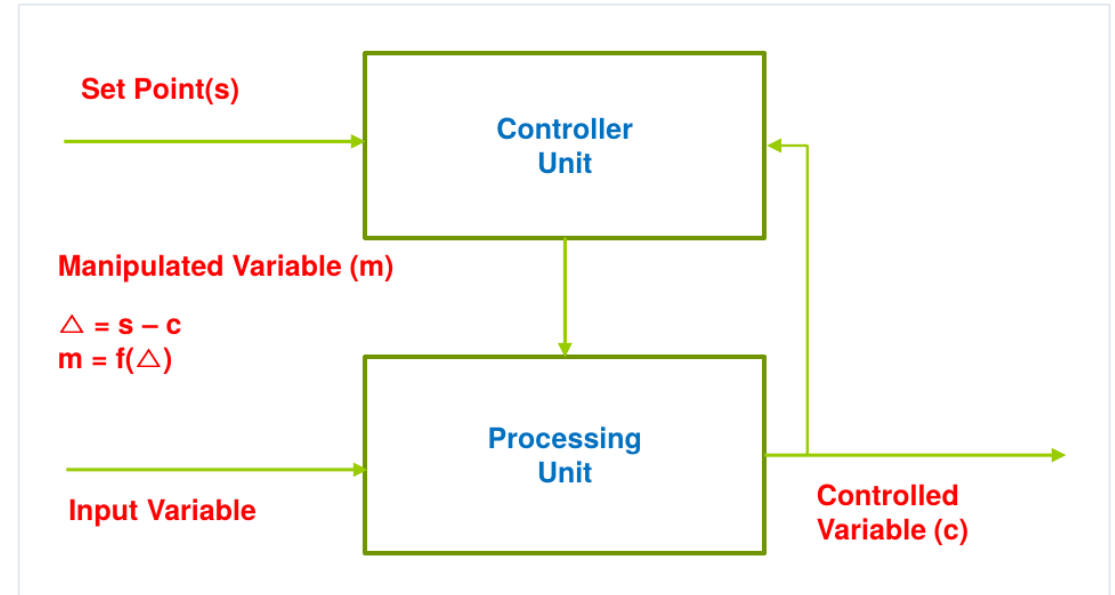


Controller Unit 과 Processing Unit 사이의 변수 흐름

Process Control Style — 주요 변수

■ 네 가지 variable

- Set Point: Controlled Variable 이 도달해야 하는 목표값 (desired value)
- Manipulated Variable: Controller 가 조정하거나 변경하는 값
- Input Variable: Processing Unit 으로 들어가는 입력 데이터
- Controlled Variable: 시스템이 만들어 내는 값으로, 센서로 측정해야 한다



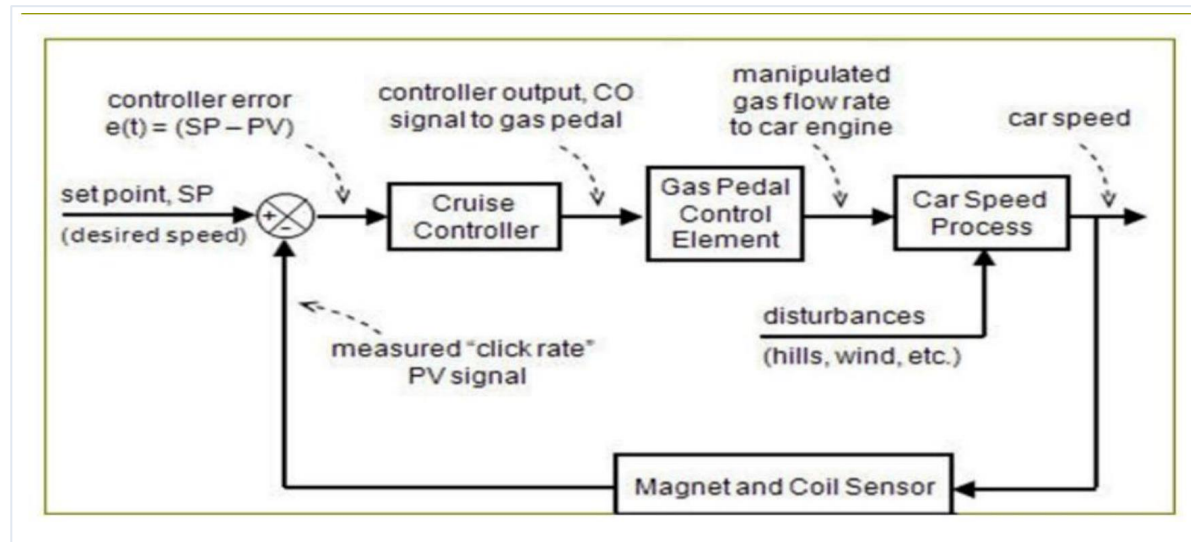
변수들이 Controller 와 Processing Unit 을 잇는 connector 역할을 한다

핵심 Controller 는 Set Point 와 Controlled Variable 의 차이를 줄이도록 Manipulated Variable 을 조정한다

적용 분야와 예제 – Cruise Control System

■ Application Area 와 Cruise Control 예제

- 출력 데이터를 안정된 수준으로 유지해야 하는 시스템에 적합 — 대표 예: Cruise Control System, QoS Control System
- Cruise Control: set point(목표 속도)와 센서가 측정한 현재 속도의 차이(error)를 Cruise Controller 가 계산한다
- Controller 가 Gas Pedal(manipulated variable)을 조절해 Car Speed Process 를 제어하고, 언덕·바람 같은 disturbance 에도 속도를 유지한다

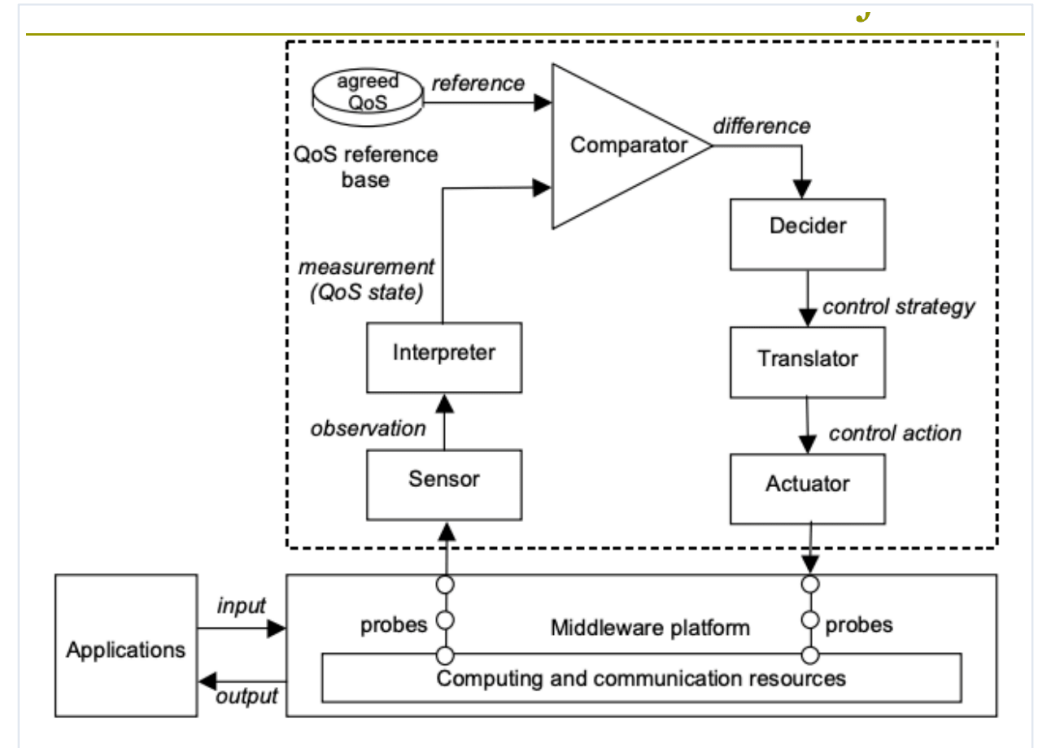


Cruise Control 의 closed loop (출처: controlguru.com)

예제 – Object Middleware 의 QoS 제어 Architecture

■ QoS Control 의 제어 loop

- Sensor 가 middleware platform 의 상태를 관찰 (observation)해 QoS state 를 측정한다
- Interpreter 가 측정값을 해석하고, Comparator 가 agreed QoS(reference)와 비교해 difference 를 구한다
- Decider 가 control strategy 를 정하고, Translator 가 이를 control action 으로 바꿔 Actuator 가 platform 에 적용한다
- Applications 의 input/output 은 그대로 두고, Computing and communication resources 만 조정해 QoS 를 유지한다



QoS-Control Architecture for Object Middleware

Consequences: 장점과 약점

■ Benefits (장점)

- 최적의 process 조건을 계속 유지할 수 있다
- 처리의 신뢰성(reliability)과 견고성(robustness)을 높인다

■ Weaknesses (약점)

- 설계가 복잡하다
- Processing Unit 이나 Control Unit 의 고장, 오동작, 사이버 공격이 제어 시스템 전체를 무너뜨릴 수 있다

Data Flow 스타일 비교: 언제 무엇을 쓰나

■ Batch Sequential / Pipeline

- 문제를 순차적인 단계(sequential stage)로 나눌 수 있을 때
- 앞 단계가 끝나기 전에 뒤 단계가 시작할 수 있을 만큼 각 단계가 점진적(incremental)이라면 pipeline 을 고려한다

■ Pipe and Filter

- 연속적인 데이터 stream 을 변환하는 문제일 때
- Filter 를 조합해 stream 을 단계별로 점진 처리한다

■ Process Control

- 예측하기 어려운 외부 교란(disturbance)에 대응하는 제어 동작이 필요할 때
- closed loop 제어 구조를 고려한다

핵심 문제의 성격(순차 단계 / 연속 stream / 외부 교란 제어)에 따라 스타일을 고른다

1.3

Data Centered Architecture

모든 component 가 공유하는 중앙 데이터 저장소를 중심으로 시스템을 구성하는 스타일

- Data-Centered Architecture 의 특징과 구조
- Repository Style
- Blackboard Style

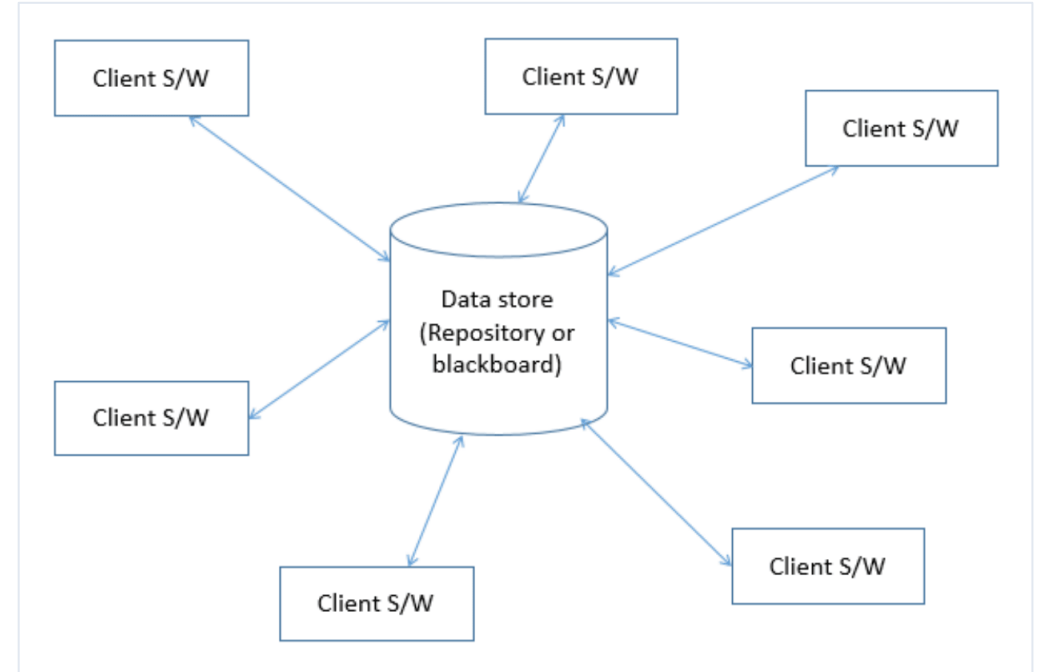
Data-Centered Architecture 개요

■ 특징

- 주변의 모든 software component 가 공유하는 중앙 데이터 저장소(centralized data store)가 핵심이다

■ Structure

- 시스템을 두 부분으로 나눈다
 - data store
 - data client: 독립적인 software component 또는 agent
- data client 사이의 상호작용·통신은 오직 data store 를 통해서만 이루어진다
 - 데이터가 client 들 사이의 유일한 통신 수단이다



Data store(Repository 또는 Blackboard)를 둘러싼 Client S/W 들

핵심 Client 끼리는 직접 대화하지 않고, 오직 공유 데이터로만 소통한다

Data-Centered Architecture 의 두 가지 Style

■ Repository Style

- data store 는 수동적(passive), client 는 능동적(active)
 - client 가 필요할 때 data store 에 접근해 읽고 쓴다
- 예: DBMS, Information System

■ Blackboard Style

- data store 는 능동적(active), client 는 수동적(passive)
 - data store 의 변화가 client 의 처리를 촉발한다
- Blackboard 의 client 를 knowledge source, listener, subscriber 라고 부른다
- 예: 지식 기반 AI 시스템, 음성·영상 인식 시스템

핵심 누가 능동적인가 — data store 인가 client 인가 — 가 두 스타일을 가르는 기준이다

Repository Style — 구성 요소

■ Elements

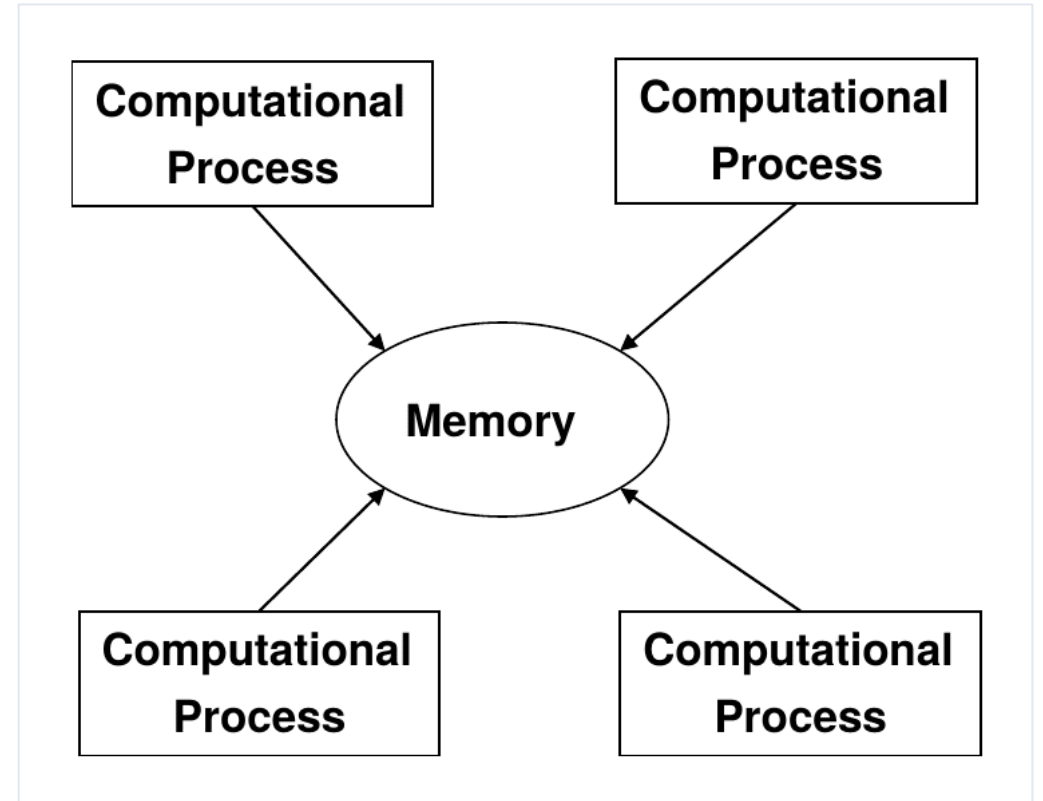
- Components: 하나의 memory(data store)와 여러 computational process(data client)
- Connectors: computational unit 과 memory 사이의 직접 데이터 접근(data access) 또는 procedure call

■ Relations

- 여러 computational process 가 하나의 memory 에 접근한다 (그림 참조)

■ Constraints

- Memory(data store)는 수동적으로 활성화된다 — client 의 요청이 있어야 동작한다
- data client 사이의 직접 상호작용은 없다

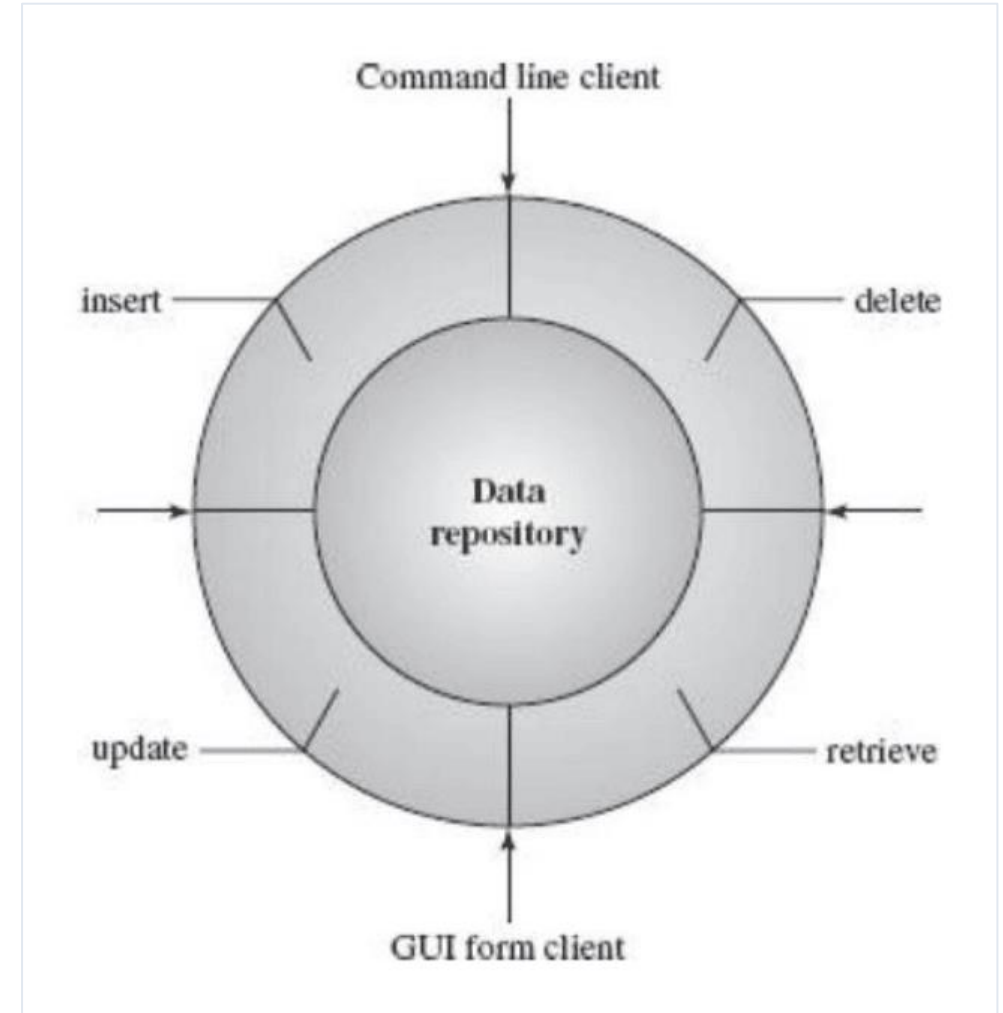


Computational Process 들이 Memory 에 data access

예제 – Relational DBMS

■ RDBMS 는 전형적인 Repository

- Command line client 와 GUI form client 가 중앙 Data repository 에 접근한다
- client 들은 insert / delete / update / retrieve 연산으로만 데이터를 다루며 서로 직접 통신하지 않는다
- Data repository 는 수동적(passive) — client 의 요청이 있을 때만 동작한다

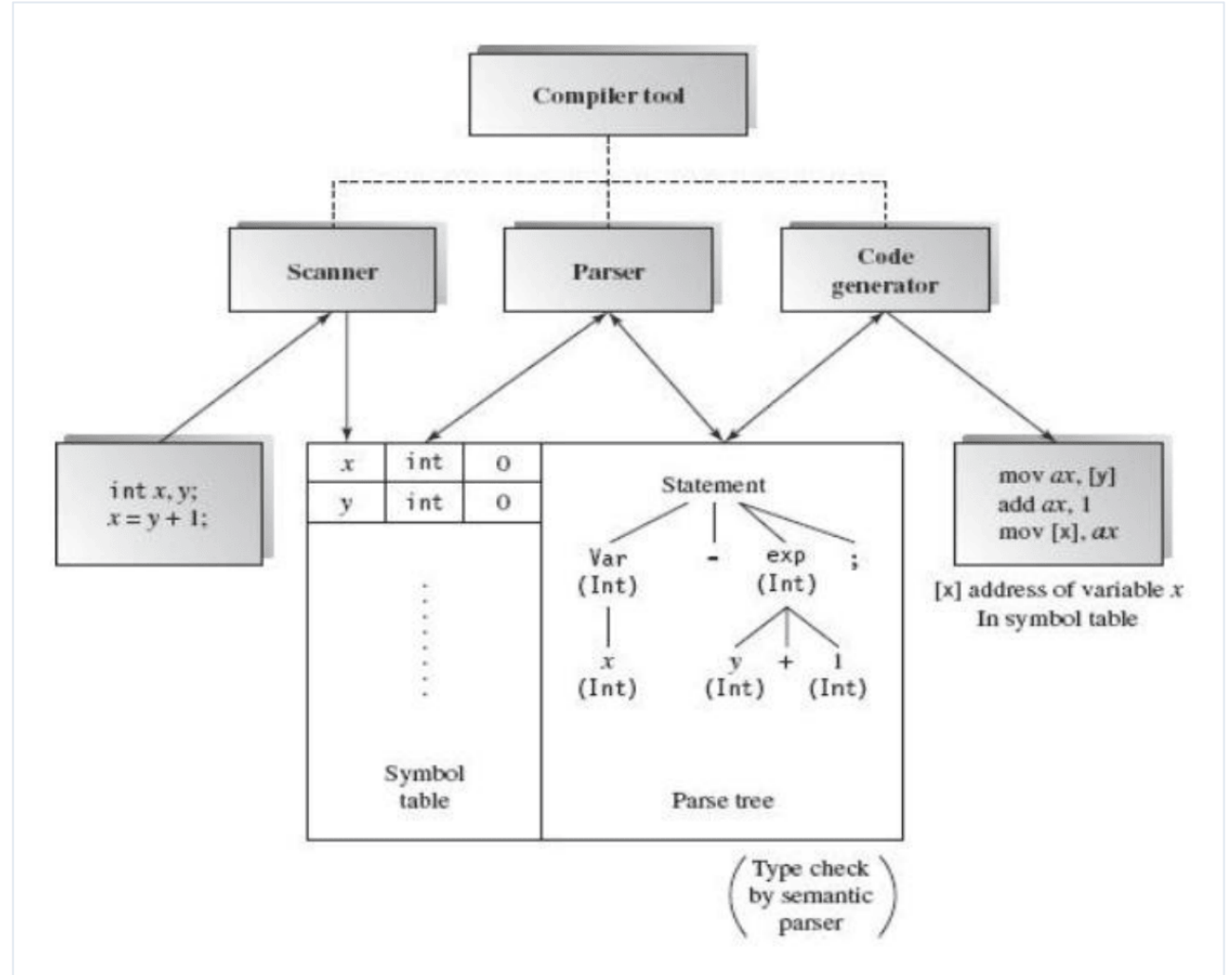


Relational DBMS

예제 – Compiler System

■ 공유 데이터 구조를 중심으로 한 Compiler

- Compiler tool 의 Scanner, Parser, Code generator 가 computational process(data client) 역할을 한다
- Symbol table 과 Parse tree 가 공유 data store 역할을 하며, 각 process 는 이 구조를 읽고 쓴다
 - 예: Parser 가 만든 Parse tree 를 Semantic parser 가 type check 하고, Code generator 가 코드를 만든다



Compiler System: Symbol table 과 Parse tree 를 공유

Consequences: 장점과 약점

■ Benefits (장점)

- 새 data client 나 data store 를 추가하기 쉽다 (Scalability)
- 특정 data client 의 기능을 바꾸기 쉽다 (Modifiability)
- data client 를 재사용할 수 있다
- client 사이에 임시 데이터를 주고받는 overhead 가 줄어든다

■ Weaknesses (약점)

- data store 의 고장이나 공격이 시스템 전체의 availability / security 를 떨어뜨린다
- data store 의 데이터 구조와 client 사이의 의존성이 높다
- 데이터가 분산되어 있으면 네트워크로 데이터를 옮기는 비용이 든다

Blackboard Style — 구성 요소

■ Elements

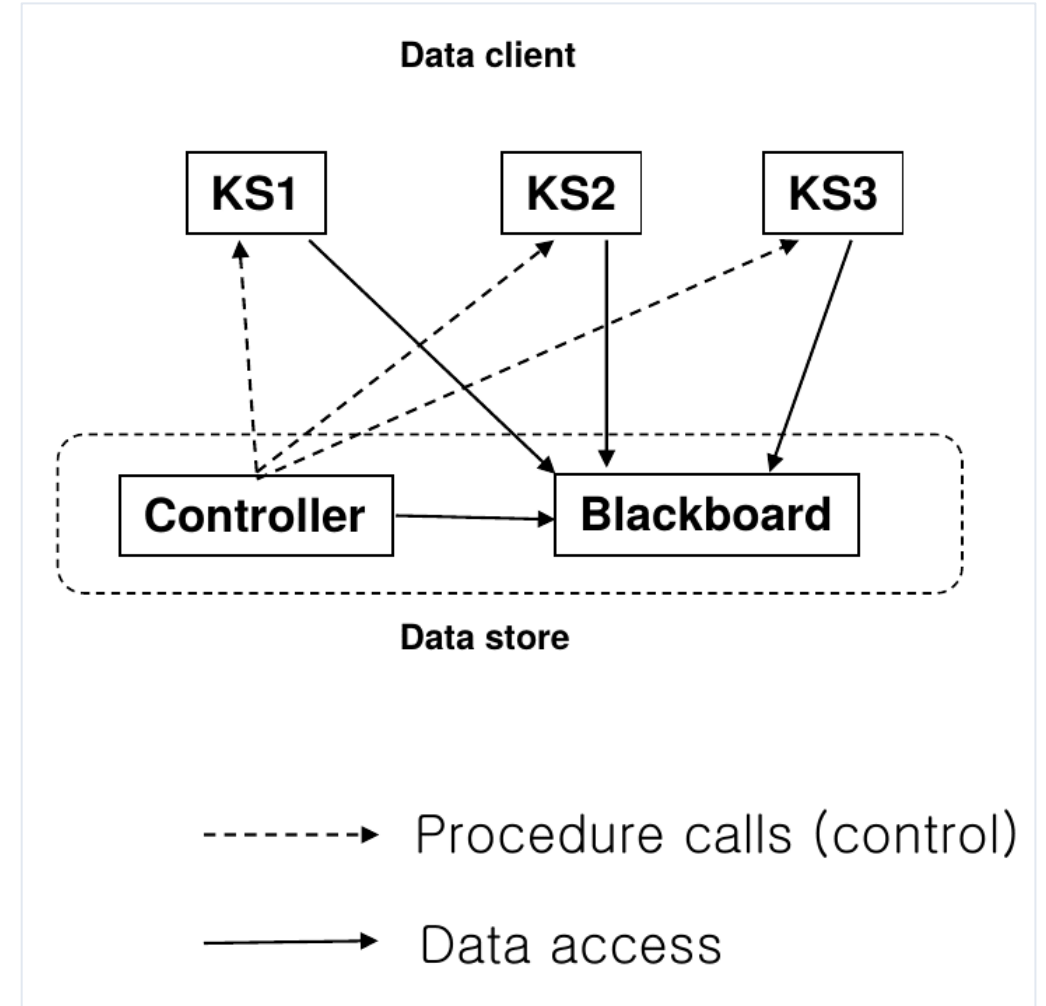
- Components
 - knowledge source(KS): data client
 - Blackboard: 중앙 데이터 저장소 역할
 - Controller: Blackboard 의 상태를 감시하고 knowledge source 의 처리를 촉발(trigger)한다
- Connectors: data access 또는 procedure call

■ Relations

- Controller 가 procedure call 로 KS 를 호출하고, KS 는 Blackboard 에 data access 한다 (그림 참조)

■ Constraints

- data store(Controller + Blackboard)는 능동적(active), client(knowledge source)는 수동적(passive)



Data store(Controller, Blackboard)와 Data client(KS1~KS3)

Blackboard Pattern (POSA)

■ Blackboard

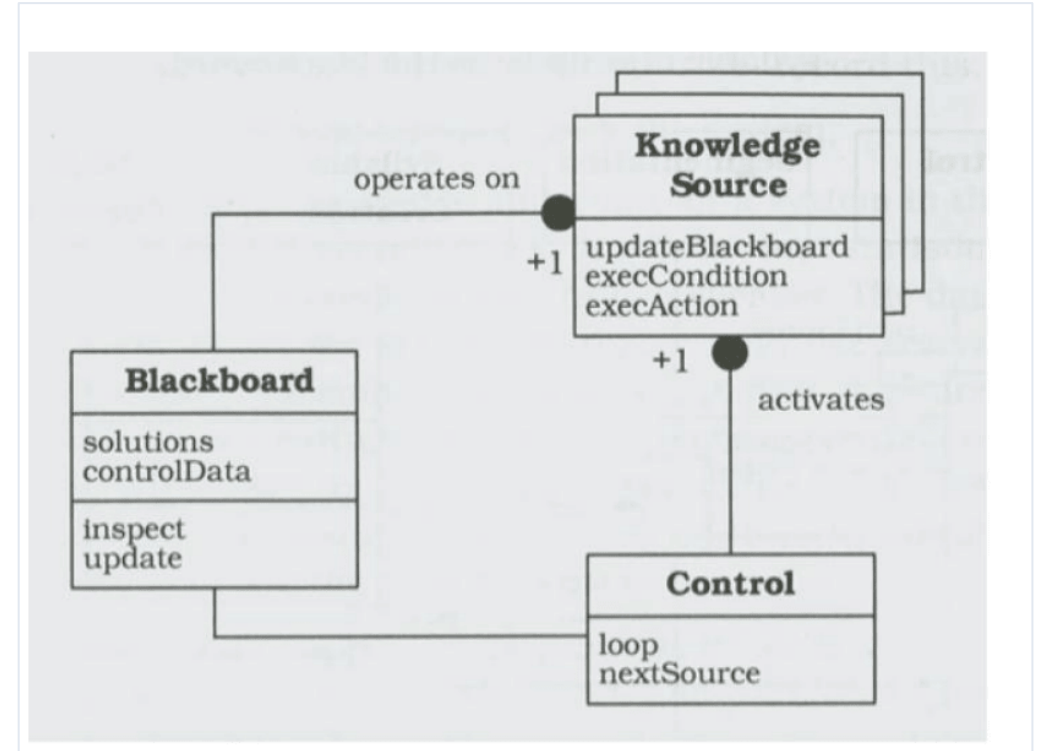
- update: Blackboard 위의 데이터를 변경한다
- inspect: Blackboard 위의 현재 해(solution)를 확인한다

■ Knowledge Source

- Blackboard 를 update 한다
- 자신이 적용 가능한지(applicability) 스스로 평가한다 (execCondition)
- 결과를 계산한다 (execAction)

■ Control

- Blackboard 의 변화를 감시하는 loop 를 돌며 다음에 취할 행동(nextSource)을 결정한다

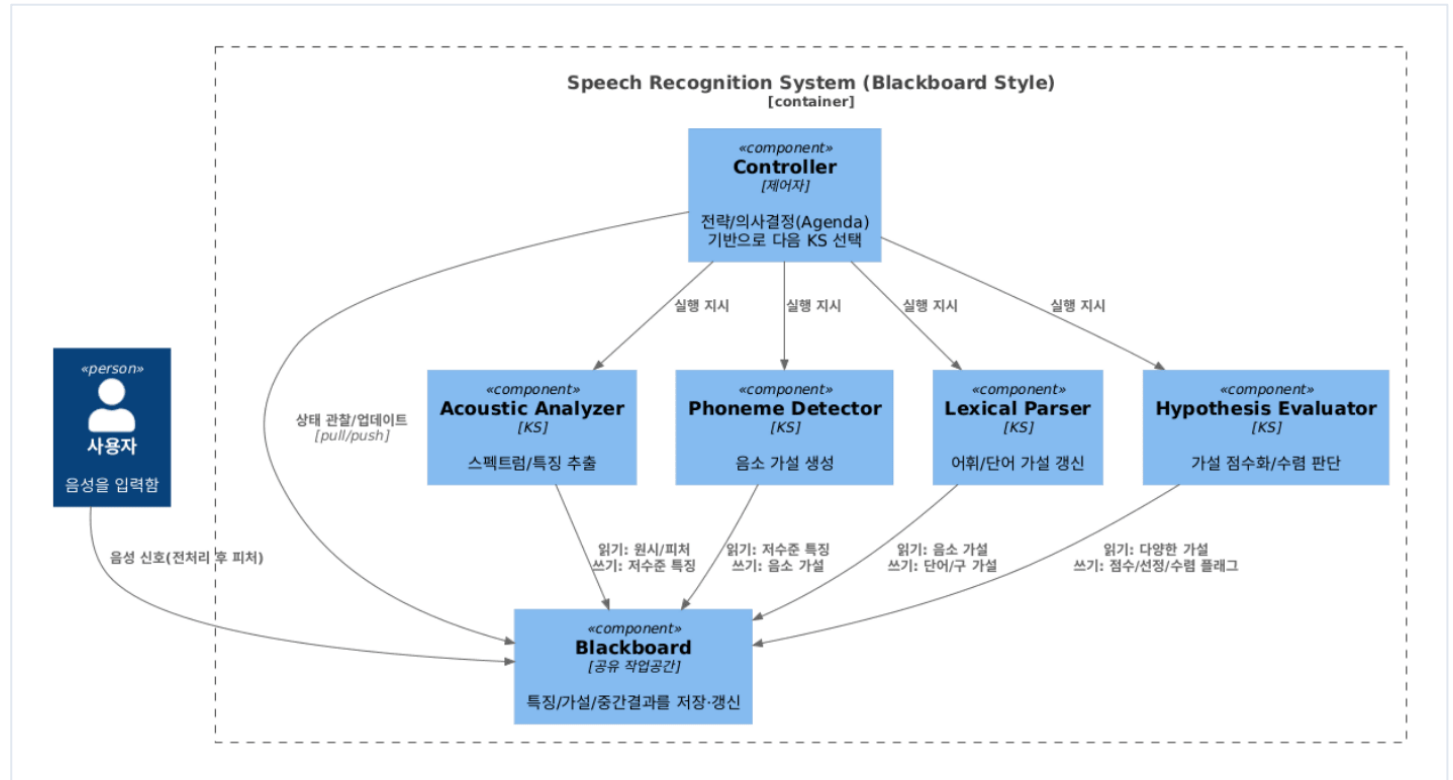


Blackboard Pattern 의 class 구조 (Buschmann 외, POSA Vol.1, 1996, pp.71-98)

예제 – Speech Recognition System

■ 음성 인식 시스템의 Blackboard 구조

- 사용자의 음성 입력을 Acoustic Analyzer, Phoneme Detector, Lexical Parser, Hypothesis Evaluator 같은 knowledge source(KS)들이 단계적으로 해석한다
- Controller 가 Blackboard 의 상태를 보고 다음에 실행할 KS 를 선택.활성화한다
- 각 KS 는 Blackboard 의 가설 (hypothesis)을 읽고 자신의 결과를 다시 Blackboard 에 기록한다

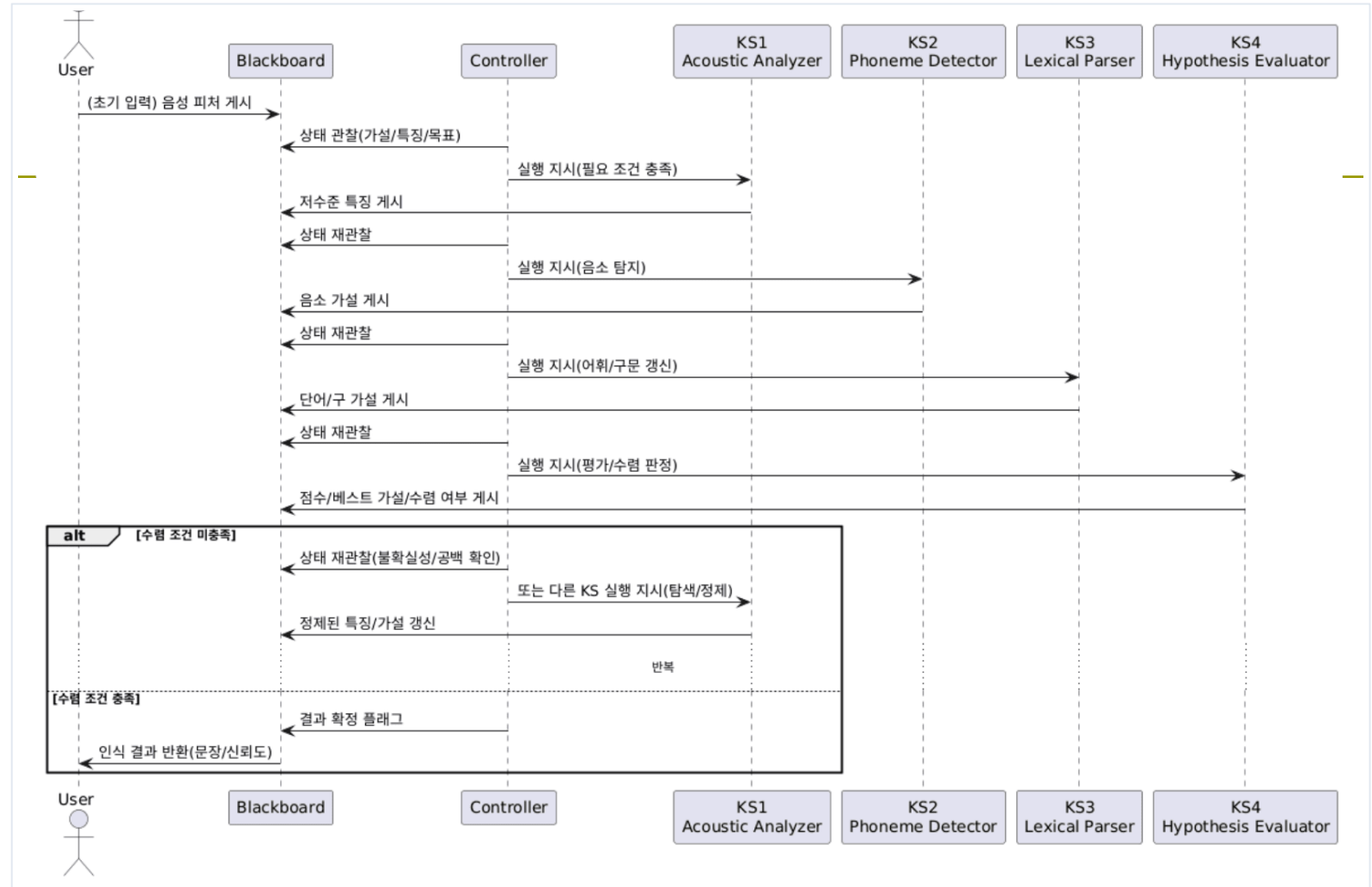


Speech Recognition System (Blackboard Style) — container 구조

예제 – Speech Recognition 의 동작 흐름

■ Sequence diagram 으 로 본 동작

- User 의 음성 입력이 Blackboard 에 기록되면 Controller 가 상태 변화를 감지한다
- Controller 는 실행 가능한 KS 를 골라 차례로 호출하고, 각 KS 는 결과를 Blackboard 에 갱신한다
- 가설이 충분히 확정될 때까지 loop 를 반복한 뒤 최종 인식 결과를 User 에게 돌려준다

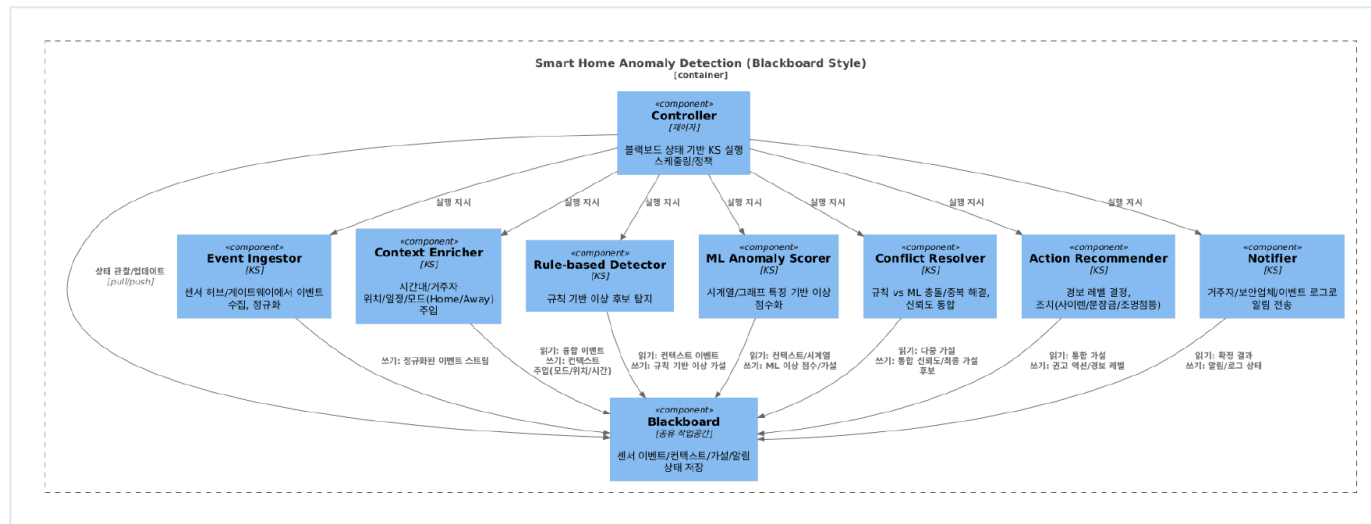


Controller 가 Blackboard 상태에 따라 KS1~KS4 를 반복 호출한다

예제 – Smart Home Anomaly Detection System

■ 스마트홈 이상 탐지

- Event Ingestor, Context Enricher, Rule-based Detector, ML Anomaly Scorer, Conflict Resolver, Action Recommender, Notifier 같은 KS 들이 Blackboard 를 공유한다
- Controller 가 Blackboard 의 변화를 보고 어떤 KS 를 다음에 실행할지 결정한다
- 규칙 기반과 ML 기반 탐지 결과를 Blackboard 위에서 합쳐 최종 판단과 알림으로 이어진다



Smart Home Anomaly Detection (Blackboard Style)

Consequences: 장점과 약점

■ Benefits (장점)

- 새 data client 나 data store 를 추가하기 쉽다 (Scalability)
- 특정 data client 의 기능을 바꾸기 쉽다 (Modifiability)
- data client 를 재사용할 수 있다
- knowledge source 들이 서로 독립적이므로 모두 병렬로 동작할 수 있다 (concurrency)
- 다양한 가설(hypothesis)을 실험해 볼 수 있다

■ Weaknesses (약점)

- data store 의 고장이나 공격이 시스템 전체의 availability / security 를 떨어뜨린다
- data store 의 데이터 구조와 client 사이의 의존성이 높다
- 근사 해(approximate solution)만 기대되므로 언제 추론을 끝낼지 결정하기 어렵다
- 테스트가 어렵다

1.4

Implicit Invocation Architecture

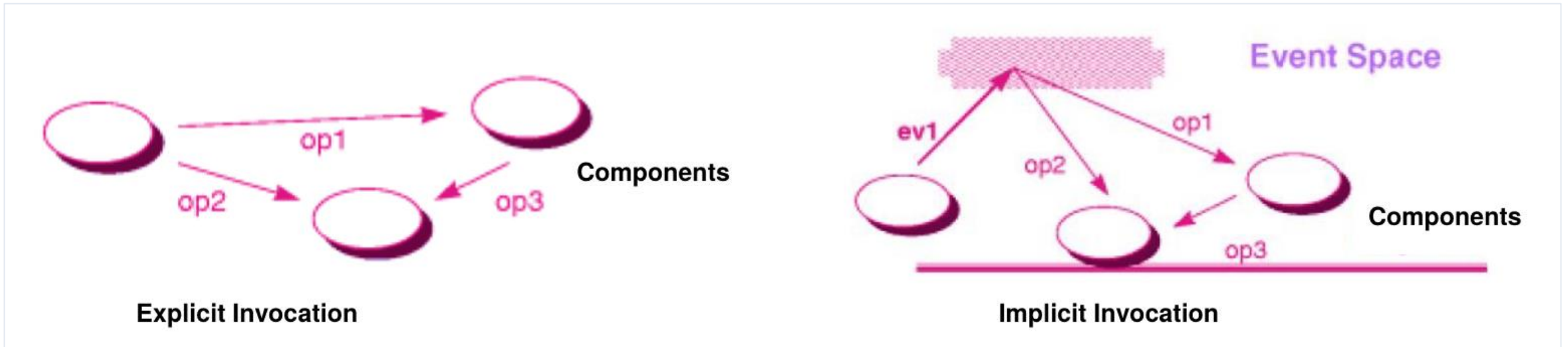
procedure 를 직접 호출하지 않고, event 나 message 를 통해 간접적으로 다른 component 를 동작시키는 스타일

- Implicit Invocation 의 개념과 구조
- Non-buffered Event-Based Style
- Buffered Message-Based Style: Point-to-Point, Publish-Subscribe

Implicit Invocation 이란

■ 개념

- Implicit Invocation(암시적 호출): procedure 를 직접 호출하는 대신, event 발표(announcement)가 다른 module 의 procedure 호출을 암시적으로 일으킨다
- Explicit Invocation(그림 왼쪽): component 가 다른 component 의 operation 을 직접 호출한다
- Implicit Invocation(그림 오른쪽): component 가 Event Space 에 event(ev1)를 발표하면, 그 event 에 등록된 operation(op1, op2, op3)들이 호출된다



Explicit Invocation 과 Implicit Invocation 의 비교

핵심 호출하는 쪽은 누가 반응할지 모른다 — 이것이 loose coupling 의 출발점이다

Implicit Invocation Architecture 의 구조와 Style

■ 특징

- software component 사이의 느슨하게 결합된(loosely coupled) 통신이 특징이다

■ Structure

- 시스템을 두 부류(party)로 나눈다
 - 한쪽: publisher, producer, sender, source
 - 다른 쪽: subscriber, consumer, receiver, target
- 두 부류 사이의 통신은 synchronous 또는 asynchronous 일 수 있다
- 통신 mode: one-to-one, one-to-many, many-to-one

■ Styles of Implicit Invocation

- Non-buffered Event-Based Style
 - Observer Pattern (one-to-many, many-to-one)
- Buffered Message-Based Style
 - Point-to-Point Message Architecture (one-to-one)
 - Publish-Subscribe Messaging Architecture (one-to-many)

핵심 buffer 유무가 Event-Based 와 Message-Based 를 가르는 핵심 차이이다

Non-buffered Event-Based Style — 구성 요소

■ Elements

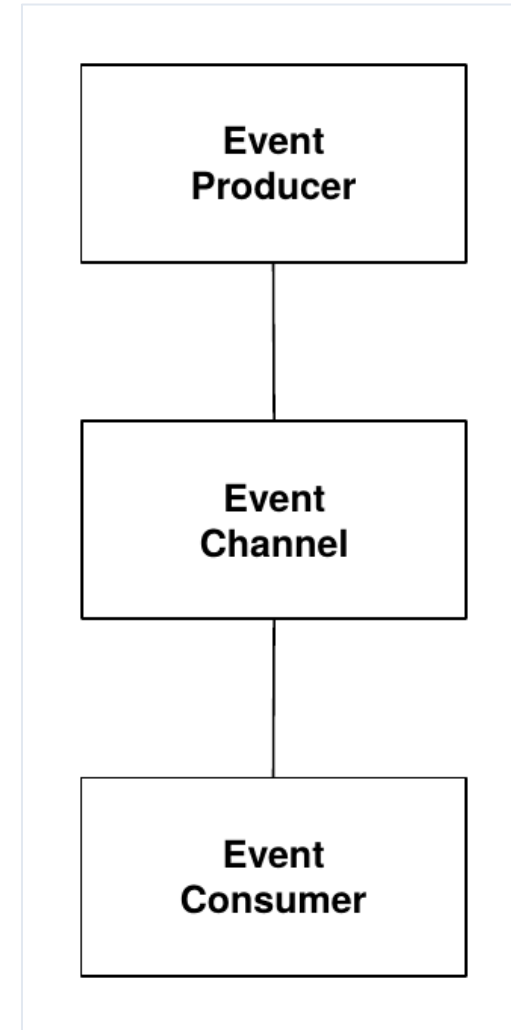
- Event Producer: 특정 조건이나 상태 변화가 생기면 event 를 감지·생성·발생(emit)시킨다
- Event Channel: producer 와 consumer 를 직접 연결하는 가벼운 통신 매체 — buffer 가 없다
- Event Consumer: event 가 발생하면 즉시 받아서 처리한다

■ Relations

- Producer → Channel → Consumer 로 event 가 전달된다 (그림 참조)

■ Constraints

- event 는 생성되는 즉시 consumer 에게 전달되어야 한다
- producer 와 consumer 는 서로 독립적으로 유지된다

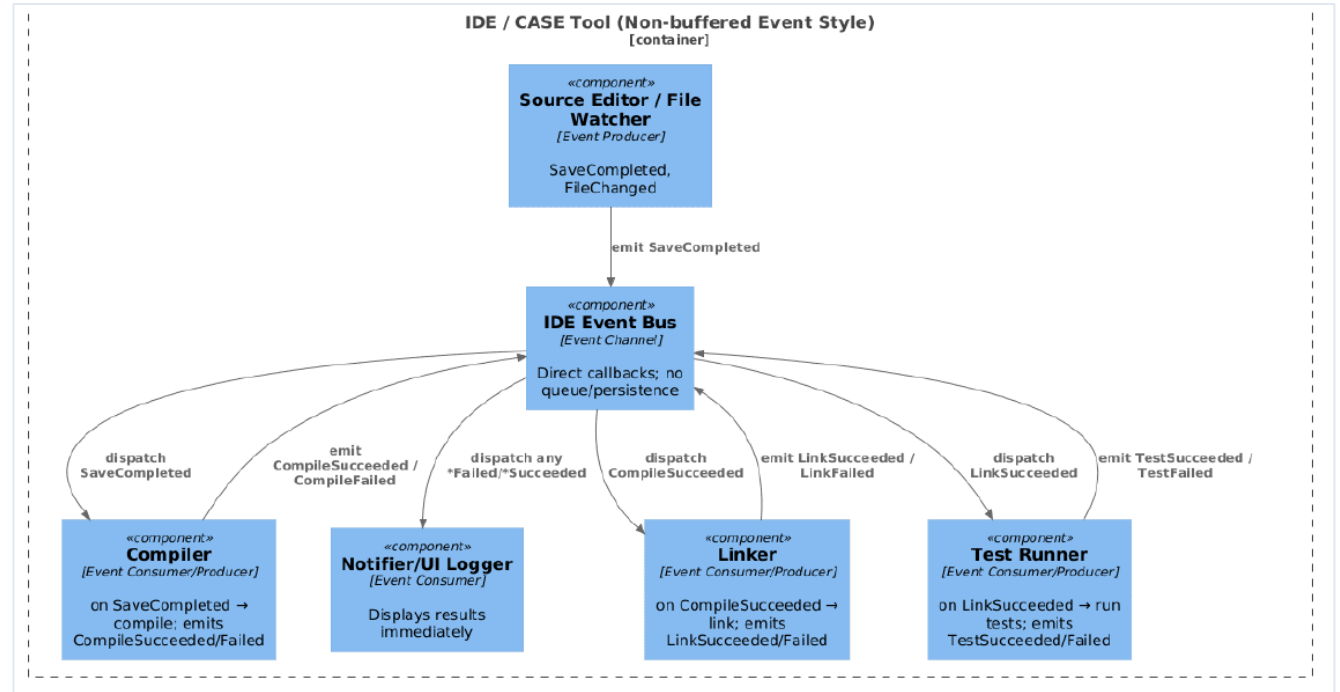


Event Producer – Event Channel – Event Consumer

적용 예제 – IDE / CASE Tool

■ 많은 IDE 와 CASE tool 이 이 스타일을 쓴다

- source file 편집이 끝나면(SaveCompleted) 그 event 가 compile 또는 재compile 을 촉발한다
- compile 이 끝나면(CompileSucceeded) 이어서 relink 처리가 촉발되고, 이런 연쇄가 계속된다
- Source Editor(producer)가 IDE Event Bus(channel)에 event 를 보내면 Compiler, Notifier, Linker, Test Runner(consumer)가 즉시 반응한다

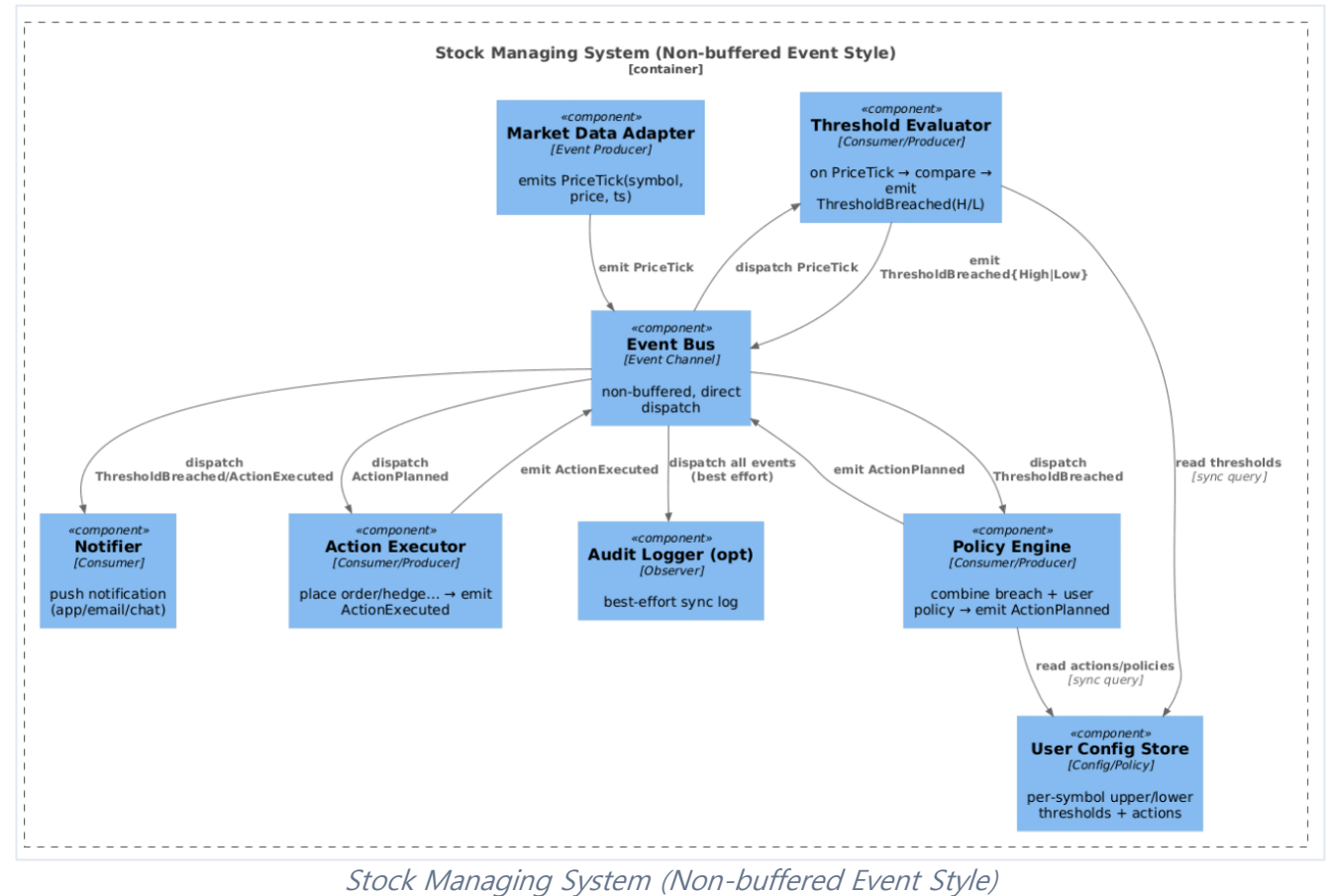


IDE / CASE Tool (Non-buffered Event Style)

적용 예제 – Stock Managing System

■ 주식 관리 시스템

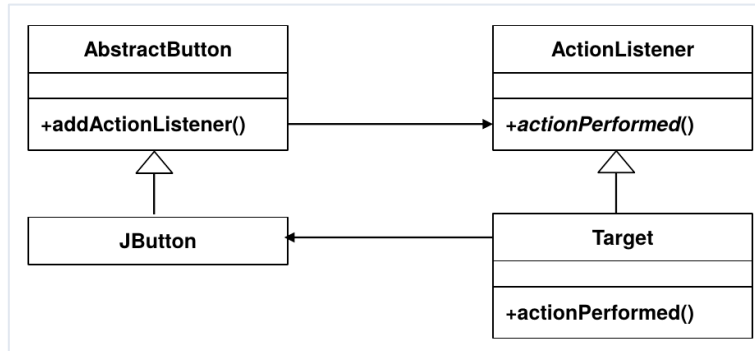
- 사용자는 특정 주식의 상한(upper)·하한(lower) threshold 를 설정할 수 있다
- 주가가 너무 높거나 낮아지면 시스템이 자동으로 사용자에게 알리고, 미리 설정해 둔 행동을 자동으로 수행한다
- Market Data Adapter 가 PriceTick event 를 내면 Threshold Evaluator 가 비교해 ThresholdBreached event 를 내고, Event Bus 가 Notifier·Action Executor·Policy Engine 에 즉시 전달한다



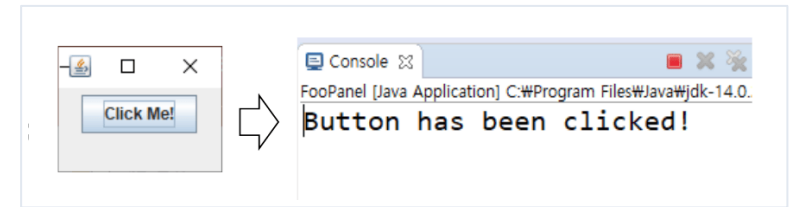
예제 – Java Swing 의 Event Listener

```
public class FooPanel extends JPanel{
    public FooPanel() {
        super();
        JButton btn = new JButton("Click Me!");
        btn.addActionListener(new Target());
        this.add(btn);
    }
    public static void main(String[] args) {
        JFrame frame = new JFrame();
        frame.setSize(100,100);
        frame.add(new FooPanel());
        frame.setVisible(true);
    }
}
class Target implements ActionListener {
    public void actionPerformed(ActionEvent ae) {
        System.out.println("Button has been clicked!");
    }
}
```

FooPanel / Target 코드



AbstractButton – ActionListener 구조



실행 결과

- JButton(event producer)에 addActionListener() 로 Target(event consumer)을 등록한다
- 버튼을 누르면 JButton 이 Target.actionPerformed() 를 직접 호출하지 않고, event 발생을 통해 암시적으로 호출한다
- JButton 은 ActionListener interface 만 알 뿐 Target 이 누구인지 모른다 — Observer Pattern

Consequences: 장점과 약점

■ Benefits (장점)

- Framework 이 풍부하다 — Java AWT, Swing 등 많은 vendor API 가 제공된다
- component 재사용성 — 기존 event listener 를 시스템의 나머지 부분에 영향 없이 끼워 넣기 쉽다
- 유지보수와 진화가 쉽다 — event source 와 listener 모두 갱신하기 쉽다
- 유연한 연결 — 실행 중에 동적으로 등록 (registration)·해제(deregistration)할 수 있다

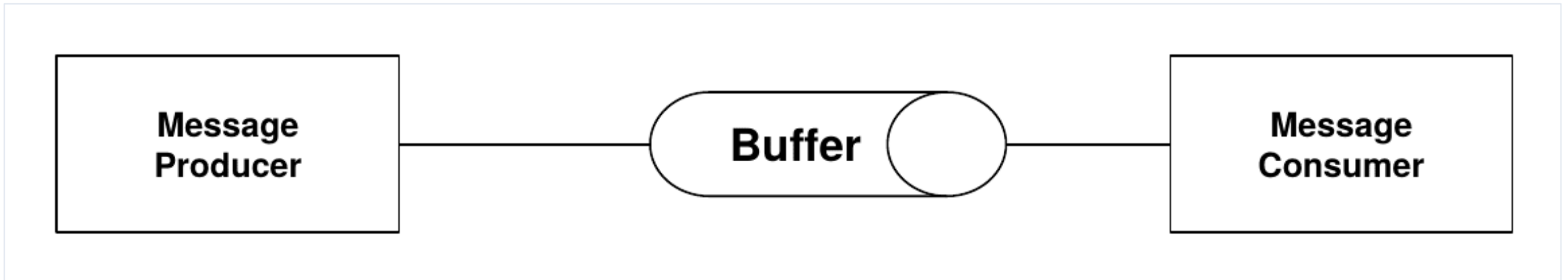
■ Weaknesses (약점)

- listener 의 응답과 응답 순서를 예측·검증하기 어려워 테스트와 디버깅이 어렵다
- 간접 호출(indirect invocation)의 overhead 가 문제가 될 수 있다
- Message-Based 방식보다 event source 와 listener 사이의 결합(coupling)이 더 강하다

Buffered Message-Based Style — 개요

■ 시스템을 두 부분으로 나눈다

- Message Producer: message 를 만들어 buffer 로 보낸다 — 그 message 를 누가 소비하는지 모른다
- Message Consumer: buffer 에서 message 를 비동기적(asynchronous)으로 받는다
- 두 부분은 buffer(message queue 또는 pub-sub connector)를 통해 비동기로 연결된다



Message Producer → Buffer → Message Consumer

핵심 Buffer 가 있으므로 producer 와 consumer 가 동시에 살아 있을 필요가 없다

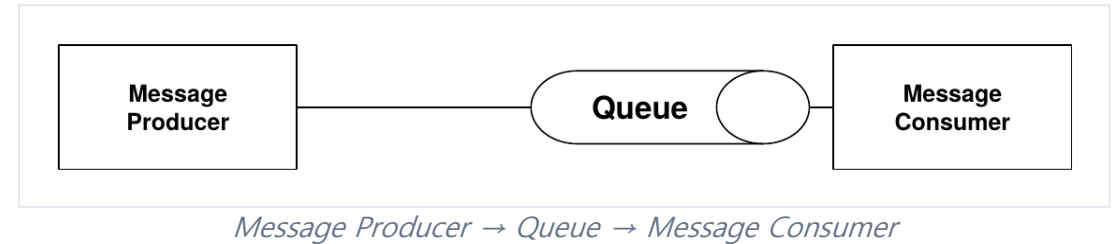
Point-to-Point Messaging Architecture

■ Elements

- Message Producer (component): 특정 queue(destination)로 message 를 보낸다
- Message Consumer (component): queue 에서 message 를 꺼내 간다
- Queue (connector): 받은 message 를 소비되거나 만료 (expire)될 때까지 모두 보관한다

■ Relations / Constraints

- Producer → Queue → Consumer 의 one-to-one 연결 (그림 참조)
- message 기반 통신은 보통 비동기 — publisher 와 subscriber 가 동시에 활성 상태일 필요가 없다



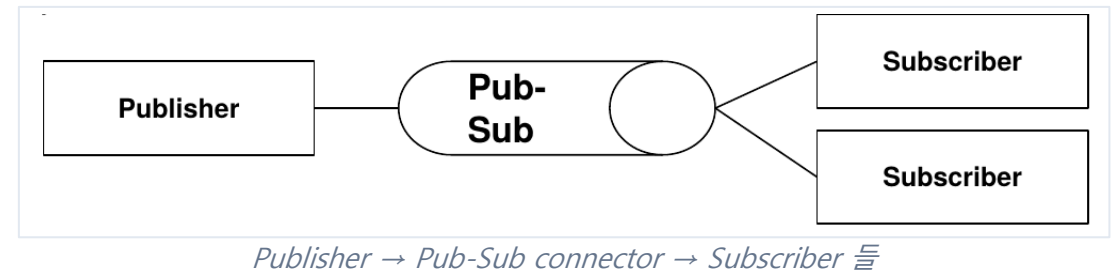
Publish-Subscribe Messaging Architecture

■ Elements

- Publisher (component): pub-sub connector 로 message 를 보낸다
- Subscriber (component): 관심 있는 특정 종류의 message 를 구독(subscribe)하고, 구독한 message 를 connector 로부터 받는다
- Pub-Sub (connector): publisher 가 발행한 message 를 구독 정보(subscription)에 따라 관련 subscriber 들에게 전달한다

■ Relations / Constraints

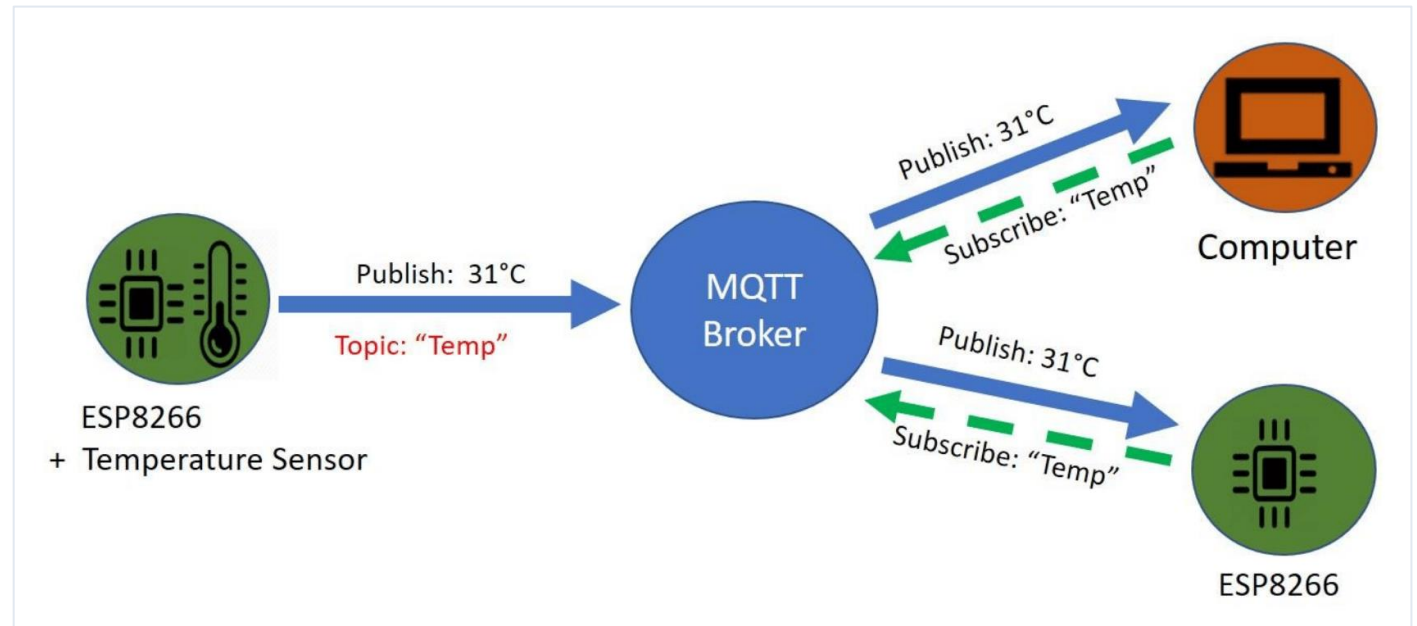
- Publisher → Pub-Sub → 여러 Subscriber 의 one-to-many 연결 (그림 참조)
- 비동기 message 통신이며, 시스템은 message 를 모든 subscriber 에게 전달한다



예제 – MQTT Broker

■ IoT 의 Publish-Subscribe

- ESP8266 + 온도 센서(publisher)가 "Temp" topic 으로 31°C 를 MQTT Broker 에 publish 한다
- Computer 와 다른 ESP8266(subscriber)은 "Temp" topic 을 미리 subscribe 해 두었다
- Broker(pub-sub connector)가 해당 topic 을 구독한 모든 subscriber 에게 message 를 전달한다
- publisher 는 누가 받는지 모르고, subscriber 는 누가 보냈는지 모른다



MQTT Broker 를 통한 온도 데이터 배포

Consequences: 장점과 약점

■ Benefits (장점)

- 익명성(Anonymity) — consumer 는 다음을 알 필요가 없다
 - 누가 message 를 만들었는지 (user independence)
 - producer 가 네트워크 어디에 있는지 (location independence)
 - message 가 언제 만들어졌는지 (time independence)
- 동시성(Concurrency) — consumer 끼리, 그리고 producer 와 consumer 사이에 동시 동작
- message 전달의 신뢰성 — acknowledgement 수준, priority, expiration 을 설정할 수 있다

■ Weaknesses (약점)

- message queue 의 용량 한계
 - 본질적 한계라기보다 구현 문제 — queue 를 동적 자료구조(linked list 등)로 만들면 줄일 수 있다
 - 다만 가용 메모리에 따른 절대적 한계는 존재한다
- overhead, 간접 호출의 복잡성, 디버깅의 어려움

1.5

Interaction-Oriented Architecture

사용자와의 상호작용(UI)을 데이터 추상화·비즈니스 로직으로부터 분리하는 스타일

- Interaction-Oriented Architecture 의 목적과 구조
- Model-View-Controller (MVC) Style — 구조, 예제, Variants(Document-View, MVP, MVVM), Consequences
- Presentation-Abstraction-Control (PAC) Style — 구조, 예제, Consequences

Interaction-Oriented Architecture 개요

■ 목적(Objective)

- 사용자와의 상호작용(interaction)을 데이터 추상화와 비즈니스 데이터 처리로부터 분리한다

■ 구조(Structure) — 시스템을 세 부분으로 분해

- Data module: 데이터 추상화와 모든 비즈니스 로직을 제공
- Control module: 제어 흐름(flow of control)과 시스템 구성(configuration) 동작을 관리
- View presentation module: 데이터 출력의 시각·청각 표현을 담당하고, 사용자 입력 인터페이스를 제공

■ 두 가지 대표 Style

- Model-View-Controller (MVC), Presentation-Abstraction-Control (PAC)
- 둘 다 세 component 로의 분해를 제안하며, 웹 애플리케이션 같은 대화형(interactive) 애플리케이션에 쓰인다
- 제어 흐름과 조직 방식이 다르다
 - PAC 는 agent 기반의 계층(hierarchical) 구조이지만, MVC 는 명확한 계층 구조가 없다

핵심 UI(상호작용)·제어·데이터를 분리하면 각 부분을 독립적으로 바꾸고 재사용할 수 있다

Model-View-Controller Style — 개요와 Elements

■ MVC 란

- 대화형 애플리케이션 설계에 널리 쓰이는 architecture style
- 정보의 내부 표현(Model)을 사용자에게 보여 주거나 사용자로부터 받는 정보(View)와 분리한다
- 변경 전파(change propagation) 메커니즘이 사용자 인터페이스와 Model 의 일관성을 보장한다

■ Elements

- Model: 기반 데이터와 비즈니스 로직을 캡슐화한다. View 나 Controller 에 의존하지 않는다
- View: 사용자에게 정보를 표시한다. 애플리케이션 로직을 갖지 않는다
- Controller: View 로부터 사용자 입력을 받아 Model 에 대한 동작(action)으로 변환한다
 - 애플리케이션 로직을 가지며, Model 과 View 사이의 데이터 흐름을 관리한다

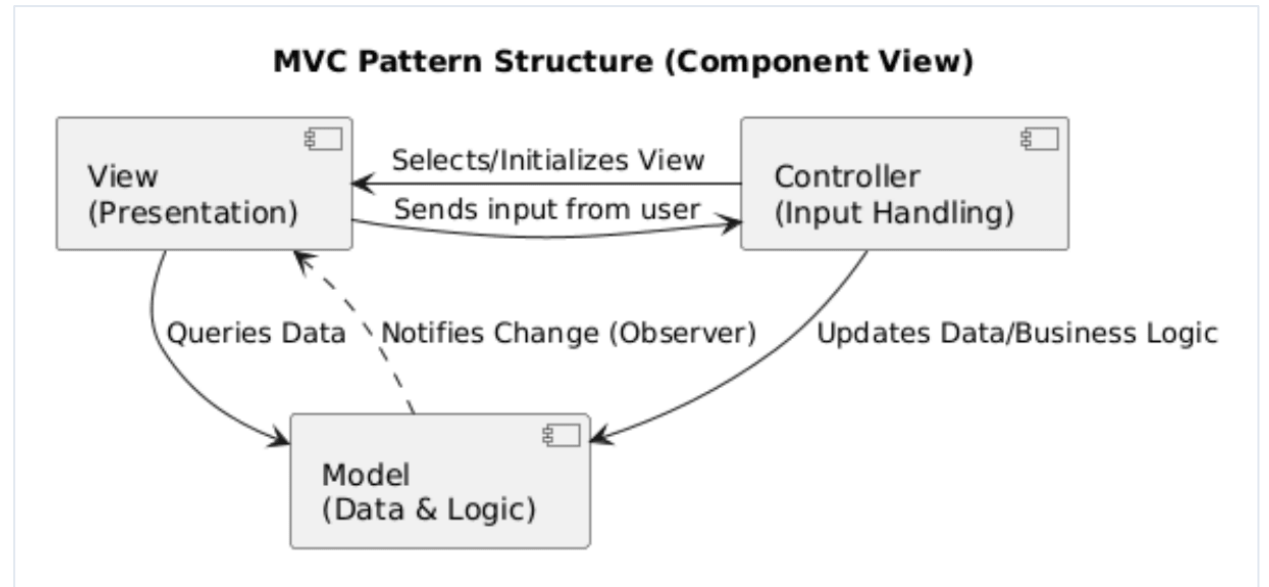
MVC — Relations(구조)와 적용 분야

■ Relations

- Controller 는 View 를 선택·초기화하고 Model 의 데이터·비즈니스 로직을 갱신한다
- View 는 사용자 입력을 Controller 로 보내고 Model 에 데이터를 질의(query)하며, Model 은 변경을 Observer 방식으로 View 에 알린다

■ Applicable Domain

- 하나의 데이터 Model 에 여러 View 가 필요하고, 새 인터페이스 View 를 쉽게 끼워 넣어야 하는 대화형 애플리케이션
- 모듈 간 경계가 뚜렷해 서로 다른 전문가(그래픽 전문가, 프로그래머, DB 개발자 등)가 각 부분을 동시에 개발할 수 있는 애플리케이션

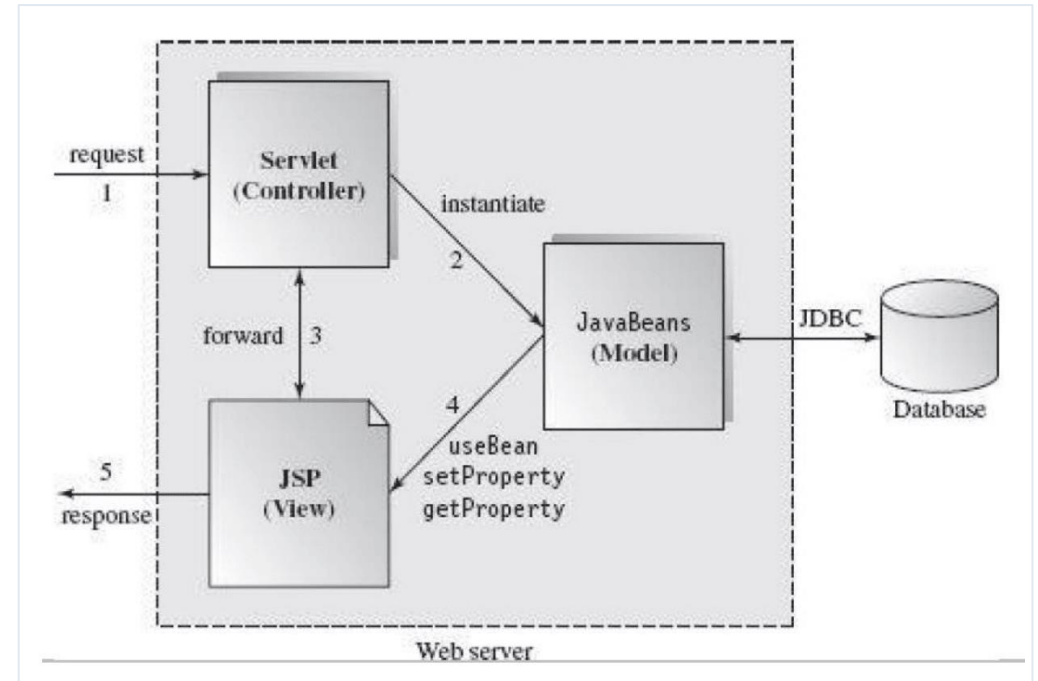


MVC Pattern Structure (Component View)

예제 — Spring MVC Framework

■ Web server 안의 MVC 구성

- Servlet 이 Controller 역할: 요청(request)을 받아(1) Model 인 JavaBeans 를 생성(instantiate)하고(2) JSP 로 forward 한다(3)
- JavaBeans 가 Model 역할: JDBC 로 Database 에 접근한다
- JSP 가 View 역할: useBean / setProperty / getProperty 로 Model 의 데이터를 읽어(4) 응답(response)을 만든다(5)



Spring MVC: Servlet(Controller) – JavaBeans(Model) – JSP(View)

핵심 요청 처리(Controller), 데이터(Model), 화면 생성(View)의 역할이 서로 다른 기술 요소에 배정된다

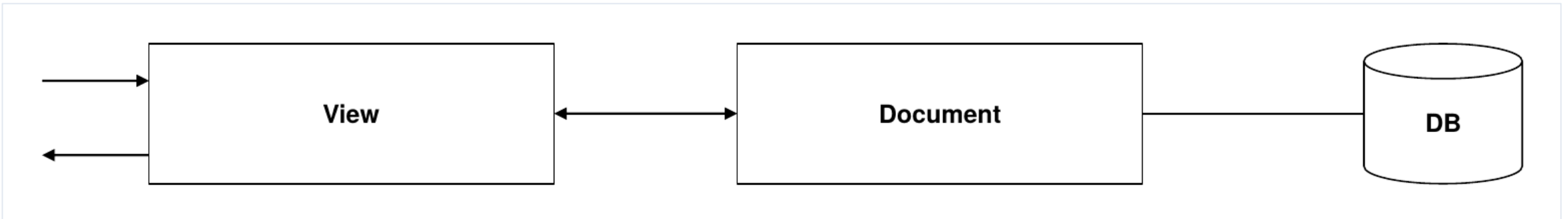
MVC Variants (1) — Document-View

■ Document-View: 시스템을 두 개의 sub-system 으로 나눈다

- View (Controller-View): 입출력 인터페이스 역할을 하며, 처리(processing)도 여기서 수행한다
- Document (Model): 모든 데이터와 도메인 서비스를 제공한다

■ View 와 Document 의 연결 — Observer 패턴

- View 가 Document 를 구독(subscribe)하고, Document 는 변경이 생기면 View 에 통지(notify)한다



View ↔ Document ↔ DB

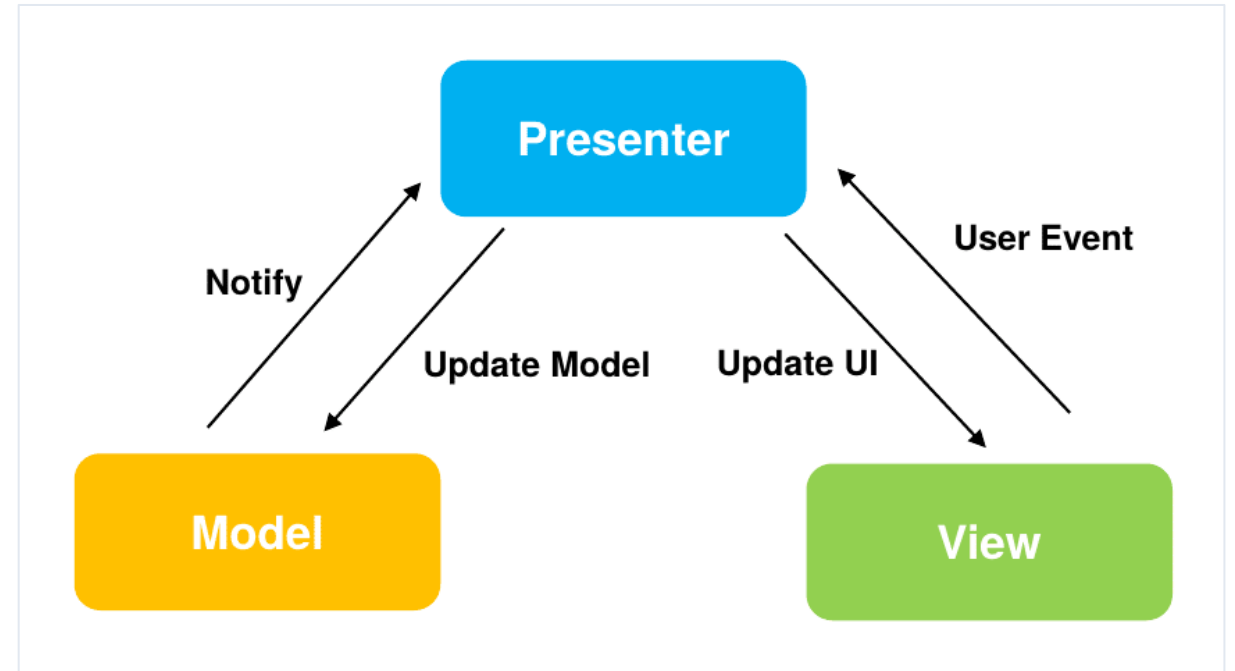
MVC Variants (2) — Model-View-Presenter (MVP)

■ Elements

- Model: 애플리케이션의 데이터와 비즈니스 로직을 담는다
- View: 사용자에게 데이터를 표시하고, 버튼 클릭 같은 사용자 입력 이벤트를 Presenter 로 전달한다
- Presenter: View 로부터 사용자 동작을 받아 Model 에 대한 명령으로 변환한다
 - Model 에서 데이터를 받아 View 에 표시할 형태로 가공한다

■ MVC 와의 차이

- MVC 의 Controller 는 사용자 입력을 관리하며 View 를 직접 갱신할 수도 있다
- MVP 의 Presenter 는 인터페이스를 통해 View 를 완전히 제어한다 → View 는 수동적(passive)이 되어 테스트하기 쉽다



MVP: User Event → Presenter → Update Model / Update UI

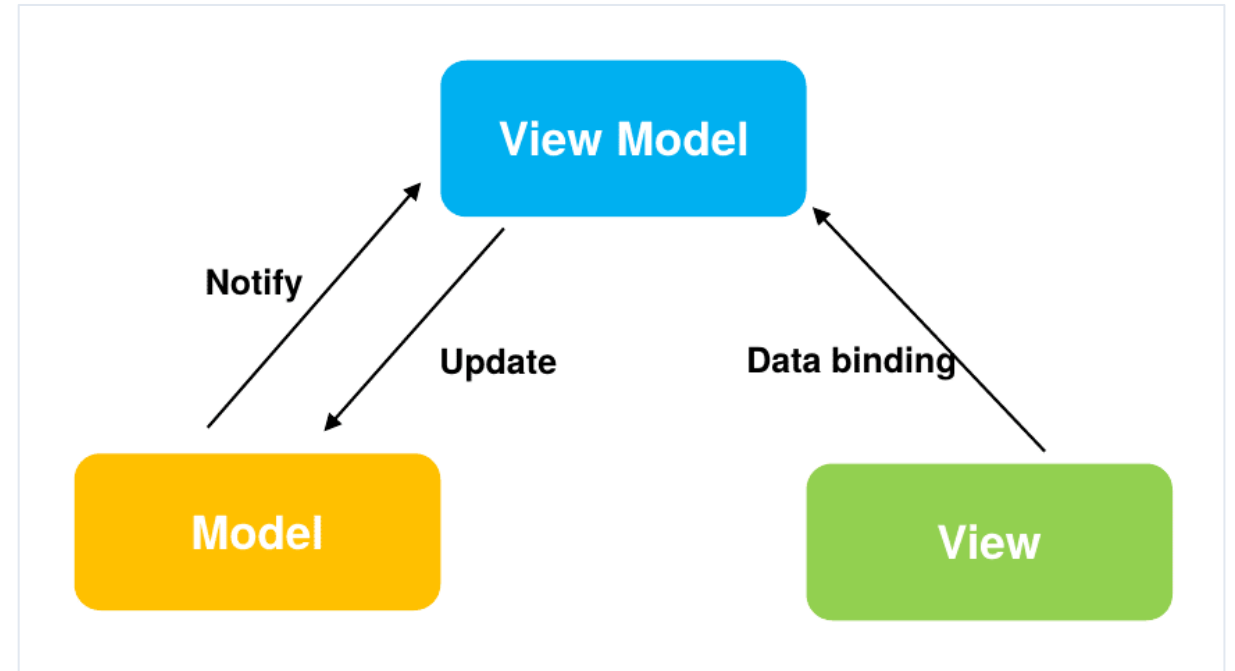
MVC Variants (3) — Model-View-ViewModel (MVVM)

■ Elements

- Model: 애플리케이션의 데이터와 비즈니스 로직을 담는다
- View: 사용자 인터페이스를 다루고 데이터를 표시하지만, 비즈니스 로직은 갖지 않는다
- ViewModel: Model 에서 데이터를 가져와 View 가 data binding 으로 쉽게 쓸 수 있는 형태로 재구성한다

■ MVC 와의 차이

- MVC 의 Controller 는 사용자 입력을 처리하고 View 를 직접 갱신한다
- MVVM 의 ViewModel 은 관찰 가능한(observable) 데이터와 명령을 노출하고, View 가 자동으로 binding 한다
 - ViewModel 은 View 를 직접 참조하지 않는다



MVVM: Model ↔ ViewModel(Notify/Update) ↔ View(Data binding)

MVC Consequences: 장점과 약점

■ Benefits (장점)

- 동기화된 View: Model 의 변경 전파 메커니즘이 연결된 모든 observer 에게 데이터 변경을 제때 알린다
- '끼워 넣을 수 있는(pluggable)' View 와 Controller: 한 Model 의 View·Controller 객체를 실행 중에도 교체할 수 있다
- 'look and feel' 교체 가능: 새 플랫폼으로 이식해도 기능 핵심(functional core)은 영향받지 않는다
 - 플랫폼별로 알맞은 View·Controller 구현만 있으면 된다

■ Weaknesses (약점)

- 복잡도 증가: UI 가 단순한 애플리케이션에서 MVC 구조를 엄격히 따르는 것이 항상 최선은 아니다
- 과도한 갱신(update) 발생 가능성
- View 의 데이터 접근 비효율: Model 인터페이스 (push vs. pull)에 따라 View 가 표시 데이터를 얻기 위해 여러 번 호출해야 할 수 있다
- Model·View·Controller 추상화가 일부 UI toolkit 과 잘 맞지 않을 수 있다

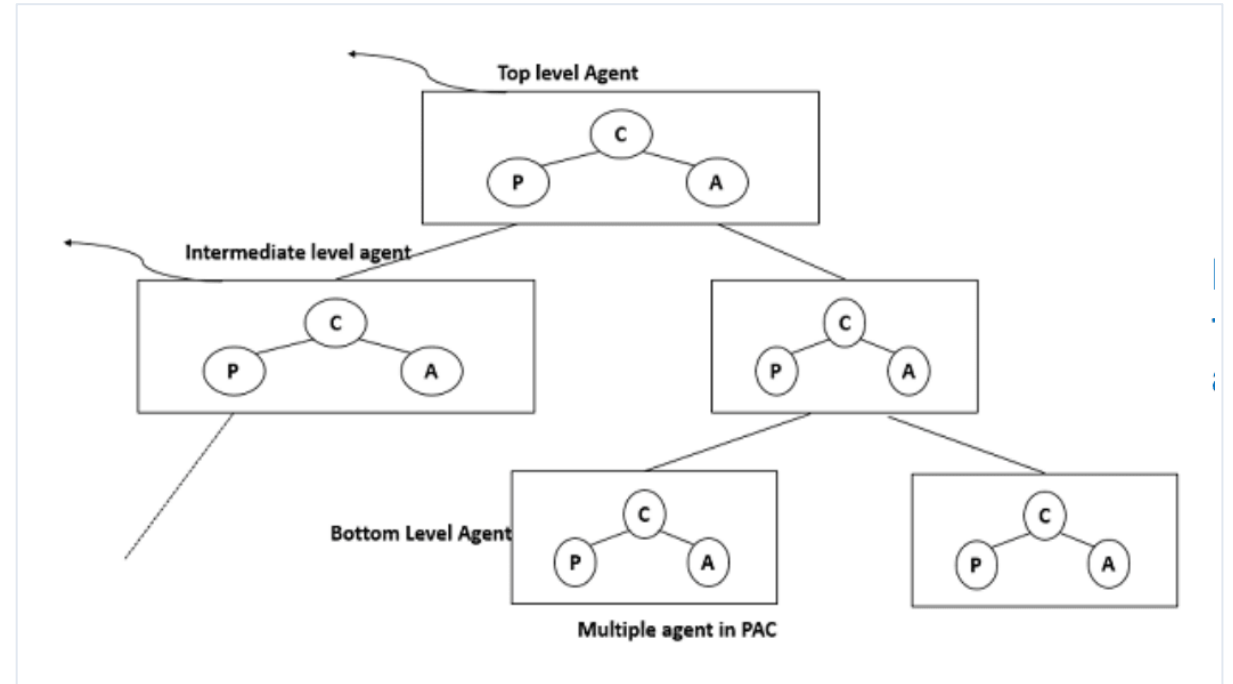
Presentation-Abstraction-Control Style — 개요

■ PAC 란

- 대화형 애플리케이션을 PAC agent 들의 트리(tree) 형태 계층으로 구성한다
- 각 agent 는 애플리케이션 기능의 특정 측면을 담당하며 Presentation, Abstraction, Control 세 component 로 이루어진다
- 최상위(top-level) agent 는 중간 계층 agent 들을 조율(orchestrate)한다
- 중간(intermediate-level) agent 는 관련된 하위 agent 들의 기능을 결합한다
- 최하위(bottom-level) agent 는 개별 기능을 담당한다

■ Applicable Domain

- 시스템을 계층적으로 협력하는 여러 agent 로 나눌 수 있는 대화형 시스템. 각 agent 는 고유한 역할을 가진다
- agent 간 결합(coupling)이 느슨해서 한 agent 의 변경이 다른 agent 에 영향을 주지 않아야 할 때



Multiple agent in PAC — Top / Intermediate / Bottom level agent

PAC Style — Elements · Relations · Constraints

■ Elements

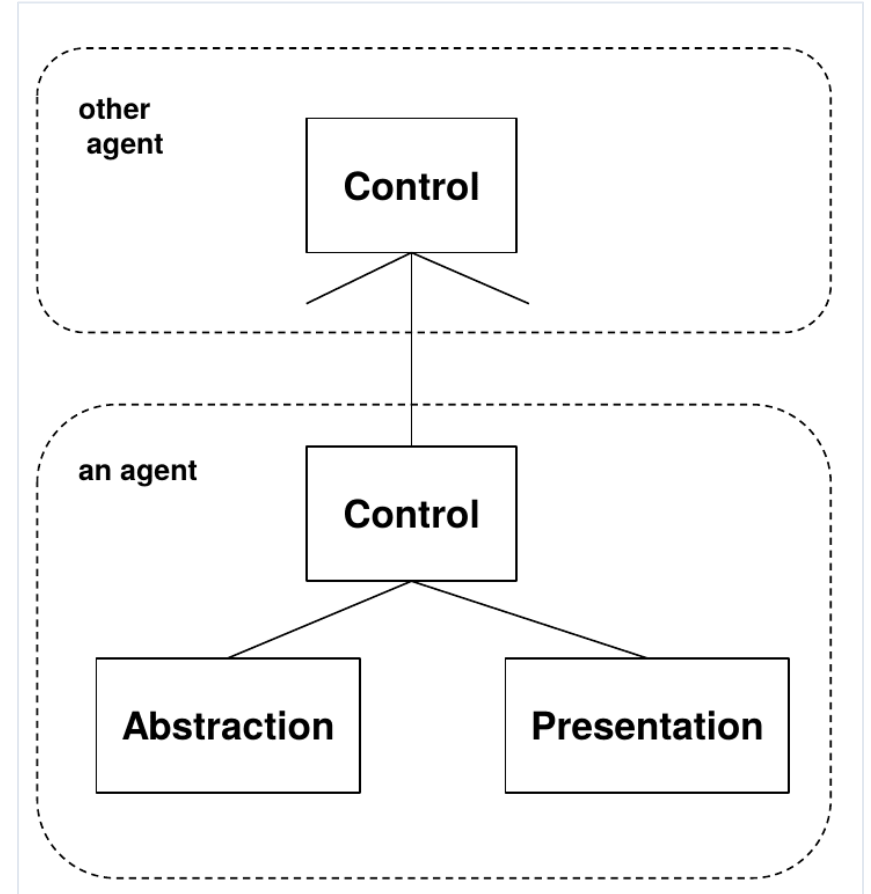
- Presentation: 사용자 인터페이스를 담당한다
- Abstraction: 애플리케이션 데이터와 그 데이터를 변경하는 로직을 유지한다
- Control: Presentation 과 Abstraction 사이를 중개하고, 다른 agent 의 Control 과의 모든 통신을 처리한다

■ Relations

- agent 안에서는 Control 이 Presentation·Abstraction 을 연결하고, agent 사이는 Control 끼리 연결된다

■ Constraints

- Abstraction component 와 Presentation component 사이에 직접 연결이 없어야 한다



PAC agent 의 구조 (Control 이 통신의 중심)

핵심 Control 이 모든 통신의 중심이며, Abstraction 과 Presentation 은 서로 직접 대화하지 않는다

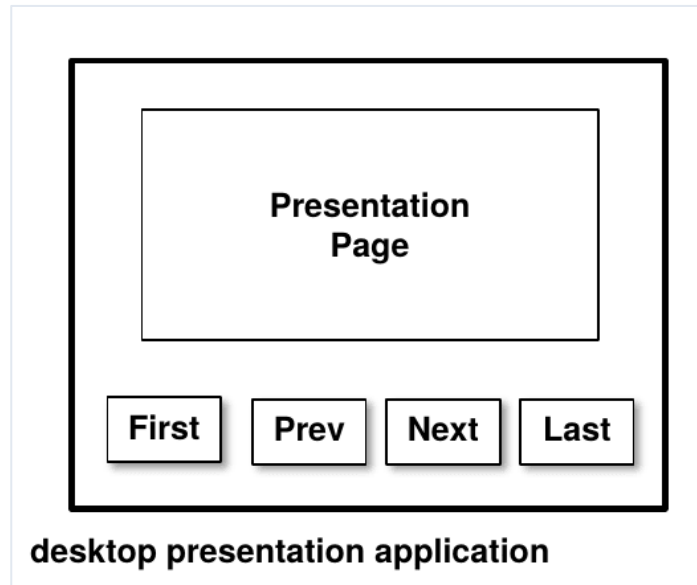
PAC 예제 — Desktop Presentation Application

■ 문제

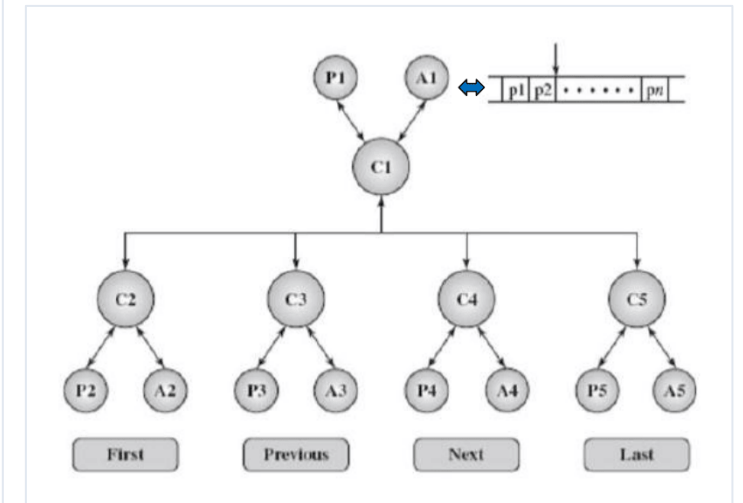
- 그래픽 문서의 presentation page 를 한 번에 한 장씩 탐색·표시하는 애플리케이션
- First, Last, Previous, Next 네 개의 버튼을 제공한다

■ PAC 구성

- 최상위 agent A1 은 데이터 저장소(문서 파일 p1...pn)와 연결된다
- 각 버튼(First, Previous, Next, Last)은 하위 agent(C2~C5)가 담당한다
- 각 agent 는 P(Presentation), C(Control), A(Abstraction)로 구성된다



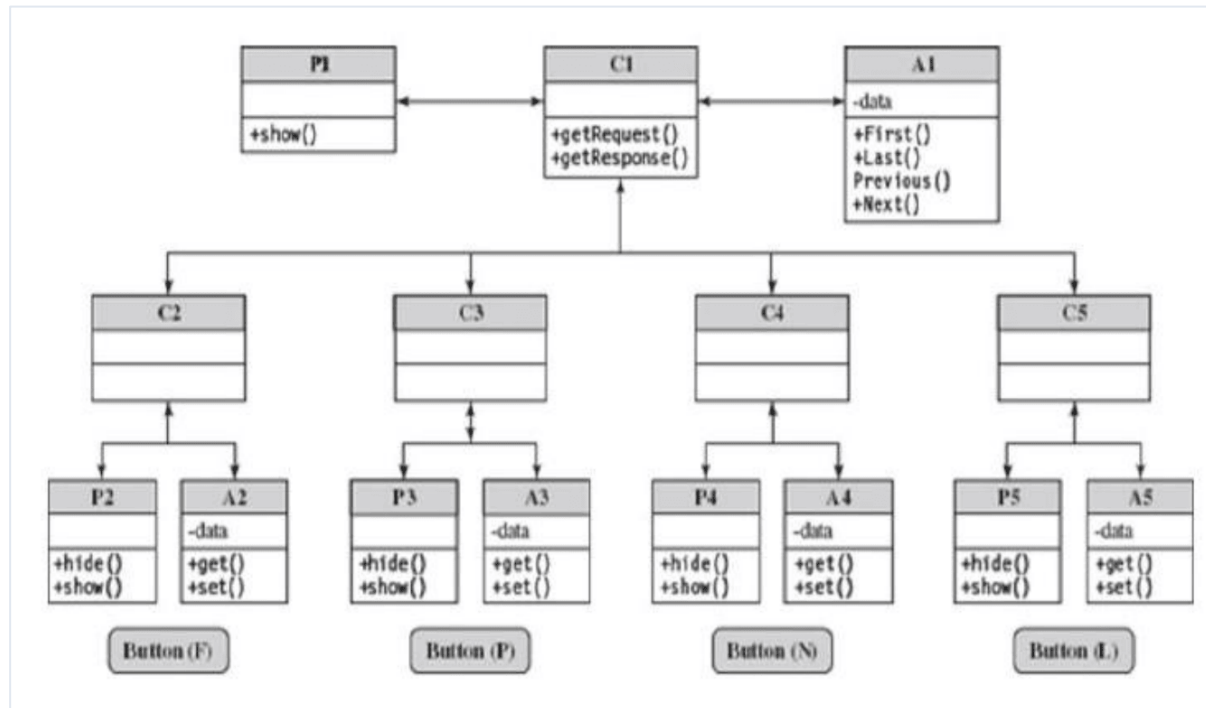
desktop presentation application 화면



PAC agent 계층 (C1 최상위, C2~C5 버튼 agent)

PAC 예제 — Agent 의 클래스 구조

- C1 은 getRequest()/getResponse() 로 A1 의 First()/Last()/Previous()/Next() 를 호출하고, P1 의 show() 로 페이지를 표시한다
- 버튼 agent 의 P(P2~P5)는 hide()/show(), A(A2~A5)는 get()/set() 을 가지며, C2~C5 가 이를 중개한다



PAC agent 들의 클래스 다이어그램: C1 – (C2, C3, C4, C5) – Button(F/P/N/L)

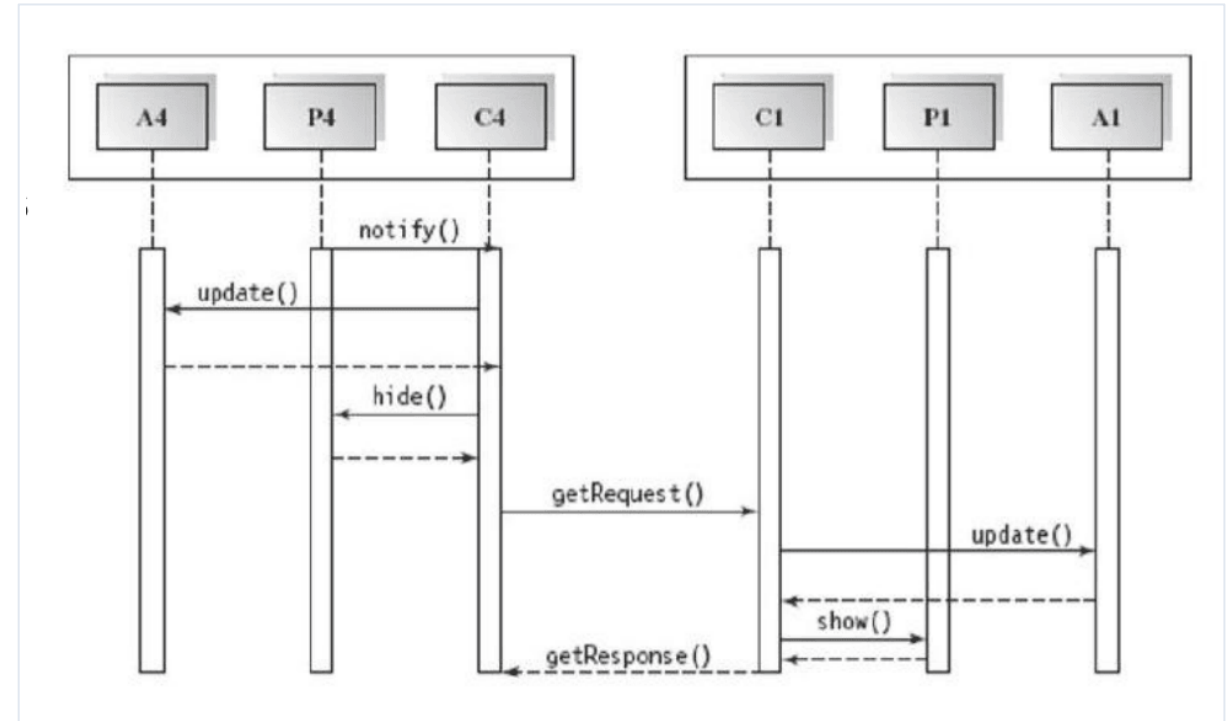
PAC 예제 — Next 버튼을 눌렀을 때의 흐름

■ 가정(Assumption)

- 현재 페이지는 문서의 끝에서 두 번째 페이지이다

■ Next 버튼을 눌러 마지막 페이지를 표시하는 과정

- 1. P4 가 C4 에게 'next' 버튼이 눌렸음을 알린다
- 2. C4 가 A4 에 update 를 보낸다
- 3. C4 가 P4 에게 표시(모양)를 갱신하라고 알린다 → 'next' 버튼을 숨긴다(hide)
- 4. C4 가 최상위 agent 의 C1 에 연락한다
- 5. C1 이 A1 에 update 를 보내 포인터를 다음(마지막) 페이지로 옮긴다
- 6. C1 이 P1 에게 마지막 페이지를 표시하라고 지시한다



Sequence diagram: A4·P4·C4 → C1·P1·A1

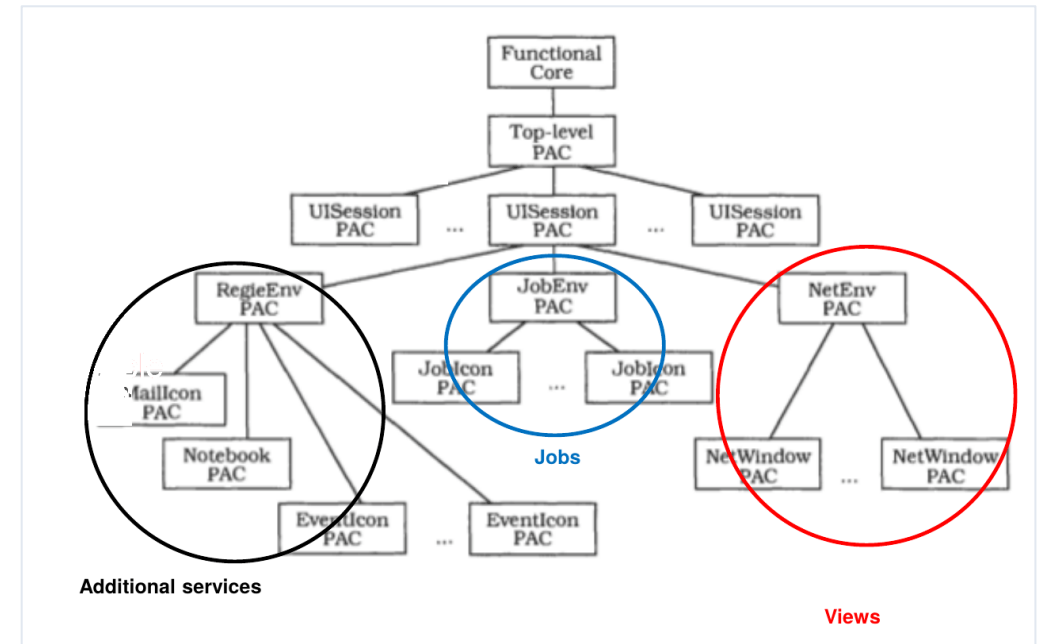
PAC 예제 — Network Traffic Management

■ 시스템의 기능

- 트래픽 데이터 수집(Gathering traffic data)
- 임계값 검사(Threshold checking)
- 로깅(Logging)
- 트래픽 데이터의 통계적 평가
- 트래픽 흐름 시각화
- 전체 네트워크에 대해 사용자가 구성할 수 있는 다양한 view 표시 등

■ PAC 계층

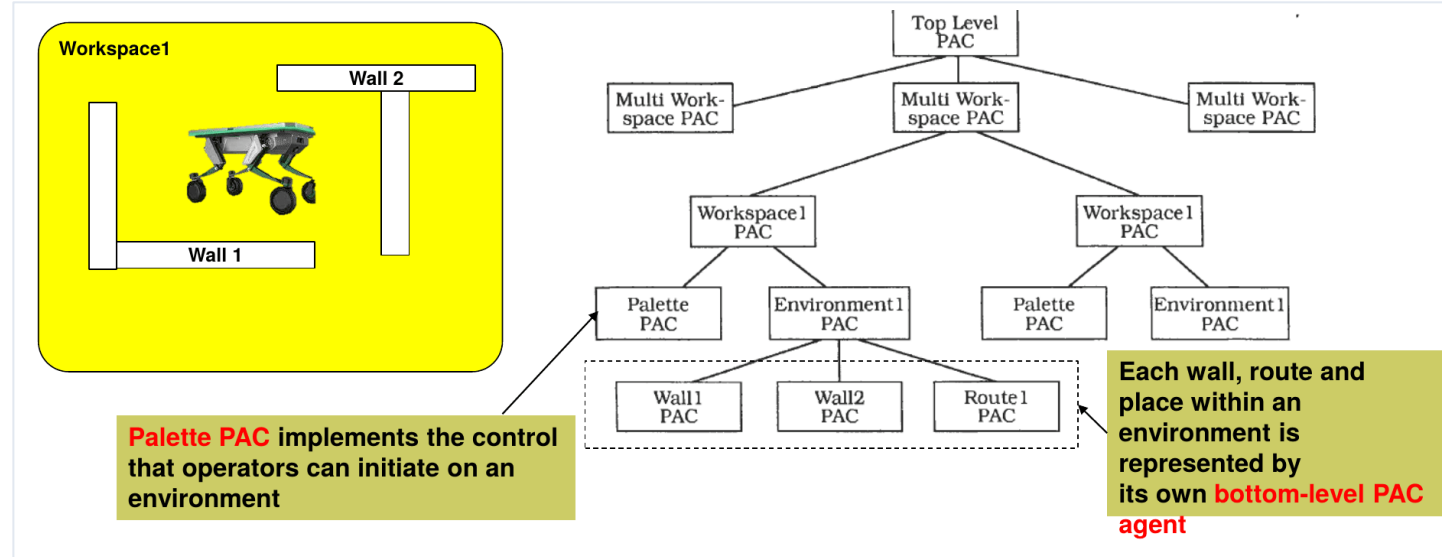
- Functional Core – Top-level PAC – UI Session PAC 아래에 Additional services(RegieEnv), Jobs(JobEnv), Views(NetEnv) agent 가 배치된다



Network Traffic Management 의 PAC agent 계층

PAC 예제 — Mobile Robot

- Mobile Robot 은 폐쇄된 위험 환경 안을 이동하며 운영자(operator)와 상호작용한다
- Top Level PAC 아래에 Multi Workspace PAC, Workspace PAC 이 있고, 그 아래에 Palette PAC 과 Environment PAC 이 있다
- Palette PAC 은 운영자가 환경에 대해 시작할 수 있는 제어(control)를 구현한다
- 환경 안의 각 벽(Wall), 경로(Route), 장소(Place)는 각각 자신만의 최하위(bottom-level) PAC agent 로 표현된다



Mobile Robot 의 workspace 와 PAC agent 계층

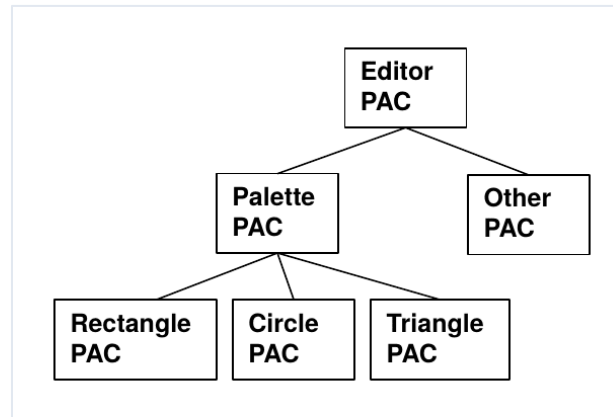
PAC Consequences: 장점과 약점

■ Benefits (장점)

- 관심사의 분리(Separation of concerns): 도메인의 서로 다른 의미 개념(semantic concept)이 별도의 agent 로 표현된다
 - 각 개념마다 전용 데이터 모델과 UI 를 다른 개념과 독립적으로 개발할 수 있다
- 변경·확장 지원: 한 PAC agent 의 Presentation-Abstraction 을 바꿔도 다른 agent 에 영향이 없다
 - 새 agent 를 끼워 넣거나 기존 agent 를 교체하기 쉽다
- multi-tasking 지원: PAC agent 를 서로 다른 thread, process, machine 에 쉽게 분산할 수 있다

■ Weaknesses (약점)

- 시스템 복잡도 증가: 모든 의미 개념을 각각 PAC agent 로 구현하면 시스템 구조가 복잡해질 수 있다
 - 예: Graphics Editor — Editor PAC 아래 Palette PAC, Other PAC, 그 아래 Rectangle/Circle/Triangle PAC
- PAC agent 간 통신 overhead 가 시스템 효율에 영향을 줄 수 있다



Graphics Editor 의 PAC agent 계층

1.6

Hierarchical Architecture

시스템 전체를 계층(hierarchy) 구조로 보고, 하위 모듈이 상위 모듈에 서비스를 제공하는 스타일

- Hierarchical Architecture 개요
- Main-Subroutine Style
- Master-Slave Style
- Layered Style — Android, OSI 7 Layer, Variants(Relaxed Layered, Layering Through Inheritance)
- Virtual Machine — CLR, JVM, KVM, Docker
- Microkernel — Eclipse, Microkernel Pattern [POSA1996]

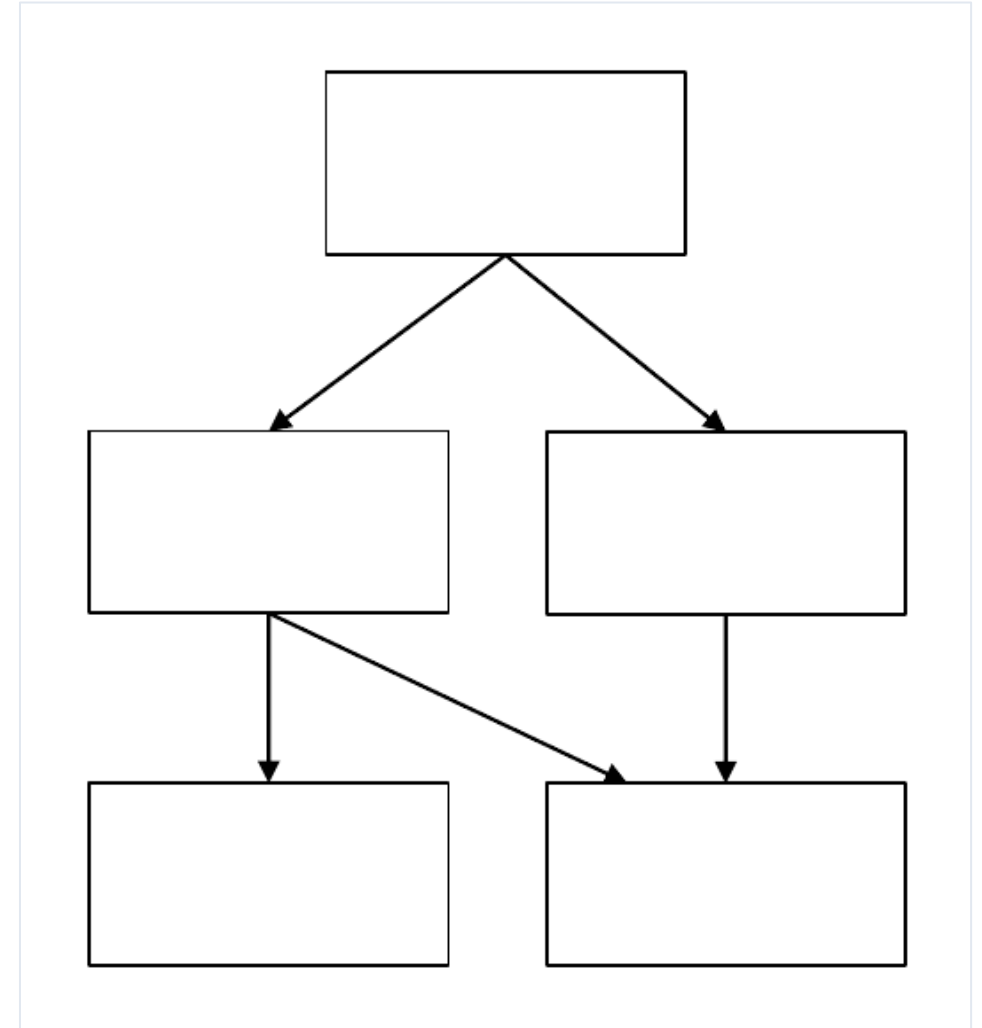
Hierarchical Architecture 개요

■ 계층 구조로 시스템 보기

- Hierarchical architecture 는 시스템 전체를 하나의 계층(hierarchy) 구조로 본다
- 소프트웨어 시스템을 계층의 서로 다른 level 에 있는 논리적 모듈로 분해한다
- 하위(lower-level) 모듈은 인접한 상위(upper-level) 모듈에 서비스를 제공하고, 상위 모듈은 하위의 메서드나 프로시저를 호출한다
- 서로 다른 level 의 모듈은 명시적 또는 암묵적 메서드 호출(call-and-return)로 연결된다

■ Hierarchical Architecture 의 Style

- Main-Subroutine
- Master-Slave
- Layered
 - Virtual Machine
 - Microkernel



계층 구조: 상위 모듈이 하위 모듈을 호출한다

Main-Subroutine Style — Elements · Relations · Constraints

■ 정의

- 시스템이 원하는 기능에 따라 subroutine 으로 계층적으로 정제(refine)되어 분해된다

■ Elements

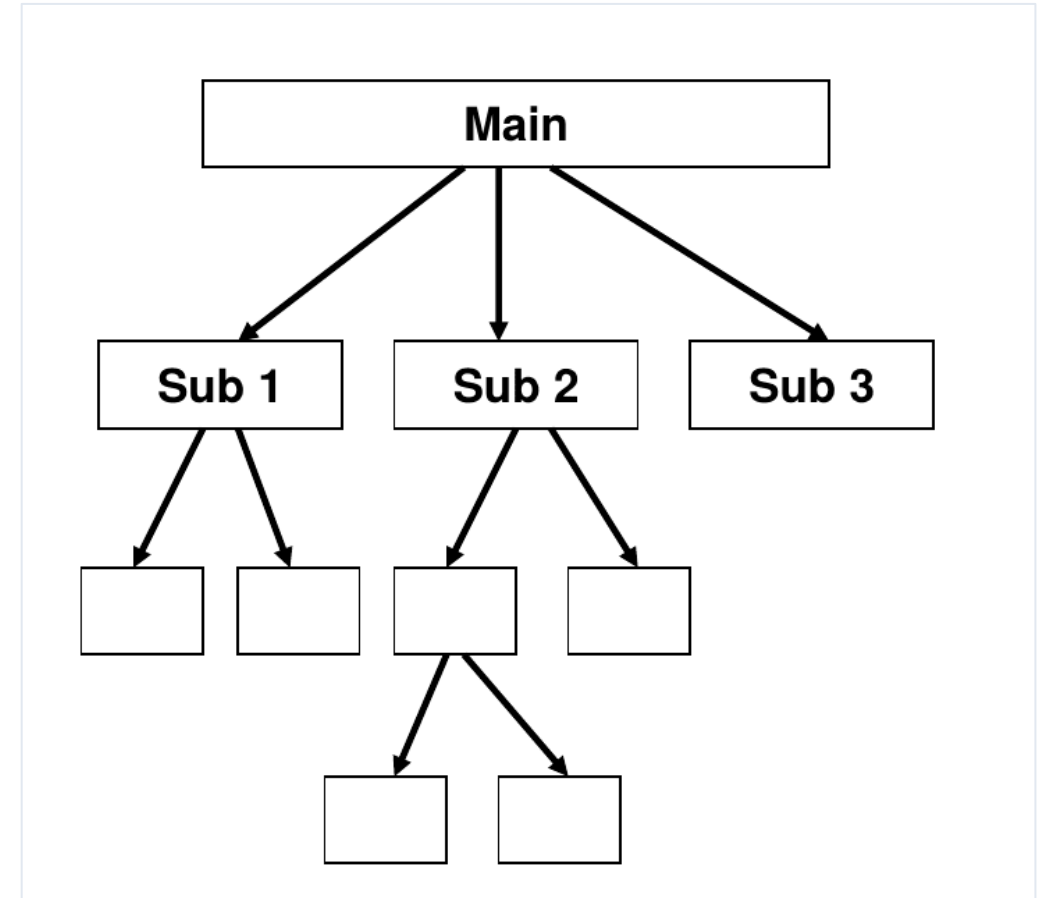
- Components: Procedure(또는 function)
- Connectors: Procedure call(또는 function call)

■ Relations

- Main 이 Sub 1, Sub 2, Sub 3 을 호출하고, 각 Sub 가 다시 하위 procedure 를 호출한다

■ Constraints

- 중앙 집중(Centralized)
- 단일 제어 흐름(a single thread of control)
- 계층의 각 component 는 부모로부터 제어를 받아 자식에게 넘겨준다

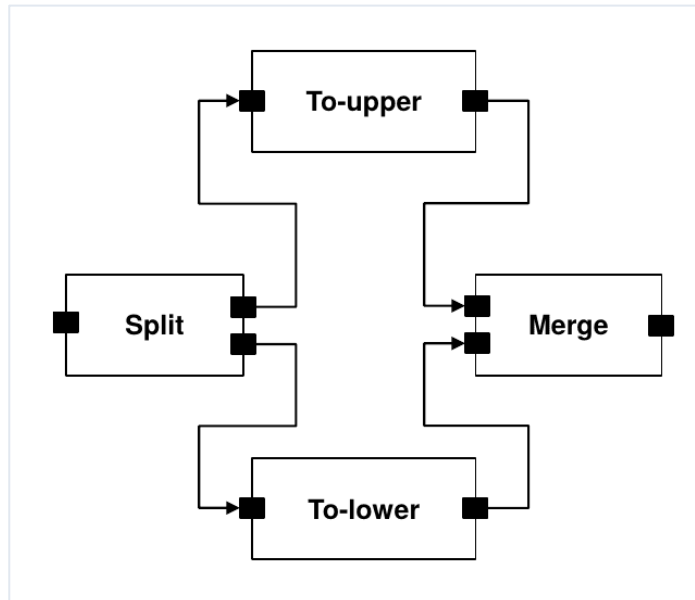


Main – Sub 1/2/3 – 하위 procedure 의 호출 계층

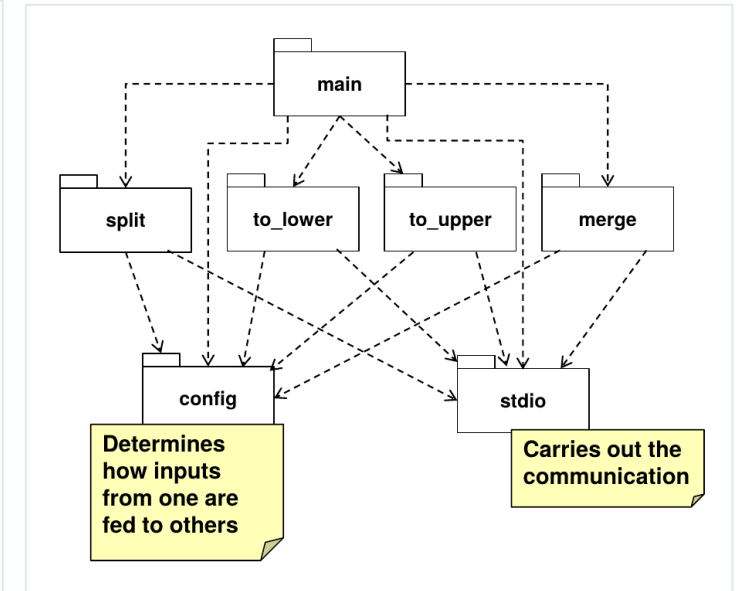
Main-Subroutine 예제 — Pipe and Filter 와의 비교

■ 같은 시스템, 두 가지 style

- Pipe and Filter style: Split 이 입력을 To-upper 와 To-lower 로 나누고, Merge 가 합친다. 각 filter 는 pipe 로 연결된다
- Main-Subroutine style: main 이 split, to_lower, to_upper, merge 를 호출하며, 이들은 config 와 stdio 를 공통으로 사용(uses)한다
 - config: 한 모듈의 입력이 다른 모듈로 어떻게 전달되는지 결정한다
 - stdio: 실제 통신(입출력)을 수행한다



Pipe and Filter Style



Main-Subroutine style (점선: uses)

Main-Subroutine Consequences: 장점과 약점

■ Benefits (장점)

- 단순함(Simplicity): 비교적 단순하고 이해하기 쉽다
· 특히 작거나 중간 규모의 프로그램에 적합하다
 - 제어 흐름이 선형(linear)이어서 프로그램 로직을 따라가기 쉽다
- 테스트 용이(Ease of testing): 모듈 구조 덕분에 단위 테스트가 쉽다
 - 각 subroutine 을 독립적으로 테스트할 수 있어 전체 테스트가 수월하다

■ Weaknesses (약점)

- 전역으로 공유되는 데이터(globally shared data)가 있어 취약하다
- 강한 결합(tight coupling) 때문에 변경의 파급 효과(ripple effect)가 커질 수 있다

핵심 이해하기 쉽고 테스트하기 좋지만, 전역 데이터와 강한 결합이 변경에 취약하게 만든다

Master-Slave Style — Elements · Relations

■ 정의

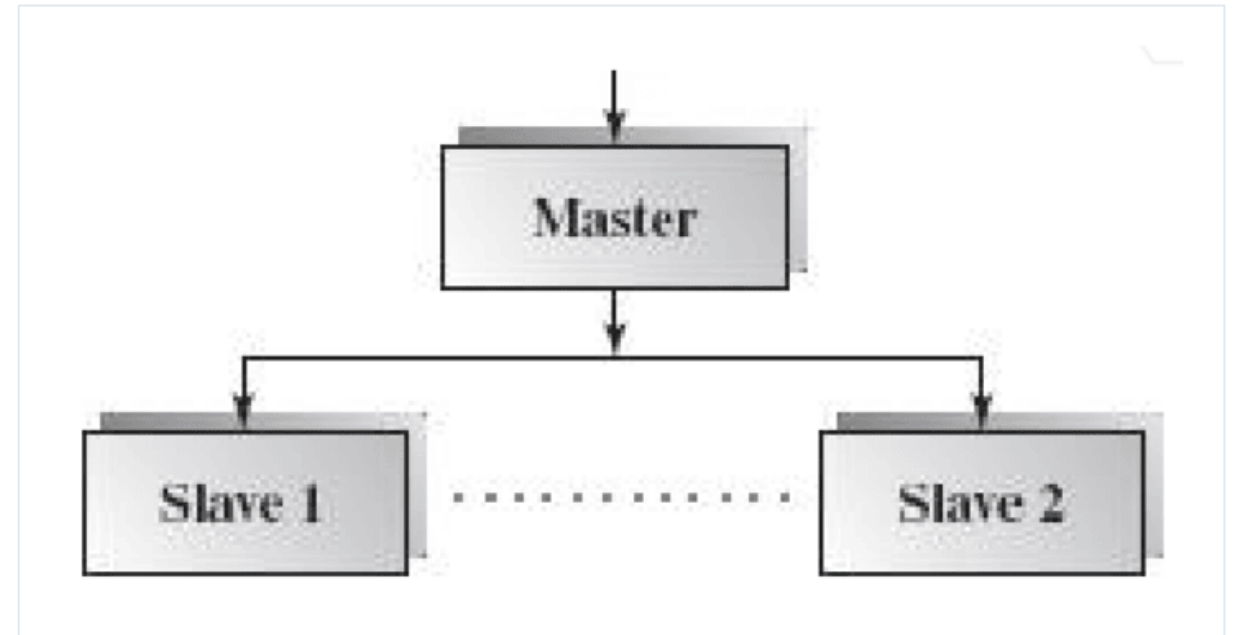
- fault tolerance(결함 허용)와 시스템 신뢰성(reliability)을 지원하는 architecture style

■ Elements

- Slave: Master에게 복제된(replicated) 서비스를 제공한다
 - 같은 기능을 서로 다른 알고리즘으로 수행하거나, 완전히 다른 기능을 수행할 수 있다
- Master: 복제된 서비스들의 호출을 구성(configure)한다
 - 모든 slave로부터 결과를 돌려받고, 일정한 선택 전략(selection strategy)에 따라 하나의 결과를 고른다

■ Relations

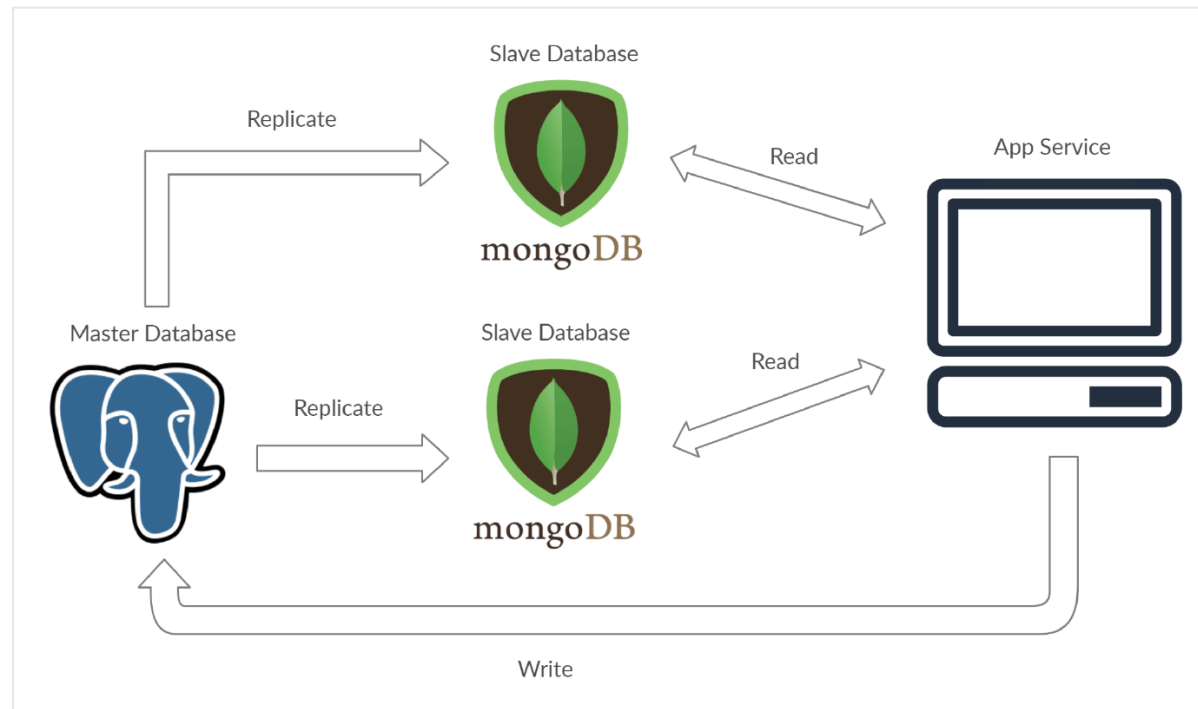
- Master가 Slave 1, Slave 2, ...를 호출하고 결과를 회수한다



Master 와 Slave 1, Slave 2, ...

Master-Slave 예제 — Database Replication

- Master Database(PostgreSQL)가 쓰기(Write)를 받고, 데이터를 Slave Database(mongoDB)들로 복제(Replicate)한다
- App Service 는 Slave Database 에서 읽기(Read)를 수행하고, 쓰기는 Master Database 로 보낸다



Master Database – Slave Database 복제 구조와 App Service 의 Read/Write

Master-Slave Consequences: 장점과 약점

■ Benefits (장점)

- Fault tolerance: slave 가 고장 나면 master 가 작업을 다른 slave 에 재할당해 시스템이 최소한의 중단으로 계속 동작한다
- 병렬 컴퓨팅(Parallel computing): 모든 slave 를 병렬로 실행할 수 있다
- 확장성(Scalability): slave 를 추가하면 늘어난 작업량을 처리할 수 있다
- 계산의 정확성(Accuracy): 같은 작업을 여러 구현에 맡기므로, 다수결(majority vote) 등으로 부정확한 결과를 쉽게 걸러낼 수 있다

■ Weaknesses (약점)

- 단일 장애점(Single Point of Failure): master 가 고장 나면 시스템 전체 동작이 영향을 받고, 작업 할당·관리가 제대로 되지 않을 수 있다
- 통신 overhead: master 와 slave 사이의 통신, 특히 분산 환경에서의 통신이 overhead 를 만든다
 - 병렬 처리로 얻은 성능 이득을 상쇄할 수 있다

Layered Style — Elements · Relations · Constraints

■ Elements

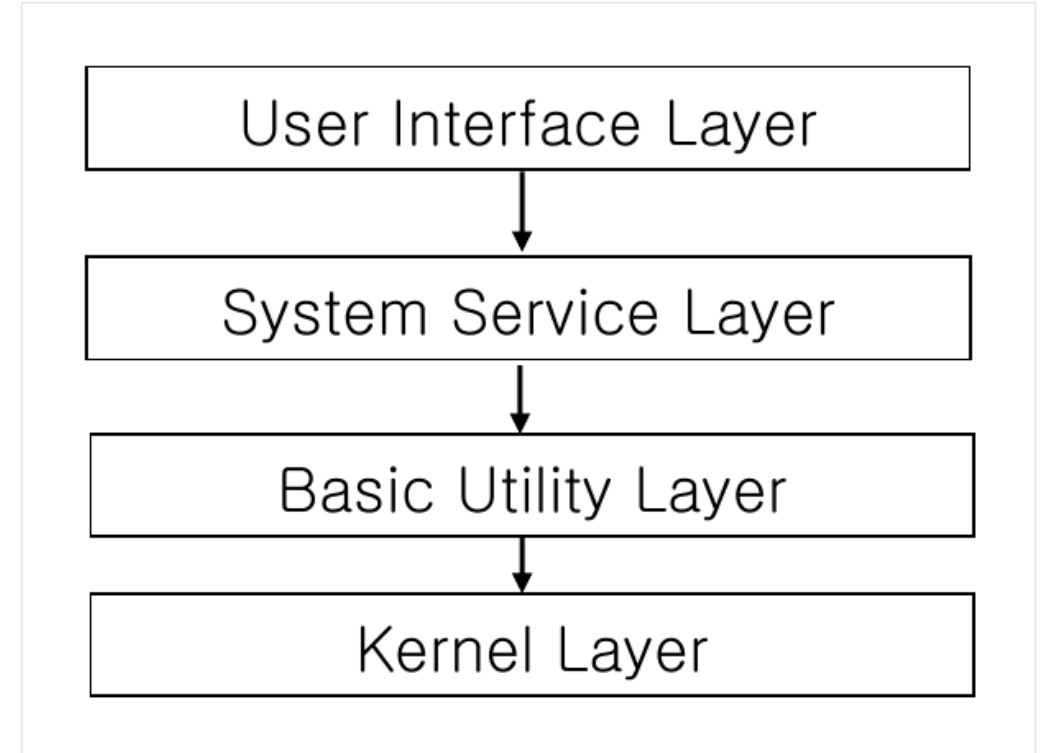
- Layer: 같은 수준(level)에 있는 component 들의 모음

■ Relations

- Allowed-to-use: 더 일반적인 depends-on 관계를 특수화한 것으로, 상위 layer 가 하위 layer 를 사용할 수 있음을 뜻한다

■ Constraints

- allowed-to-use 관계는 순환(circular)하면 안 된다
- layer 는 최소 두 개 이상이어야 한다

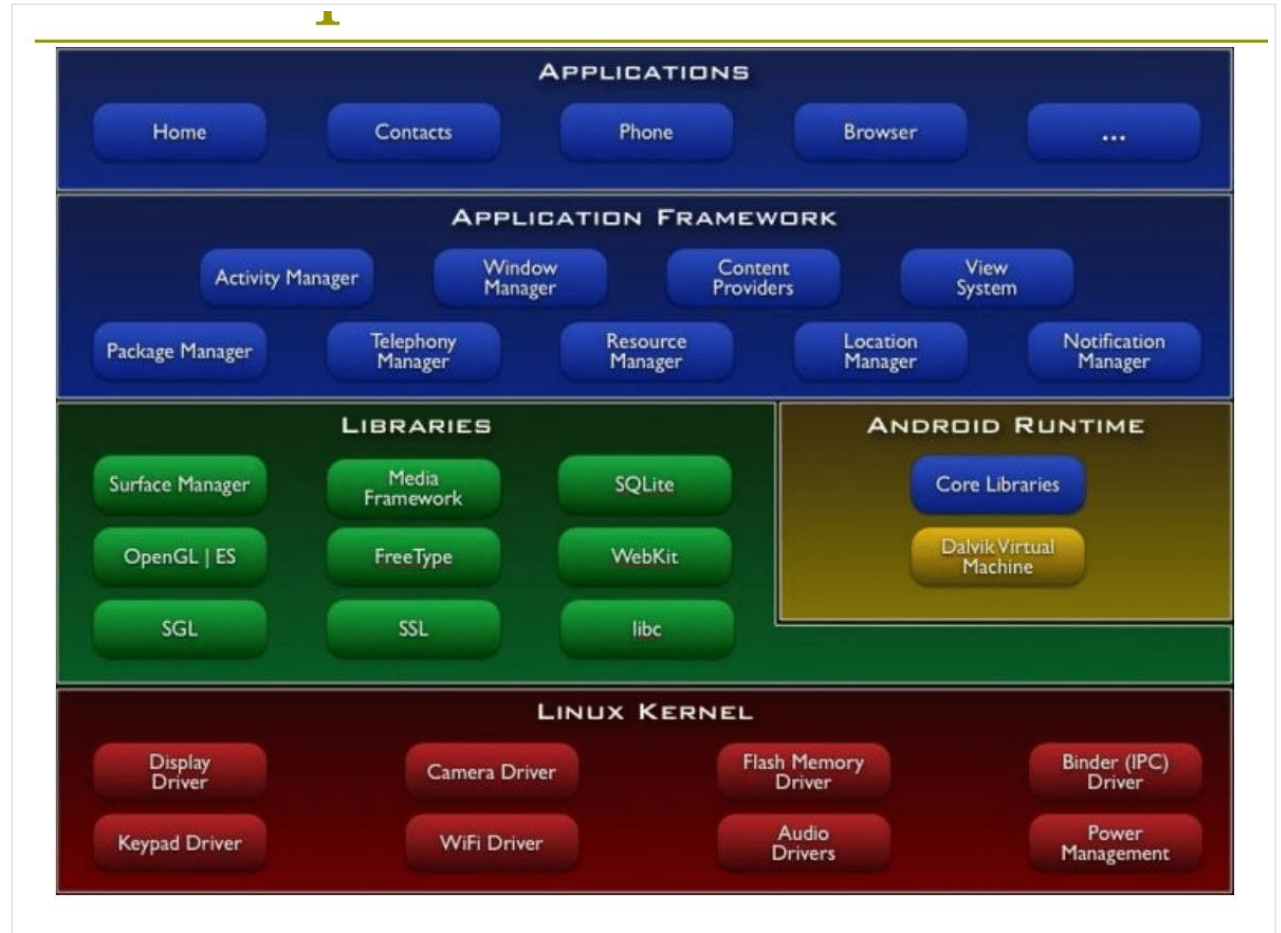


User Interface – System Service – Basic Utility – Kernel Layer

핵심 각 layer 는 바로 아래 layer 의 서비스만 사용하고, 사용 관계는 한 방향으로만 흐른다

Layered 예제 — Android Architecture

- Applications: Home, Contacts, Phone, Browser 등
- Application Framework: Activity Manager, Window Manager, Content Providers, View System 등
- Libraries(Surface Manager, SQLite, WebKit, OpenGL 등)와 Android Runtime(Core Libraries, Dalvik VM)
- Linux Kernel: Display/Camera/WiFi Driver, Power Management 등



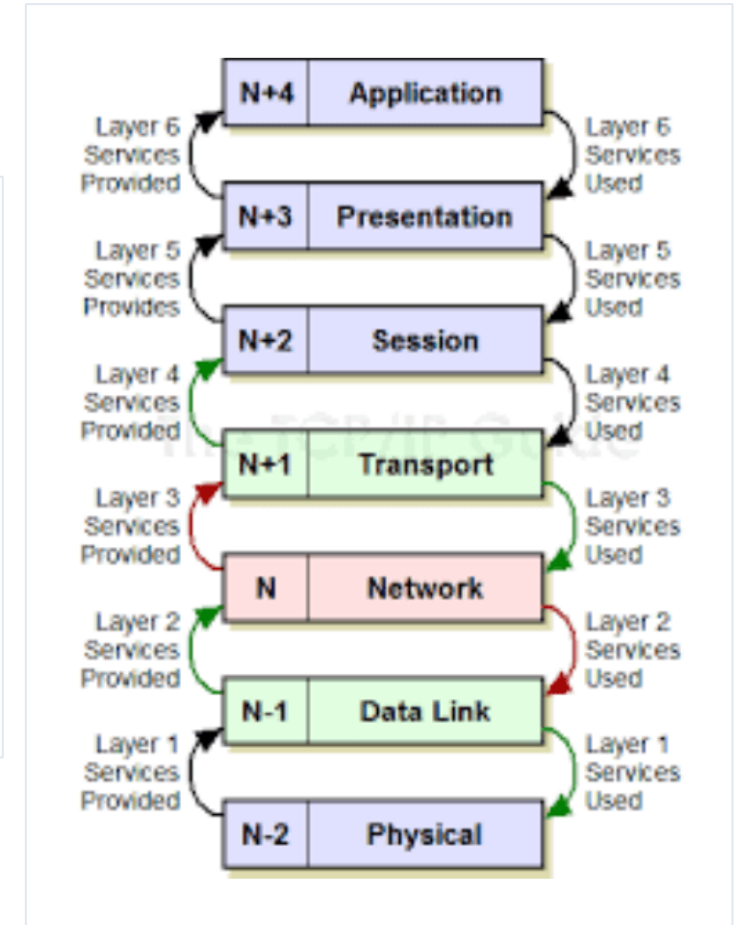
Android 플랫폼의 계층 구조

Layered 예제 — OSI 7 Layer Architecture

- 각 layer 는 통신의 특정 측면을 다루며, 바로 아래 layer 의 서비스를 사용한다
- 7 Application → 6 Presentation → 5 Session → 4 Transport → 3 Network → 2 Data Link → 1 Physical
- layer N 은 layer N-1 이 제공(provide) 하는 서비스를 사용(use)하고, layer N+1 에 서비스를 제공한다

7	Application Layer	Human-computer interaction layer, where applications can access the network services
6	Presentation Layer	Ensures that data is in a usable format and is where data encryption occurs
5	Session Layer	Maintains connections and is responsible for controlling ports and sessions
4	Transport Layer	Transmits data using transmission protocols including TCP and UDP
3	Network Layer	Decides which physical path the data will take
2	Data Link Layer	Defines the format of data on the network
1	Physical Layer	Transmits raw bit stream over the physical medium

OSI 7 Layer 의 역할 (출처: imperva.com)



Layer 간 Services Provided / Used

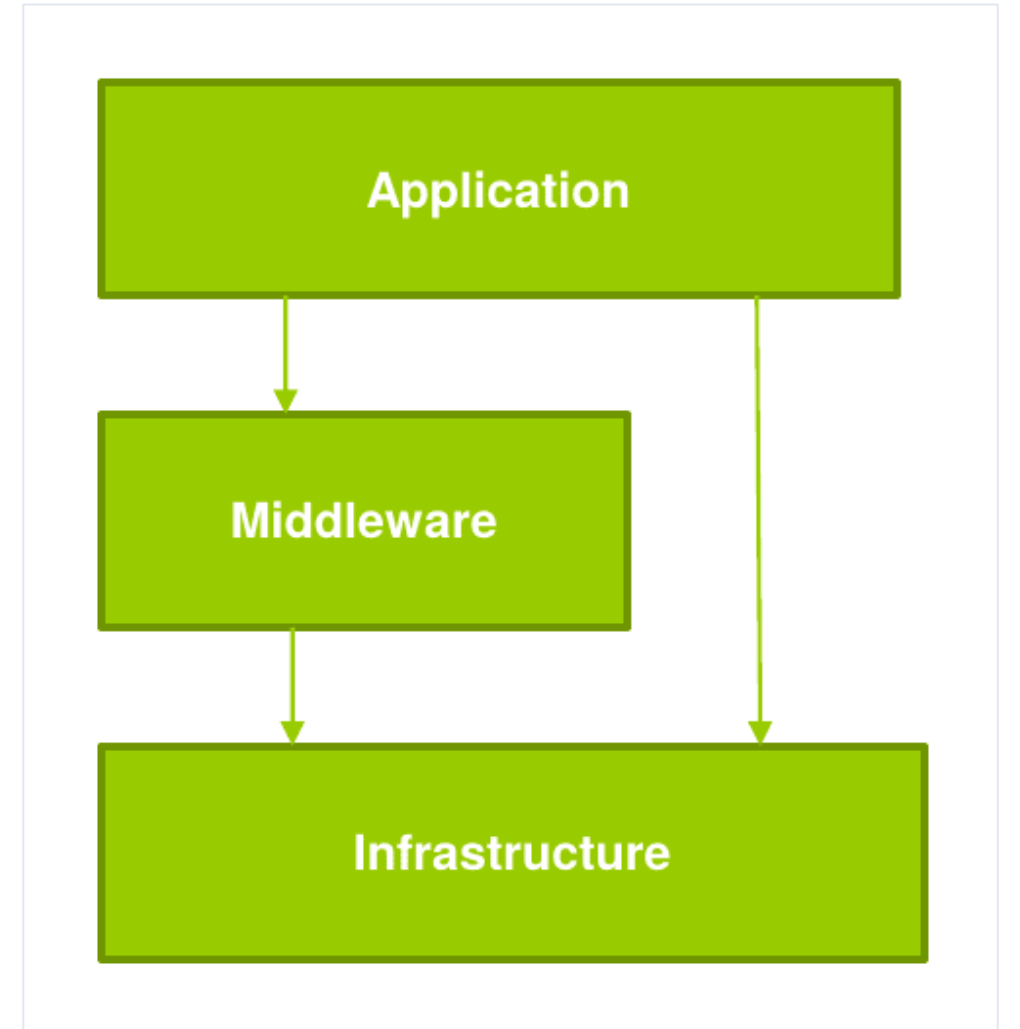
Layered Variants (1) — Relaxed Layered System

■ Relaxed Layered System

- 상위 layer 가 바로 아래 layer 뿐 아니라 그 아래 layer(예: Application → Infrastructure)도 직접 사용할 수 있게 허용한다
- 성능(Performance) vs 유지보수성(Maintainability) 의 trade-off
- 예: Unix Operating System, X Window System

■ 왜 허용하는가

- Infrastructure 시스템은 Application 시스템보다 변경이 드물고, 유지보수성보다 성능이 더 중요하기 때문이다



Application – Middleware – Infrastructure (Application 이 Infrastructure 를 직접 사용)

Layered Variants (2) — Layering Through Inheritance

■ Layering Through Inheritance

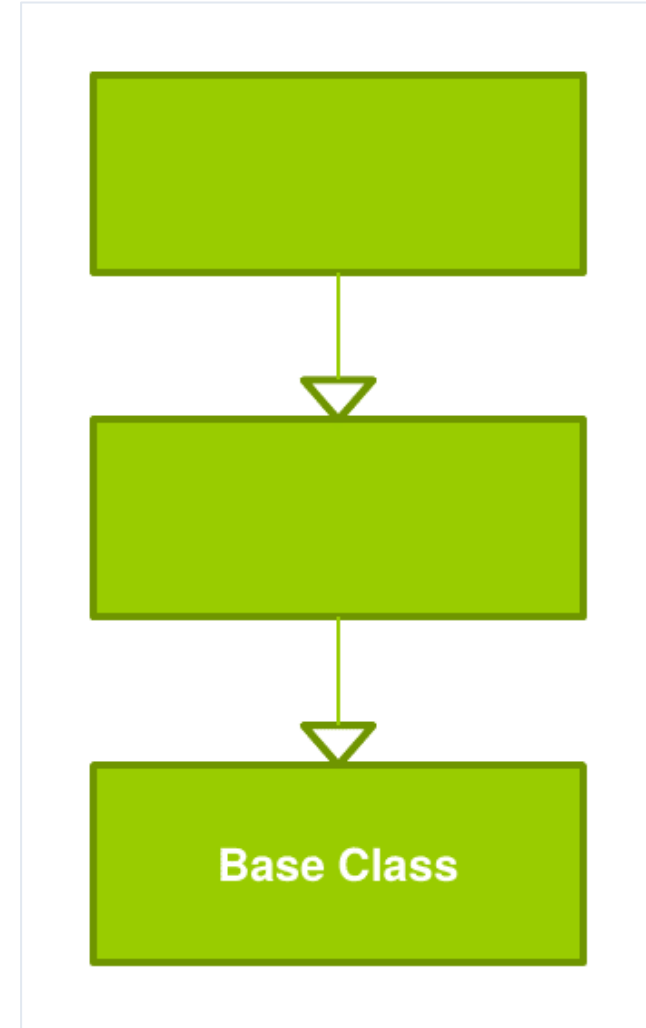
- 하위 layer 의 서비스를 요청하는 상위 layer 가 하위 layer 의 구현을 상속(inherit)한다

■ Benefits

- 상위 layer 가 필요에 따라 하위 layer 의 서비스를 수정할 수 있다

■ Weaknesses

- base class 의 데이터 배치(data layout)가 바뀌면 모든 subclass 를 다시 컴파일해야 한다



상속으로 이어진 layer: Base Class 를 상위 layer 가 상속

Layered Consequences: 장점과 약점

■ Benefits (장점)

- 독립성(Independence): 각 component 를 서로 다른 일정으로 독립적으로 배포·유지보수·갱신할 수 있다
 - 팀원들이 최소한의 의존성으로 서로 다른 부분을 병렬로 작업할 수 있다
 - component 를 서로 독립적으로 테스트할 수 있다
- 보안(More secure): 각 layer 가 다른 layer 로부터 내부 정보를 숨길 수 있다
- 재사용성(Reusability): 각 layer 는 응집도가 높고 하위 layer 에만 결합되어 재사용·교체·대체가 쉽다

■ Weaknesses (약점)

- layer 가 너무 많으면 성능이 떨어진다
- layer 의 알맞은 granularity(입도)를 정하기 어렵다
 - layer 가 너무 적으면 재사용성·변경 용이성·이식성이라는 패턴의 잠재력을 충분히 살리지 못한다
 - layer 가 너무 많으면 불필요한 복잡성과 overhead 가 생긴다
- 복잡성을 더하므로 단순한 애플리케이션에는 쓸 수 없다

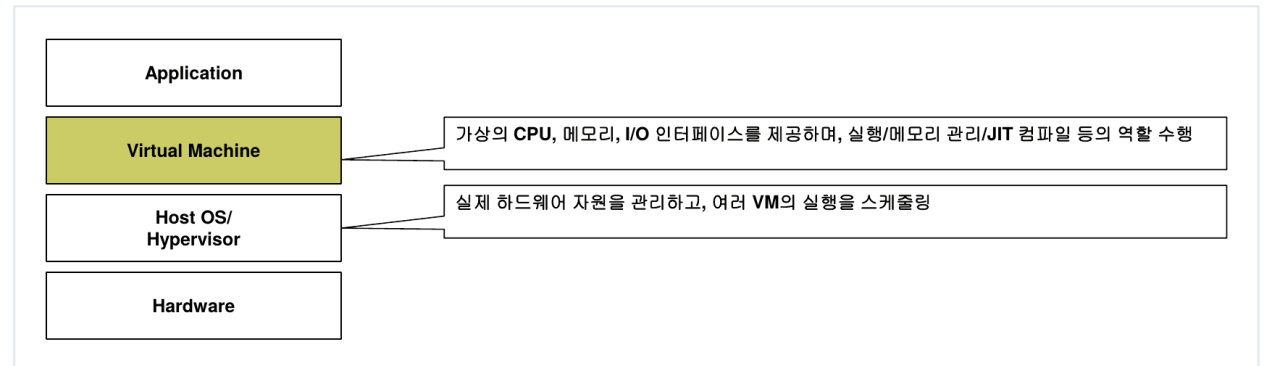
Virtual Machine — 개요와 Elements

■ Virtual Machine 이란

- 하드웨어나 운영체제 위에서 가상의 실행 환경 (Virtual Execution Environment)을 제공하는 소프트웨어 추상화 계층

■ Elements (계층)

- Application: VM 위에서 실행되는 응용 프로그램
- Virtual Machine: 가상의 CPU, 메모리, I/O 인터페이스를 제공하며 실행, 메모리 관리, JIT 컴파일 등의 역할을 수행한다
- Host OS / Hypervisor: 실제 하드웨어 자원을 관리하고, 여러 VM의 실행을 스케줄링한다
- Hardware: 실제 물리 자원



Application – Virtual Machine – Host OS/Hypervisor – Hardware

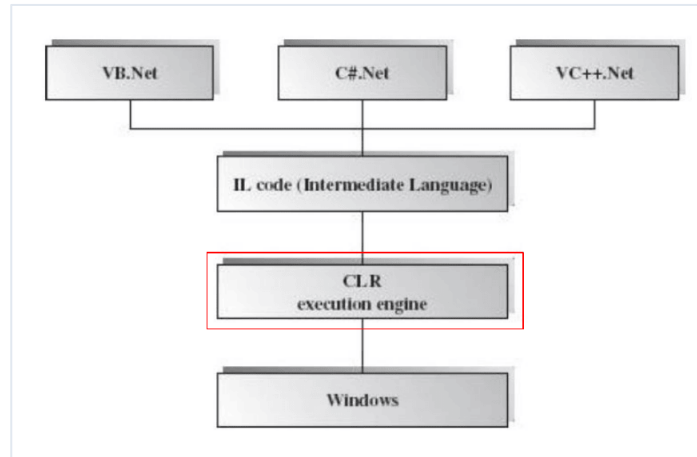
Virtual Machine 예제 — CLR 과 JVM

■ CLR (Microsoft .NET)

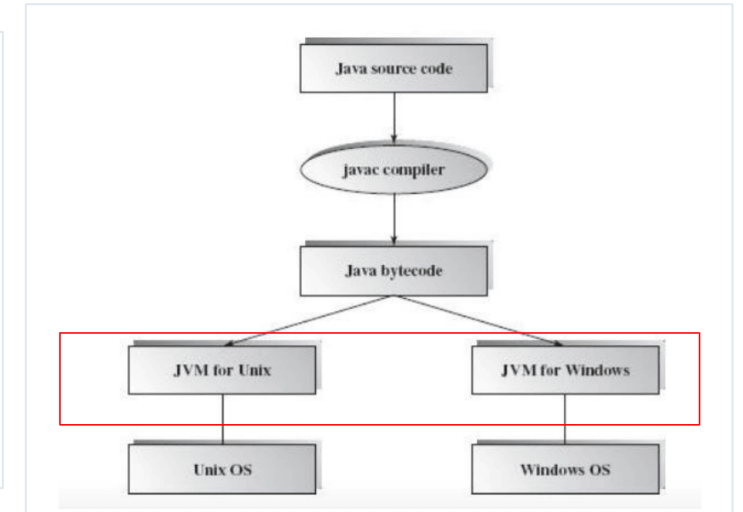
- Common Language Runtime: 하나의 중간 언어(IL)를 사용해 VB.NET, VC.NET, C# 등 여러 모듈을 통합하는 virtual machine 역할을 한다
- CLR execution engine 이 IL code 를 Windows 위에서 실행한다

■ Java Virtual Machine (JVM)

- 컴파일 시스템(예: javac)이 만든 Java bytecode 와 기타 Java 내부 코드는 JVM 을 지원하는 어떤 운영체제에서도 실행할 수 있다
- JVM for Unix / JVM for Windows 가 각 OS 위에서 같은 bytecode 를 실행한다



CLR: VB/C#/VC++ → IL code → CLR → Windows



JVM: Java source → javac → bytecode → JVM → OS

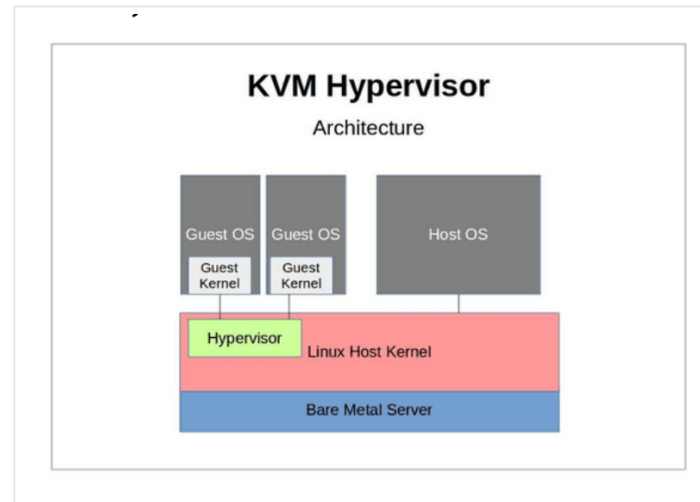
Virtual Machine 예제 — KVM 과 Docker

■ KVM

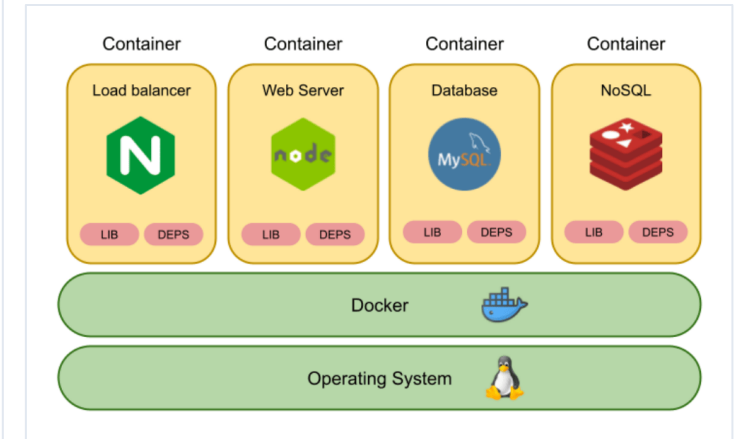
- 커널 기반 가상 머신(Kernel-based Virtual Machine): 물리적 Linux 시스템에 설치하여 가상 머신을 생성할 수 있는 소프트웨어 기능
- OS 수준 가상화와 완전한 하드웨어 가상화를 수행한다
- Bare Metal Server – Linux Host Kernel(Hypervisor) 위에서 Guest OS 들이 실행된다

■ Docker 기반 Microservices

- 서버리스 및 컨테이너 기반의 가벼운 VM 실행 환경
- Operating System – Docker 위에 Load balancer, Web Server, Database, NoSQL 등이 각각 Container 로 실행된다



KVM Hypervisor Architecture



Docker Container 기반 실행 환경

Virtual Machine Consequences: 장점과 약점

■ Benefits (장점)

- 이식성(Portability)과 machine platform 독립성
- 소프트웨어 개발의 단순화
- 재해(disaster) 상황의 동작 모델을 시뮬레이션할 수 있다

■ Weaknesses (약점)

- 인터프리터 방식의 특성 때문에 실행이 느리다
- 새 계층이 추가되어 overhead 가 늘어난다

핵심 플랫폼 독립성을 얻는 대신 실행 속도와 계층 overhead 를 지불한다

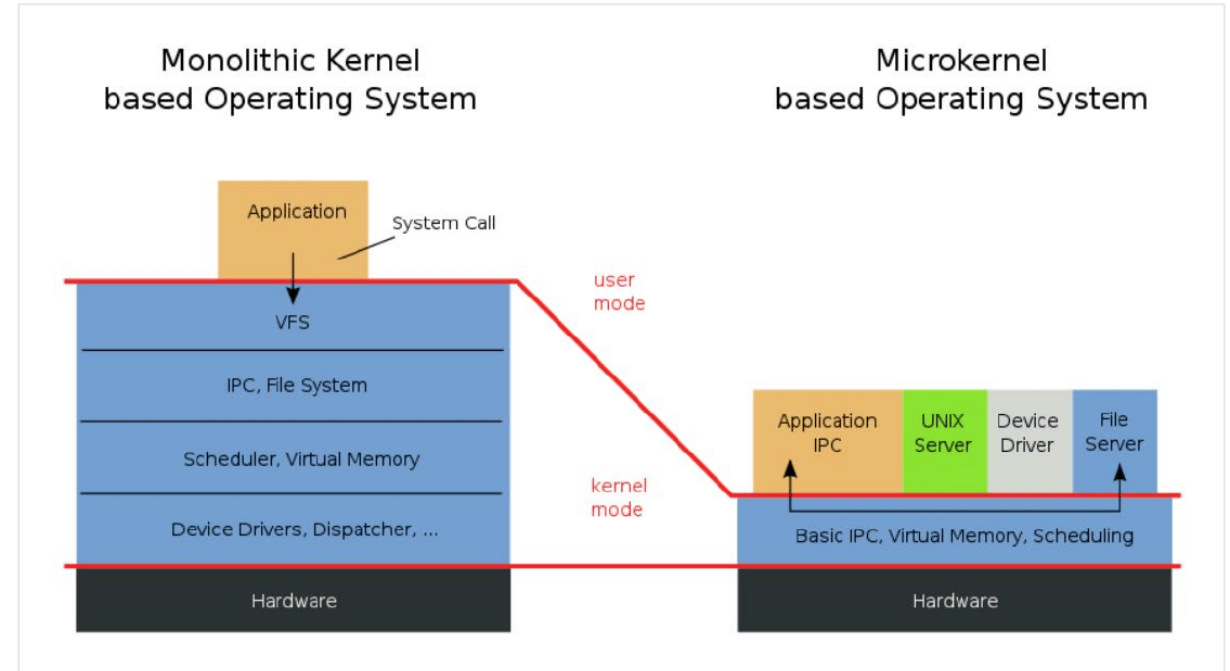
Microkernel — 개요

■ Microkernel 이란

- 원래 microkernel 은 운영체제(OS)를 구현하는 데 필요한 메커니즘을 제공할 수 있는 거의 최소한 (near-minimum)의 소프트웨어를 뜻한다

■ Monolithic Kernel vs Microkernel 기반 OS

- Monolithic: VFS, IPC, File System, Scheduler, Virtual Memory, Device Driver 등이 모두 kernel mode 에서 동작한다
- Microkernel: kernel 에는 Basic IPC, Virtual Memory, Scheduling 만 남기고 Application IPC, UNIX Server, Device Driver, File Server 는 user mode 로 올린다



Monolithic Kernel 기반 OS 와 Microkernel 기반 OS (출처: wikipedia.org/wiki/Microkernel)

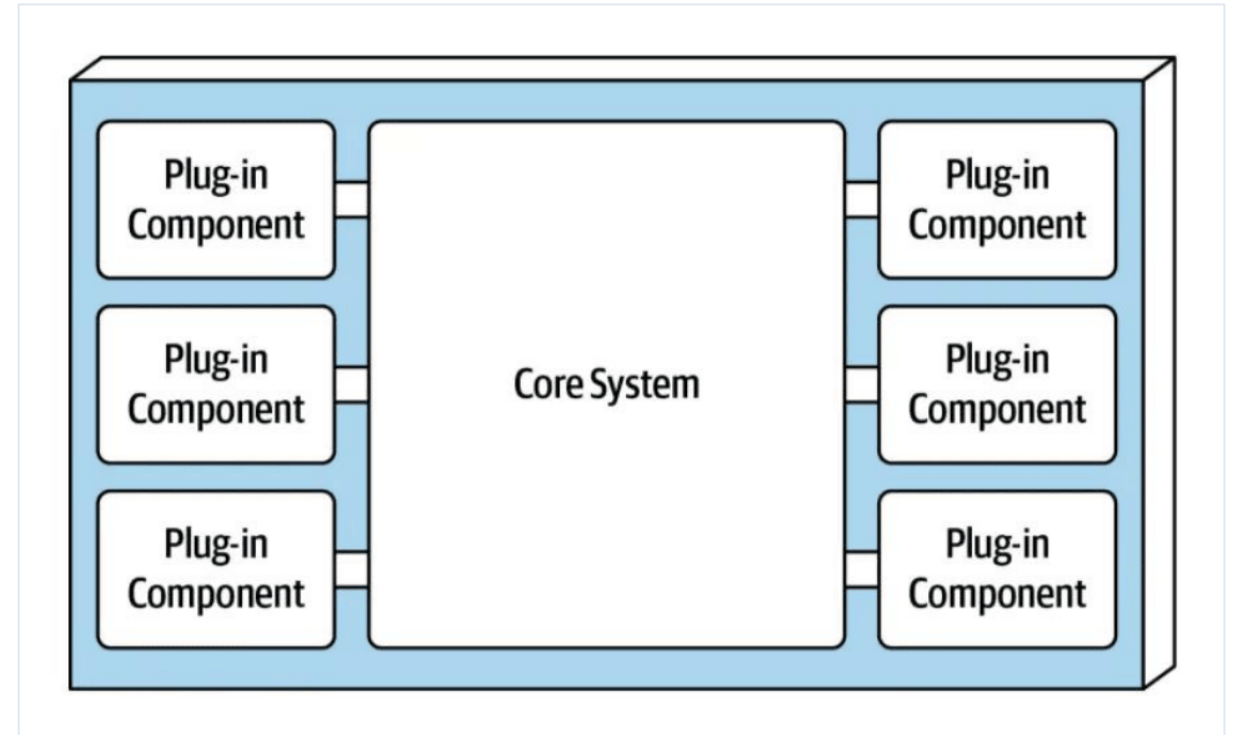
Microkernel Style — Elements · Constraints

■ Elements

- Core System: 시스템을 실행하는 데 필요한 최소한의 기능
- Plug-in Component: 독립적으로 동작하는 component 로, 특화된 처리나 부가 기능을 담아 core system 을 강화·확장한다

■ Constraints

- 최소성(Minimality): microkernel 자체는 필수 기능만 담아 최소한으로 유지해야 한다

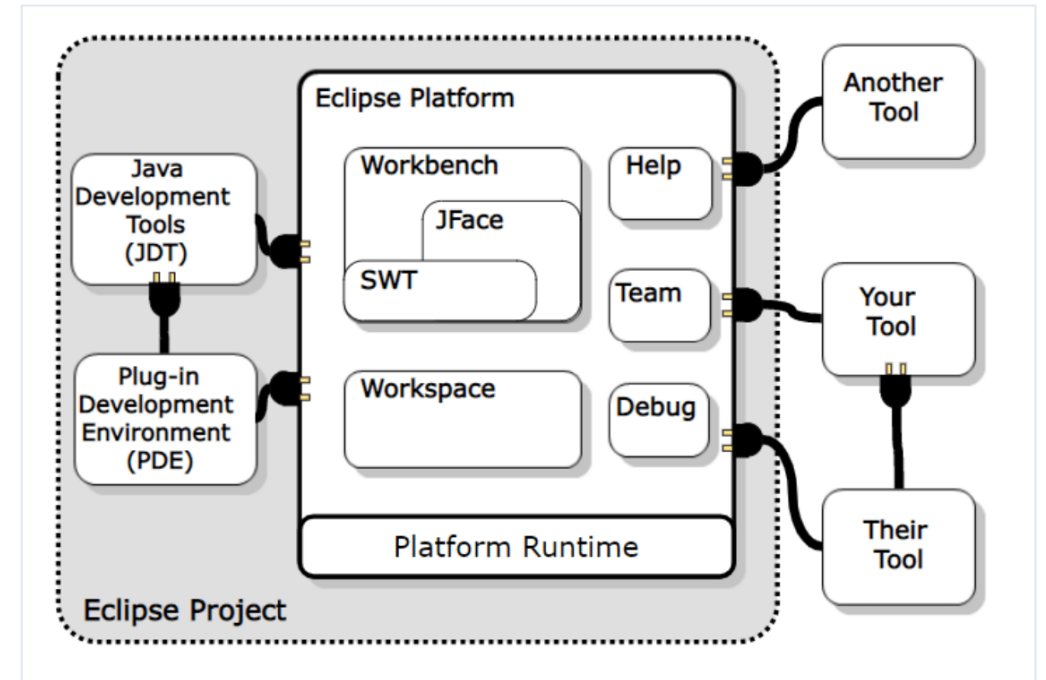


Core System 과 Plug-in Component

핵심 핵심은 최소한으로, 나머지는 plug-in 으로 — 기능 확장이 core 에 영향을 주지 않는다

Microkernel 예제 — Eclipse Architecture

- Eclipse Platform: Platform Runtime 위에 Workbench(JFace, SWT), Workspace, Help, Team, Debug 가 있다
- Java Development Tools(JDT), Plug-in Development Environment(PDE) 는 platform 에 붙는 plug-in 이다
- Another Tool, Your Tool, Their Tool 처럼 외부 도구도 같은 방식으로 plug-in 으로 추가된다



Eclipse Project 의 plug-in 구조

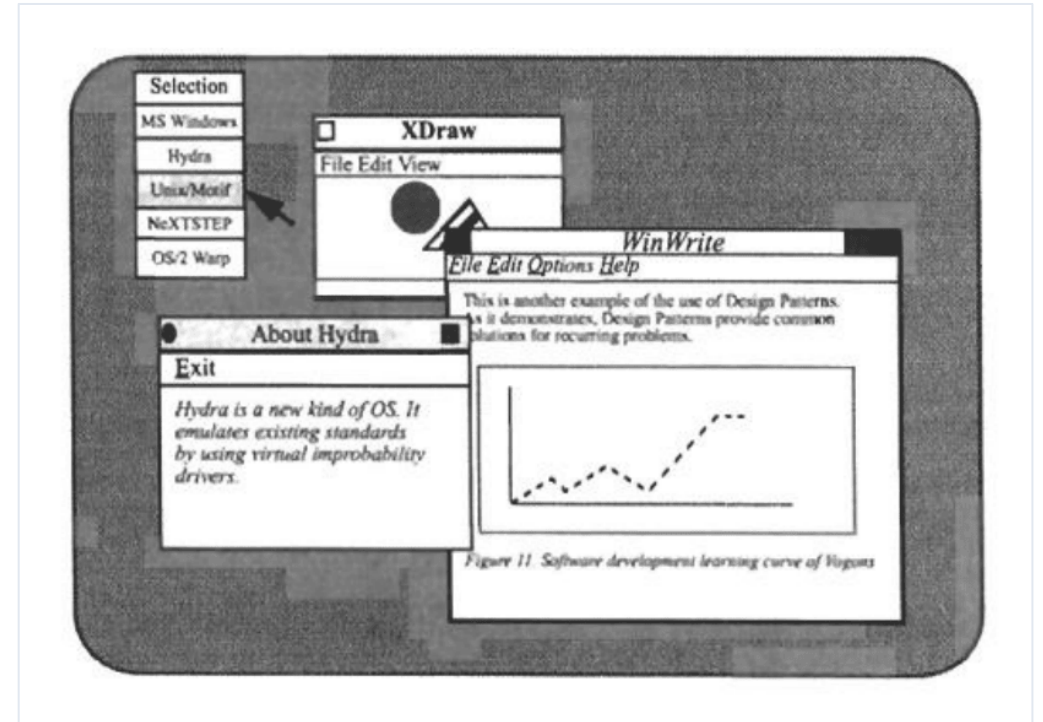
Microkernel Pattern [POSA1996] — 문제 상황

■ Problem — 새 OS 'Hydra' 개발

- 데스크톱 컴퓨터용 새 운영체제 Hydra 를 개발한다
- 관련된 하드웨어 플랫폼으로 이식(portable)할 수 있어야 한다
- Windows, UNIX 등 다른 인기 OS 용으로 작성된 애플리케이션을 실행할 수 있어야 한다
- Hydra 는 현재 실행 중인 모든 애플리케이션을 자신의 main window 안에 표시한다

■ 출처

- [POSA1996] Buschmann 외, Pattern-Oriented Software Architecture – A System of Patterns, Wiley 1996

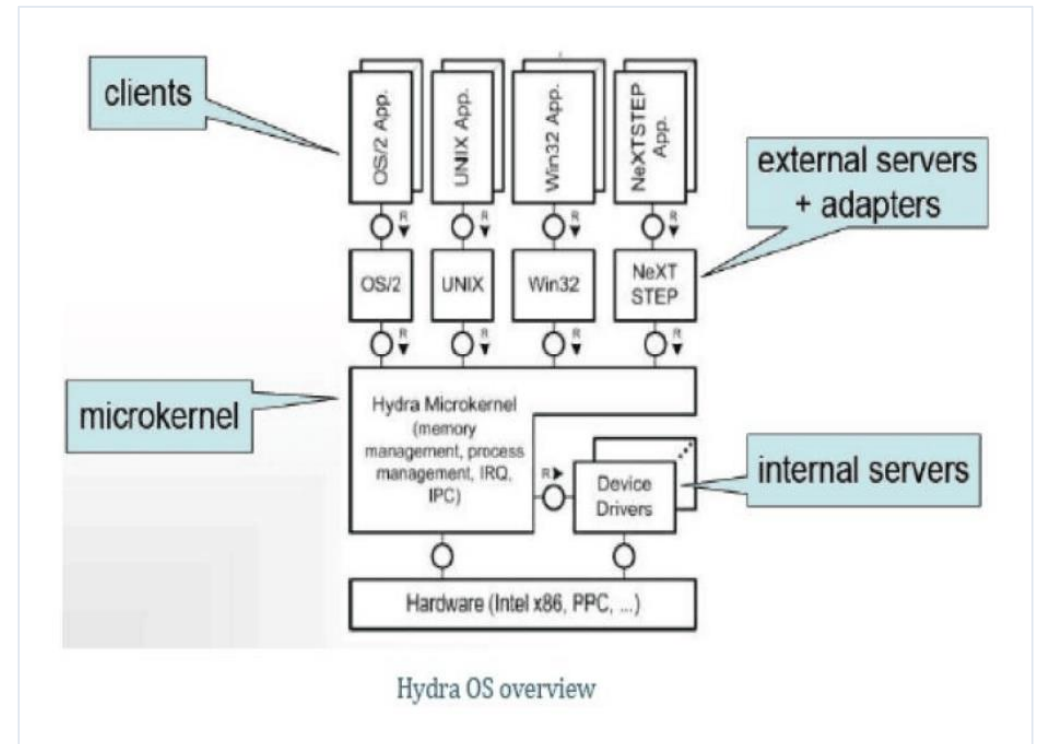


Hydra main window 안의 MS Windows·Unix·OS/2 앱

Microkernel Pattern — 5가지 참여 Component

■ Microkernel architecture pattern 의 참여 component

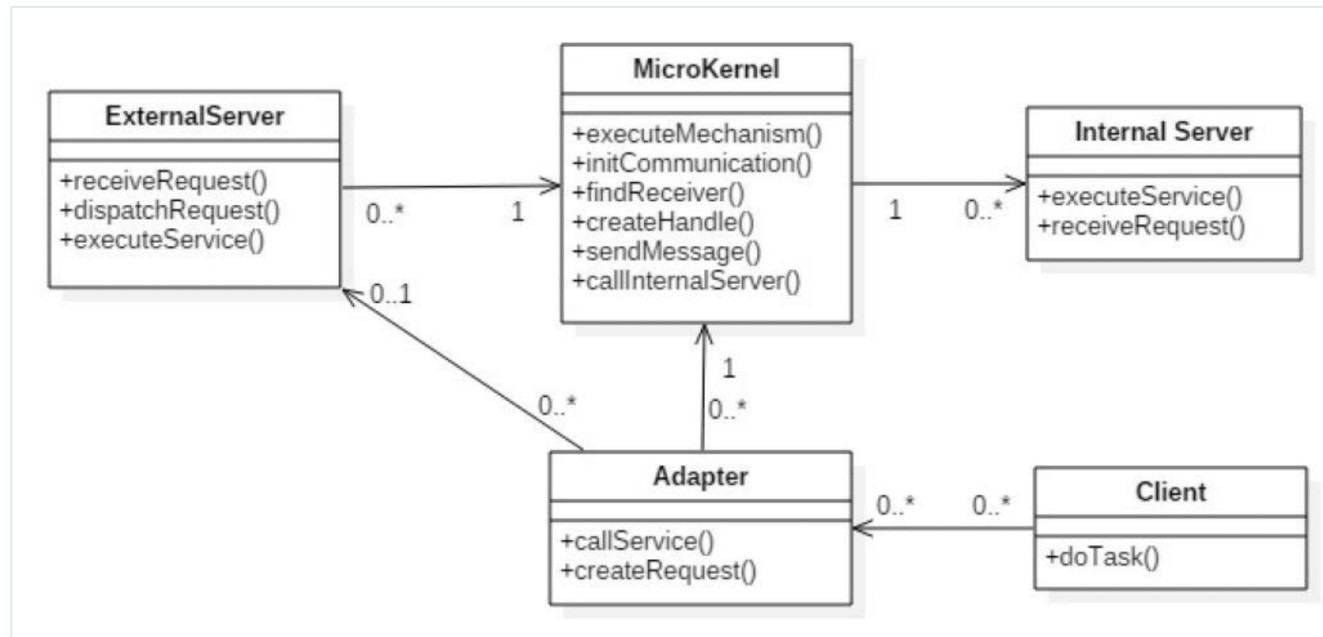
- Microkernel: 최소한의 기능 핵심(minimal functional core)
- Internal Server: 확장 기능(extended functionality)
- External Server: microkernel 에 대한 client 별 view
- Adapter: client 별 인터페이스
- Client: 정확히 하나의 external server 와 연결된 애플리케이션



Hydra OS overview

Microkernel Pattern — Structure

- Client 는 Adapter 를 통해(callService, createRequest) External Server 에 요청하고, External Server 는 Microkernel 의 메커니즘을 실행한다(executeMechanism, findReceiver, sendMessage)
- Microkernel 은 Internal Server 의 서비스를 호출한다(callInternalServer → executeService, receiveRequest)

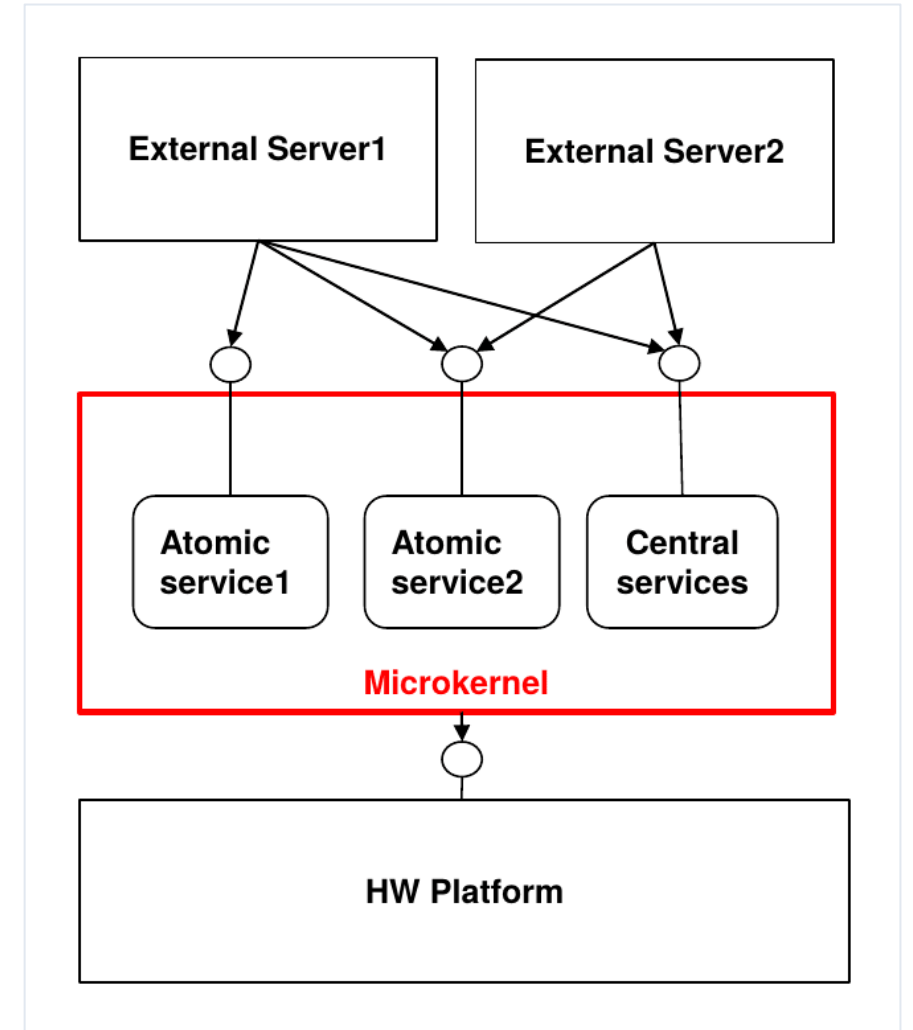


Microkernel Pattern 의 클래스 구조: Client – Adapter – ExternalServer – MicroKernel – Internal Server

Microkernel Component

■ 역할

- 중심 서비스(central services)를 구현한다 — 예: 통신 기능, 자원 관리(resource handling)
- 원자적 서비스(atomic services)를 구현한다 — 더 복잡한 기능의 기본 토대가 된다
 - 예(OS 프로세스 모델의 atomic service): 프로세스 생성 서비스(Unix, OS/2 Warp), 기존 주소 공간 복사 서비스(Unix)
- 시스템 고유의 의존성(예: 하드웨어 종속 부분)은 microkernel 안에 캡슐화된다



External Server 가 Microkernel 의 서비스를 사용

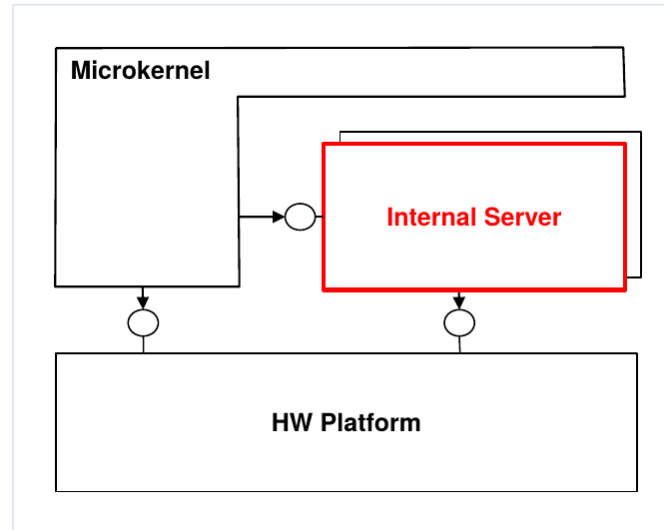
Internal Server 와 External Server Component

■ Internal Server

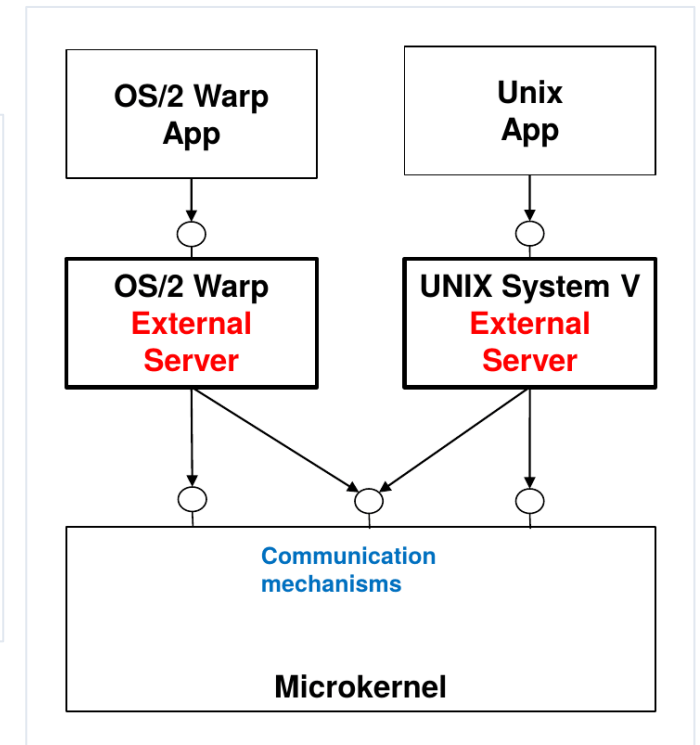
- microkernel 이 제공하는 기능을 확장한다
- 기반 하드웨어나 소프트웨어 시스템에 대한 일부 의 존성을 캡슐화한다
- microkernel 이 service request 를 통해 internal server 의 기능을 호출한다

■ External Server

- microkernel 을 이용해 기반 애플리케이션 도메인에 대한 자신만의 view 를 구현한다 (예: OS/2 Warp, UNIX System V External Server)
- 각 external server 는 별도의 process 로 실행된다
 - microkernel 이 제공하는 통신 기능을 이용해 client 의 service request 를 받는다



Microkernel → Internal Server

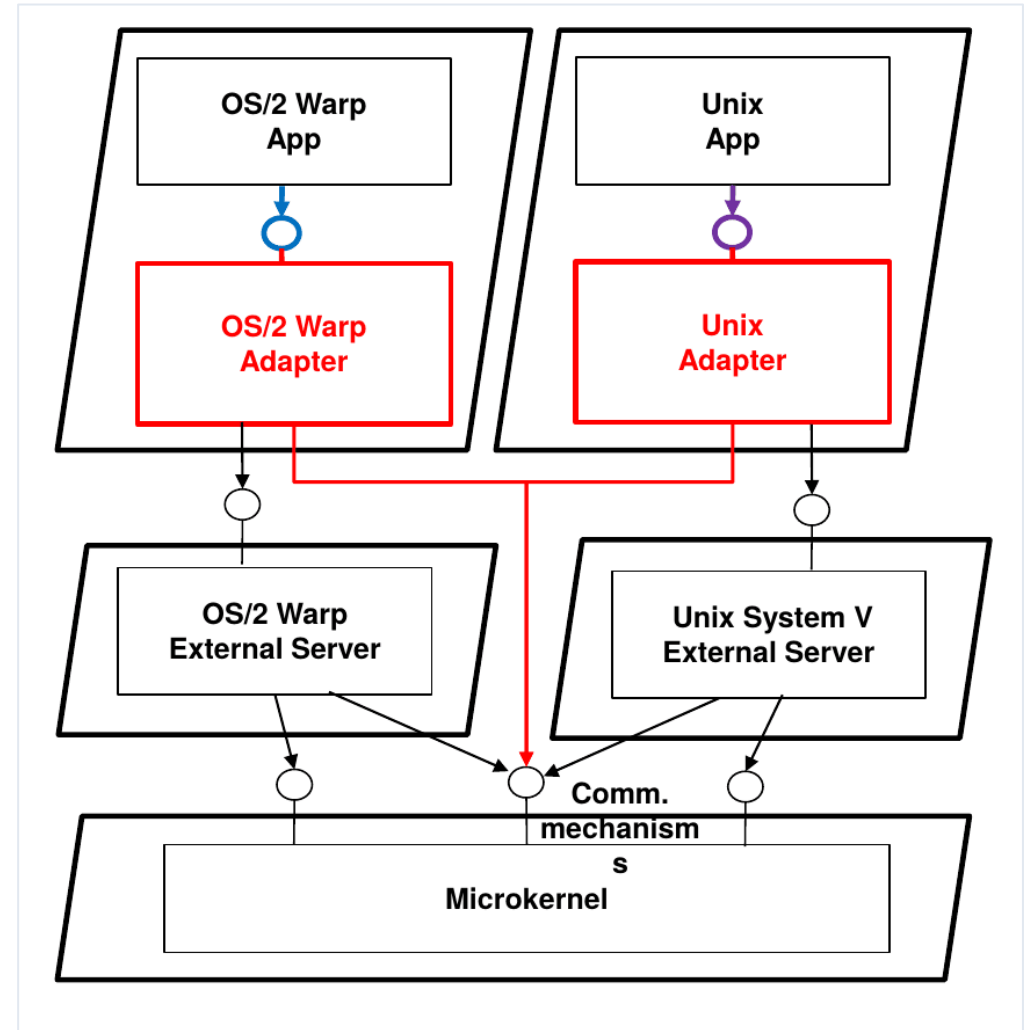


App → External Server → Microkernel

Adapter Component

■ Adapter

- client 와 external server 사이의 인터페이스를 나타낸다
- client 가 자신의 external server 의 서비스에 이식성 있게 (portable) 접근할 수 있게 해 준다
- 예: OS/2 Warp App 은 OS/2 Warp Adapter 를 통해, Unix App 은 Unix Adapter 를 통해 각자의 External Server 에 접근한다
- Adapter 와 External Server 는 microkernel 의 통신 메커니즘(Comm. mechanisms)으로 연결된다



App – Adapter – External Server – Microkernel

Microkernel Consequences: 장점과 약점

■ Benefits (장점)

- 유연성과 확장성(Flexibility and Extensibility): 새로운 view 가 필요하면 external server 만 추가하면 된다
 - 시스템에 기능을 더할 때는 internal server 를 추가하거나 확장하기만 하면 된다
- 가용성(Availability): 분산 microkernel 구조에서는 같은 server 를 여러 machine 에서 실행해 가용성을 높일 수 있다
- 이식성(Portability): microkernel 을 새 하드웨어 환경으로 옮길 때 하드웨어 종속 부분만 수정하면 된다

■ Weaknesses (약점)

- 'monolithic system' 보다 느리다
- 설계와 구현이 복잡하다

1.7

Distributed Architecture

네트워크로 연결된 여러 컴퓨팅·저장 장치에 시스템을 나누어 배치하는 스타일

- Client-Server Style
- Multi-tier Style
- Broker Style
- Dispatcher Style
- Service-Oriented Architecture (SOA)
- Microservice Architecture

Distributed Architecture(분산 Architecture) 개요

■ 정의

- 시스템을 통신 네트워크로 연결된 여러 컴퓨팅 장치와 저장 장치의 집합으로 나눈다
- 분산 시스템 안의 subsystem 이나 component 는 다양한 방법으로 서로 통신한다
 - message passing, remote procedure call(RPC), remote method invocation(RMI) 등

■ 분류 (Types of classification)

- Multi-tier — 하위 스타일: Client-Server
- Broker — 하위 스타일: Dispatcher
- Service-Oriented Architecture(SOA) — 하위 스타일: Microservice
- Edge-based (Deployment Architecture 에서 다룬다)

핵심 분산 Architecture 는 '무엇을 어디에 두고, 어떻게 통신할 것인가' 를 결정하는 스타일이다

Client-Server Style — 구성 요소

■ 개요

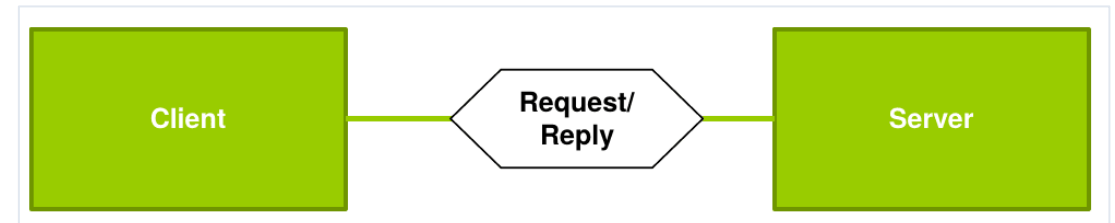
- 시스템을 client 와 server, 두 종류의 component 로 분해한다

■ Elements

- Client component: server 에 요청(request)을 보낸다
- Server component: 요청을 받아 처리(DB 데이터 제공, 문서 인쇄 등)하고 응답(reply)을 돌려준다
- Request/reply connector: request/reply 프로토콜을 쓰는 data connector 로, client 가 server 의 서비스를 호출할 때 사용

■ Constraints

- Component 들을 tier(계층) 로 배치할 수 있다

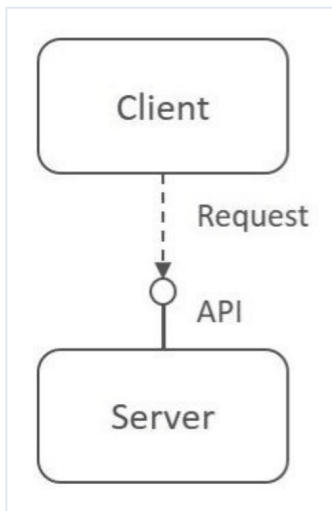


Client 와 Server 를 잇는 Request/Reply connector

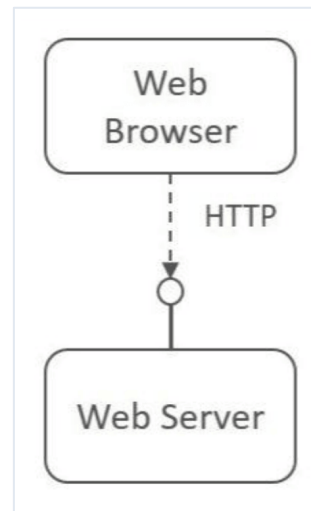
Client-Server Style — 예제

■ 대표적인 Client-Server 조합

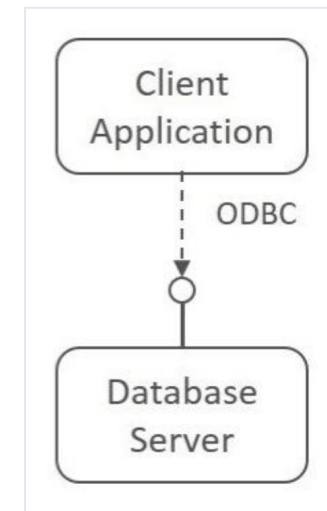
- Client → (Request, API) → Server: 가장 일반적인 형태
- Web Browser → (HTTP) → Web Server
- Client Application → (ODBC) → Database Server
- 연결 방식(API, HTTP, ODBC)이 달라도 '요청하고 응답받는' 구조는 같다



API 호출



HTTP

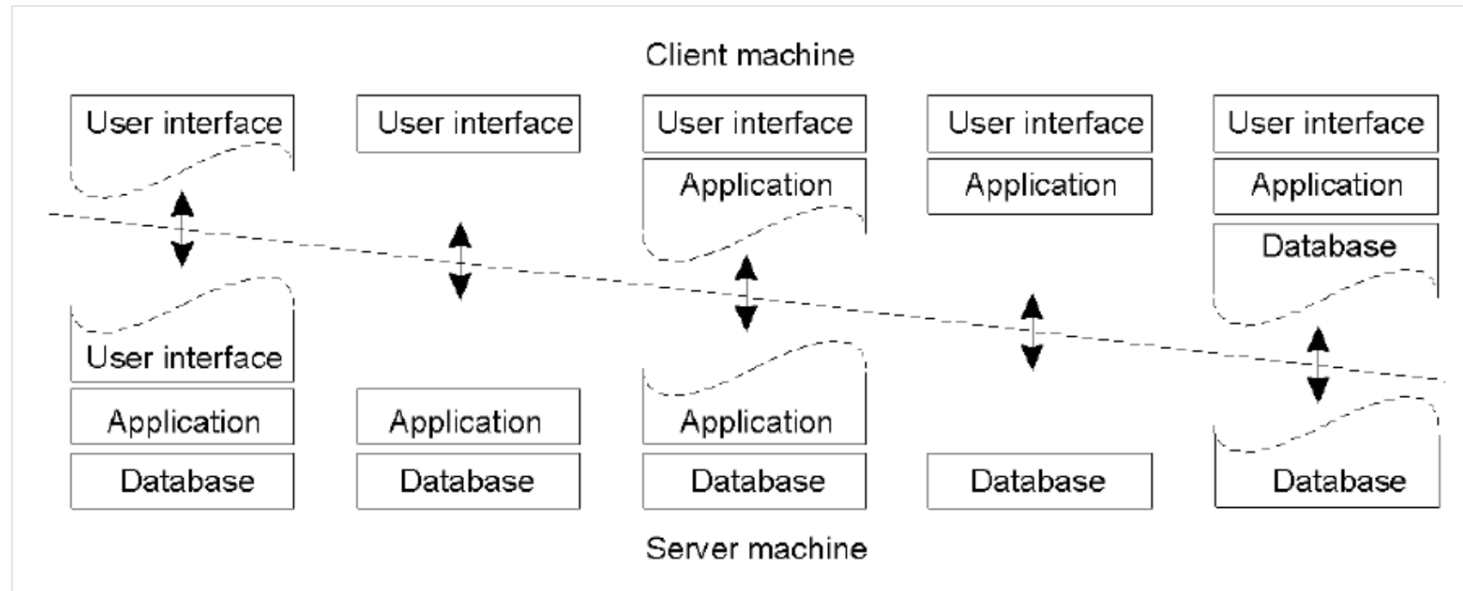


ODBC

Client-Server 변형: Thin vs. Thick Client

■ 어디까지를 client 에 둘 것인가

- Thin client: user interface 만 client 에 두고 application 과 database 는 server 에서 처리
- Thick(Fat) client: application 로직, 경우에 따라 database 일부까지 client 에서 담당
- User interface / Application / Database 를 어떻게 나누느냐에 따라 여러 중간 형태가 있다



왼쪽(thin client)에서 오른쪽(thick client)으로 갈수록 client 의 역할이 커진다

Client-Server Consequences: 장점과 약점

■ Benefits (장점)

- UI 표현과 business logic 처리 같은 책임을 분리할 수 있다
- Server component 를 재사용할 수 있다
- 동시성(concurrency)과 확장성(scalability)의 가능성
- 많은 client 가 고성능 server 에 접근할 때 자원을 효과적으로 사용한다

■ Weaknesses (약점)

- 보안 문제가 복잡해진다
- Server 가 성능 병목(bottleneck)이 될 수 있다
- Server 가 single point of failure 가 될 수 있다
- 테스트 용이성(testability)이 제한된다

Multi-tier Architecture Style 개요

■ 정의

- 시스템을 서로 다른 기능 단위인 tier 로 분해한다
- 각 tier 는 고유한 책임을 가지며, 서로 상호작용해 전체 application 의 기능을 완성한다

■ Layer vs. Tier

- Layer: 소프트웨어를 구성하는 요소들을 논리적으로 구조화하는 방법
- Tier: 시스템 인프라를 물리적으로 구조화하는 방법
 - 즉 layer 는 코드의 구분, tier 는 실행 환경(장비)의 구분이다

■ Variants

- 2-tier Architecture (= Client-Server Architecture)
- 3-tier Architecture
- 4-tier Architecture

핵심 Layer 는 논리적(logical) 구분, Tier 는 물리적(physical) 구분이다

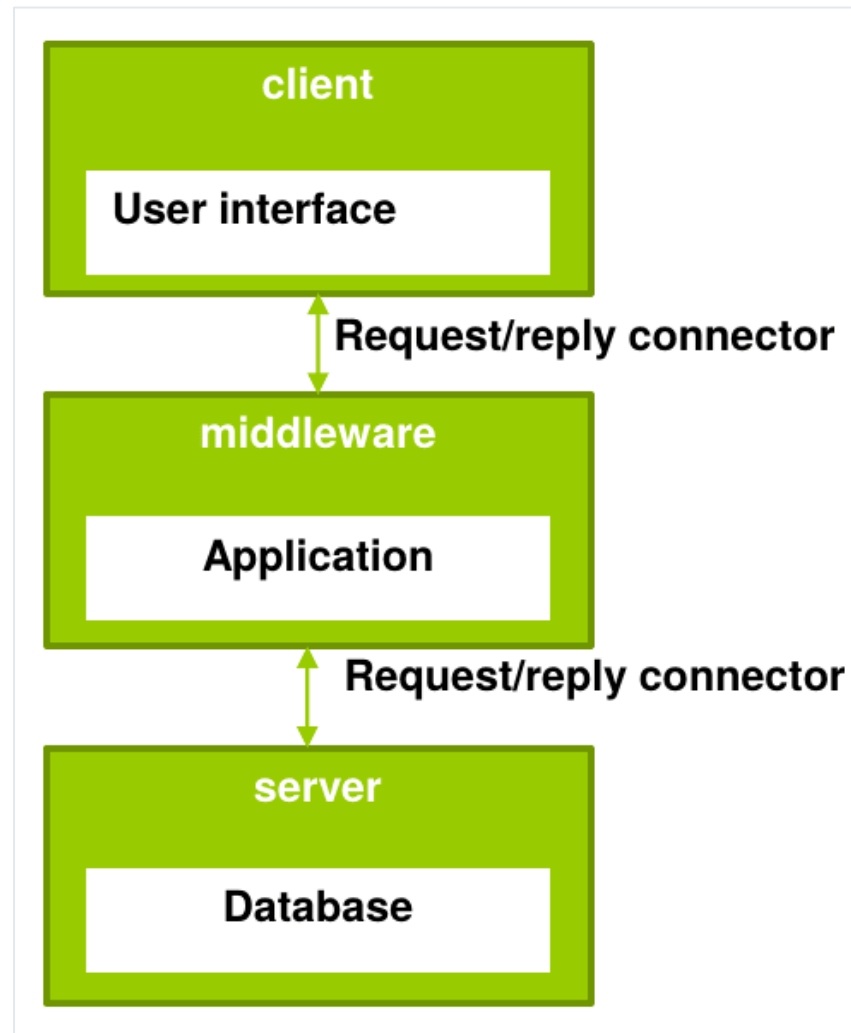
3-tier Architecture Style — 구성 요소

■ Elements

- Client — Presentation Tier (user interface)
- Middleware — Application Tier
 - business logic 과 application 고유 처리의 개발을 지원
- Server — Data management Tier (database)
- Request/reply connector

■ Relations

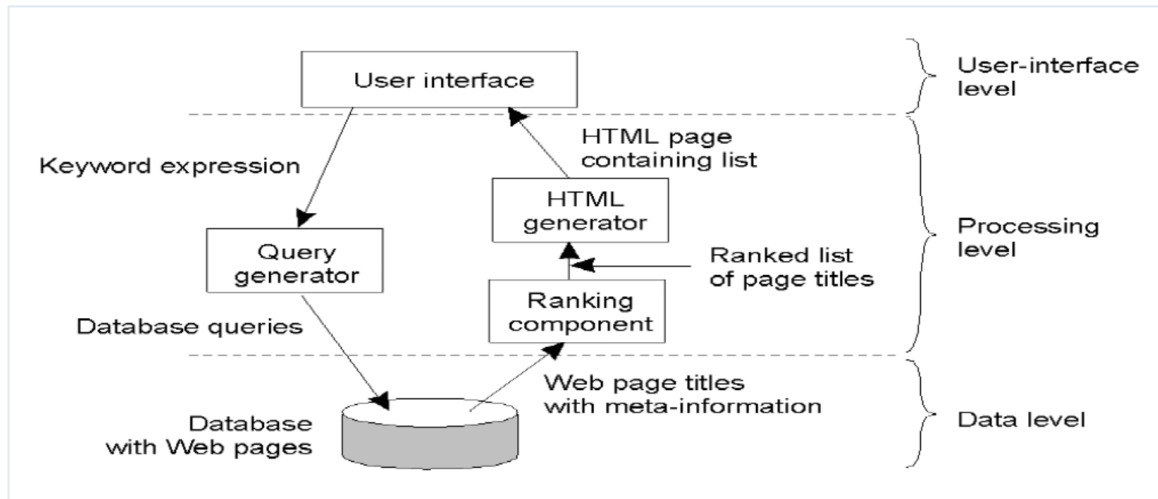
- client ↔ middleware, middleware ↔ server 가 각각 request/reply connector 로 연결된다



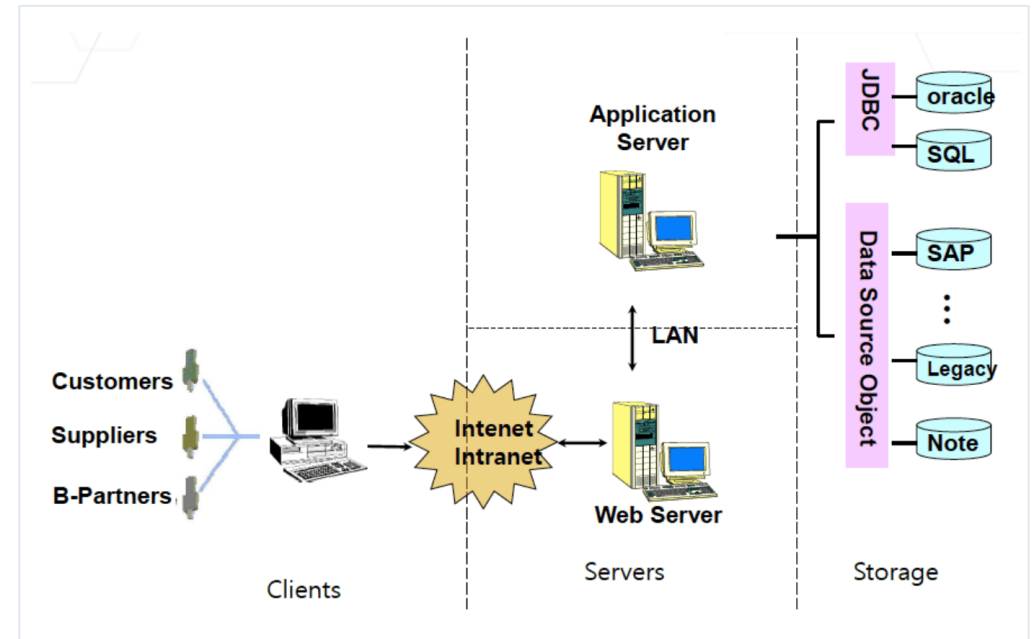
3-tier: client – middleware – server

3-tier / 4-tier Architecture 예제

- 3-tier 예제(왼쪽): 검색 시스템 — User-interface level / Processing level(Query generator, Ranking, HTML generator) / Data level(웹 페이지 DB)
- 4-tier 예제(오른쪽): Clients(고객·공급자·파트너) – Web Server – Application Server – Storage(Oracle, SQL, SAP, Legacy 등)



3-tier 검색 시스템



4-tier 기업 시스템

Multi-tier Consequences: 장점과 약점

■ Benefits (장점)

- 재사용성과 수정 용이성(Modifiability)
 - business 관련 변경은 middle tier 만 고치면 된다
- Middle tier 가 multithreading 을 지원해 확장성과 성능을 높일 수 있다
- 보안 강화
 - presentation / application / data tier 를 분리하면 민감한 business logic 과 데이터를 최종 사용자의 직접 접근에서 보호할 수 있다

■ Weaknesses (약점)

- 복잡성과 overhead 증가
- 테스트 도구가 부족해 testability 가 떨어진다
- Server 가 여러 대로 늘어나면 server 의 reliability 와 availability 가 더욱 중요해진다

Broker Architecture Style 개요

■ 정의

- 분산 컴퓨팅에서 등록된 server 와 client 사이의 통신을 조정·중개하는 middleware Architecture
- Client 와 server component 는 Broker 를 통해 서로 분리(decoupled)된다

■ Broker component 의 역할

- 통신 조정(coordinating communication)을 책임진다
 - 적절한 server 찾기(locating)
 - 요청 전달과 배분(forwarding, dispatching)
 - 응답을 client 에게 되돌려 보내기

핵심 Client 는 server 가 어디 있는지 몰라도 된다 — Broker 가 대신 찾아 연결해 준다

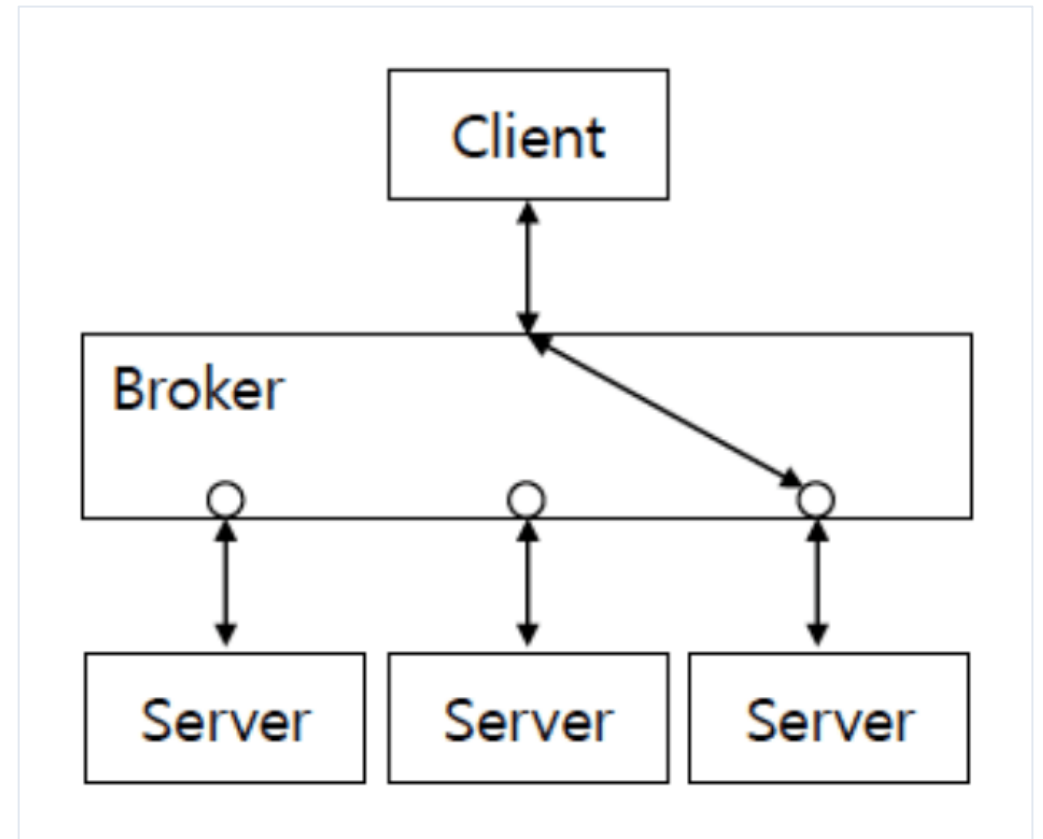
Broker Style — 구성 요소와 제약

■ Elements

- Servers: broker 에 등록(register)하고, client 에게 method interface 를 제공
- Clients: broker 를 통해 server 의 method 에 접근, server method 호출 형식이 같다
- Broker 의 일: 적절한 server 찾기, server 로 요청 전달, 결과와 exception 을 client 로 전송

■ Constraints

- Client 와 server 는 직접 상호작용하지 않는다

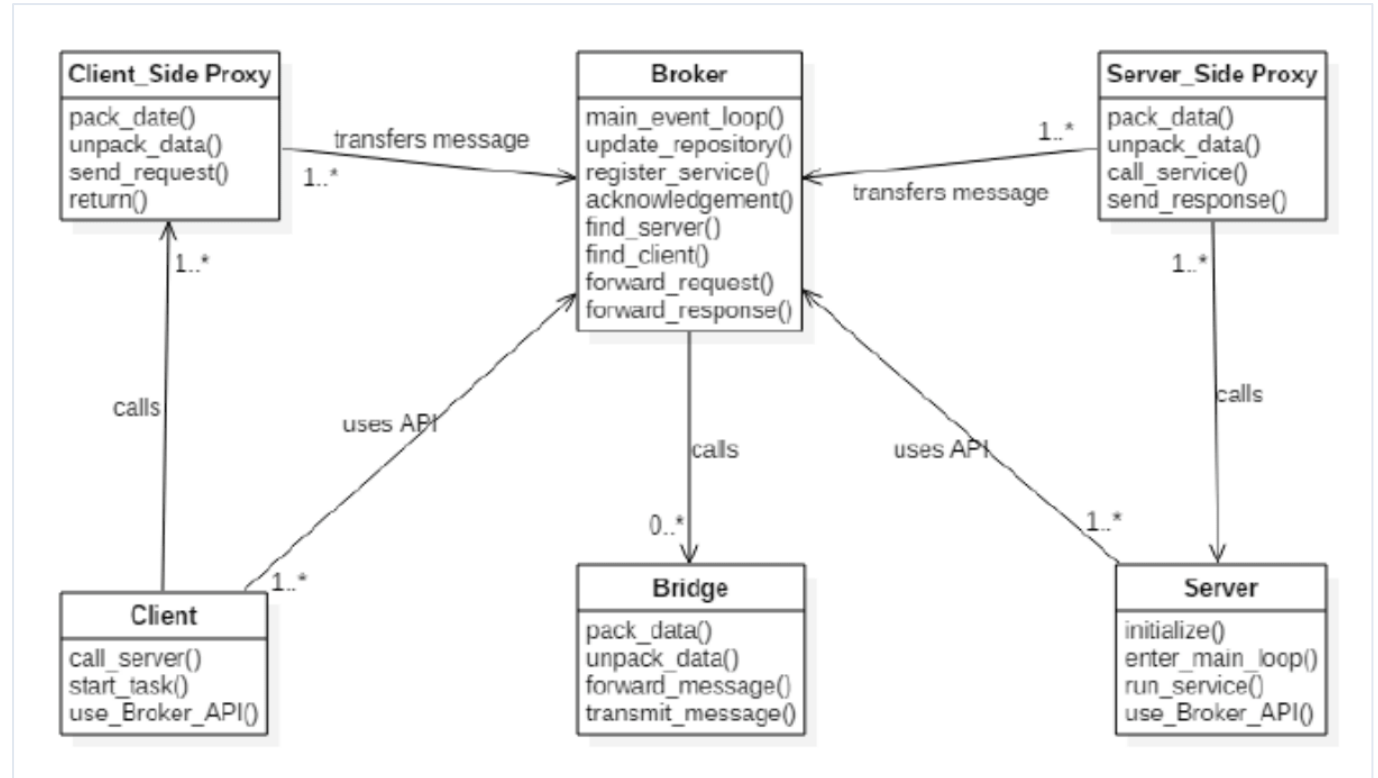


Relations: Client – Broker – Servers

Broker 예제 — CORBA 통신 Architecture

■ CORBA (Common Object Request Broker Architecture)

- Client 은 Client_Side Proxy 를, Server 는 Server_Side Proxy 를 통해 Broker 와 메시지를 주고받는다
- Proxy 가 데이터를 pack/unpack 하고 Broker 가 find_server / find_client / forward_request / forward_response 를 수행
- Bridge 는 서로 다른 Broker 사이의 메시지를 전달한다

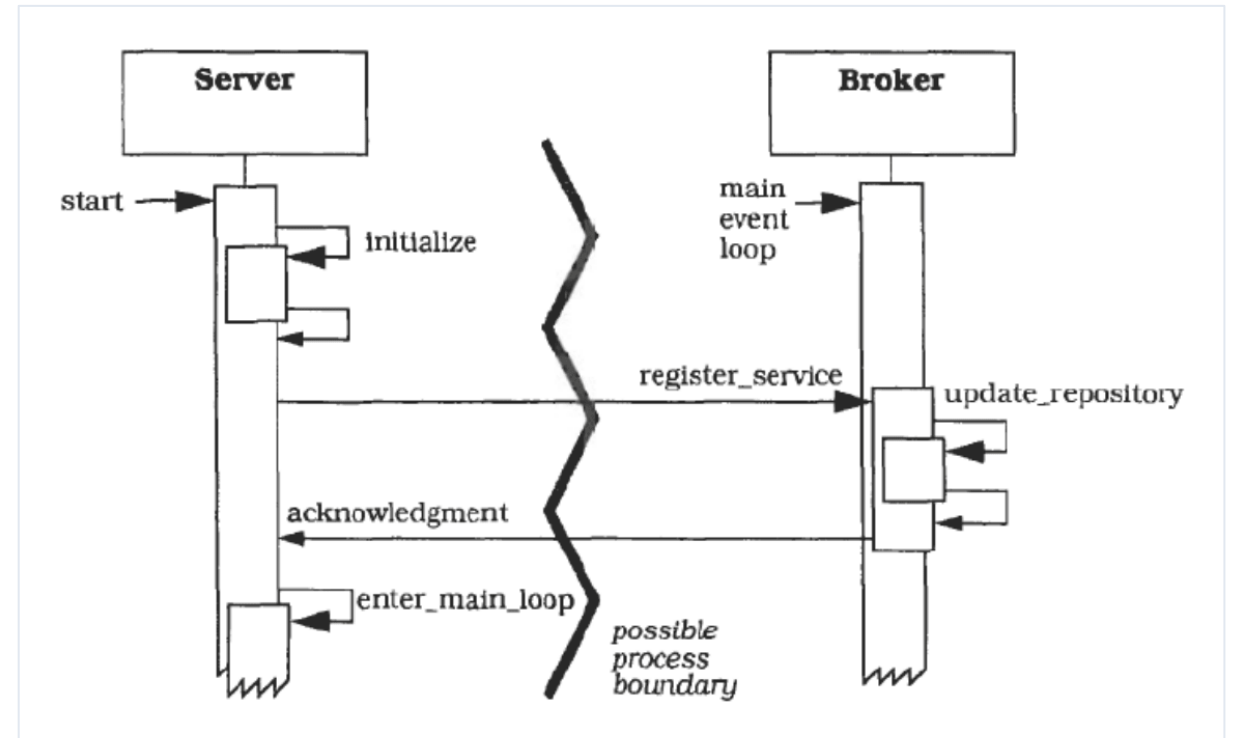


CORBA 의 Client / Proxy / Broker / Bridge / Server 구조

Broker 동적 시나리오 (1) — Server 등록

■ Server registration

- Server 가 시작(start)하면 initialize 후 Broker 에 register_service 를 보낸다
- Broker 는 main event loop 안에서 update_repository 로 server 정보를 저장하고 acknowledgment 를 돌려준다
- 등록이 끝나면 Server 는 enter_main_loop 로 요청 대기 상태에 들어간다

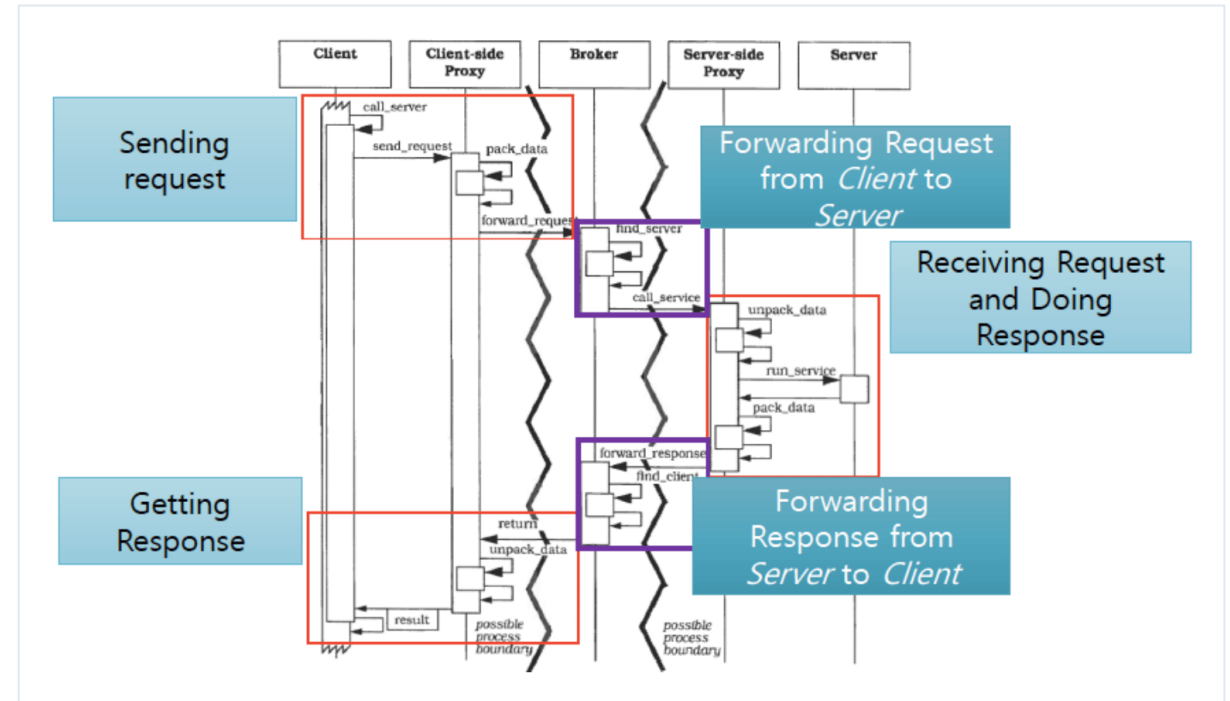


Server - Broker 등록 sequence

Broker 동적 시나리오 (2) — 서비스 처리

■ Service process

- Sending request: Client 가 Client-side Proxy 를 통해 요청을 보낸다
- Forwarding request: Broker 가 server 를 찾아 요청을 Server-side Proxy 로 전달
- Receiving request and doing response: Server 가 요청을 처리한다
- Forwarding response / Getting response: 응답이 Broker 와 Proxy 를 거쳐 Client 로 돌아간다

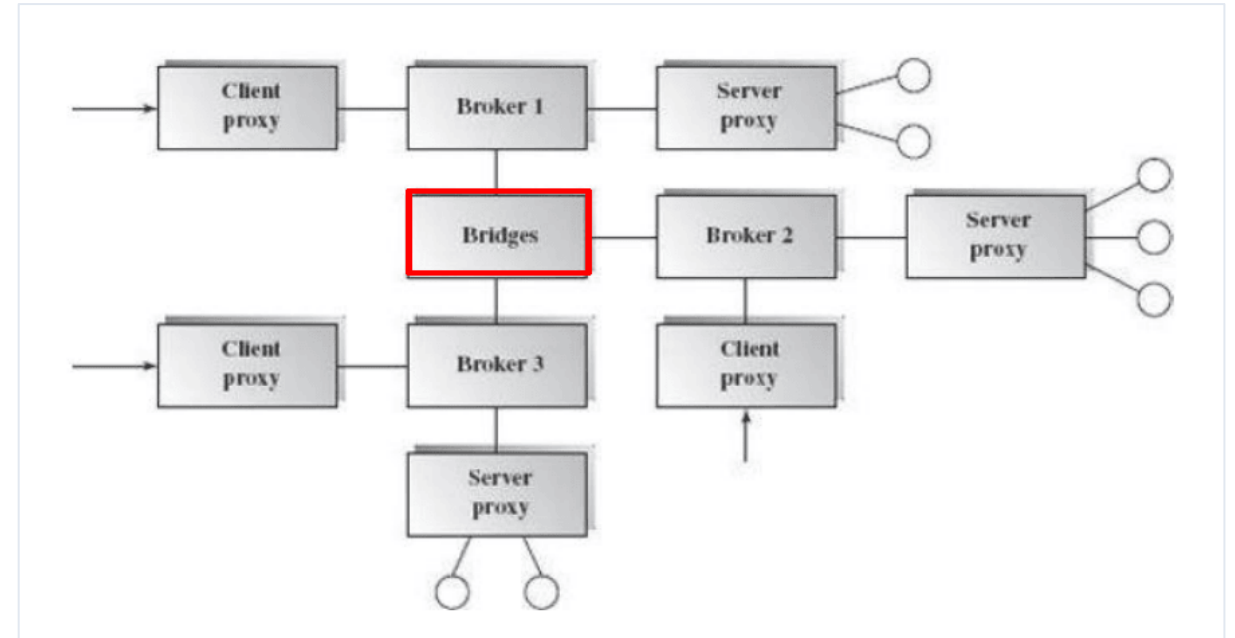


Client → Proxy → Broker → Proxy → Server 왕복 과정

Broker 의 Bridge

■ Bridge 란

- 서로 다른 통신 프로토콜을 사용하는 두 네트워크를 연결한다
- 하부 네트워크 구현 세부 사항을 감싸고 (encapsulate) 서로 다른 broker 를 중재(mediate) 한다
 - 요청과 parameter 를 한 형식에서 다른 형식으로 변환할 수 있다



Bridge 가 Broker 1 · Broker 2 · Broker 3 사이를 연결

Broker Consequences: Benefits 상세

■ 위치 투명성 (Location transparency)

- Broker 가 고유 식별자로 server 를 찾아 주므로 client 는 server 의 위치를 알 필요가 없다

■ Component 의 변경·확장 용이성

- Server 가 바뀌어도 interface 가 같으면 client 에는 기능적 영향이 없다
- Broker 내부 구현을 바꿔도 API 가 같으면 성능 외에는 영향이 없다

■ 이식성 (Portability)

- API, proxy, bridge 같은 간접 계층으로 OS 와 네트워크 세부 사항을 client-server 로부터 숨긴다

■ 상호운용성 · 재사용성 · 런타임 변경

- Bridge 가 공통 메시지 교환 프로토콜을 구현하면 서로 다른 Broker 시스템끼리 상호운용(interoperate)할 수 있다
- 새 client application 을 만들 때 기존 서비스를 기반으로 기능을 구성할 수 있다
- 실행 중에 server component 를 추가·제거할 수 있다

Broker Consequences: 장점과 약점

■ Benefits (장점)

- Location transparency (위치 투명성)
- Component 의 changeability 와 extensibility
- Broker 시스템의 portability
- Broker bridge 를 통한 interoperability
- 기존 서비스의 reusability
- 실행 중 server component 추가·제거 (runtime changes)

■ Weaknesses (약점)

- 비효율성: proxy 와 broker 를 거치는 간접 호출 때문
- 낮은 fault tolerance
 - server 나 broker 가 실행 중 죽으면 이에 의존하는 모든 application 이 계속 진행할 수 없다
 - component 를 복제(replication)하면 신뢰성을 높일 수 있다
- 테스트와 디버깅이 어렵다
 - 관련된 component 가 많아 지루하고 번거로운 작업이 된다

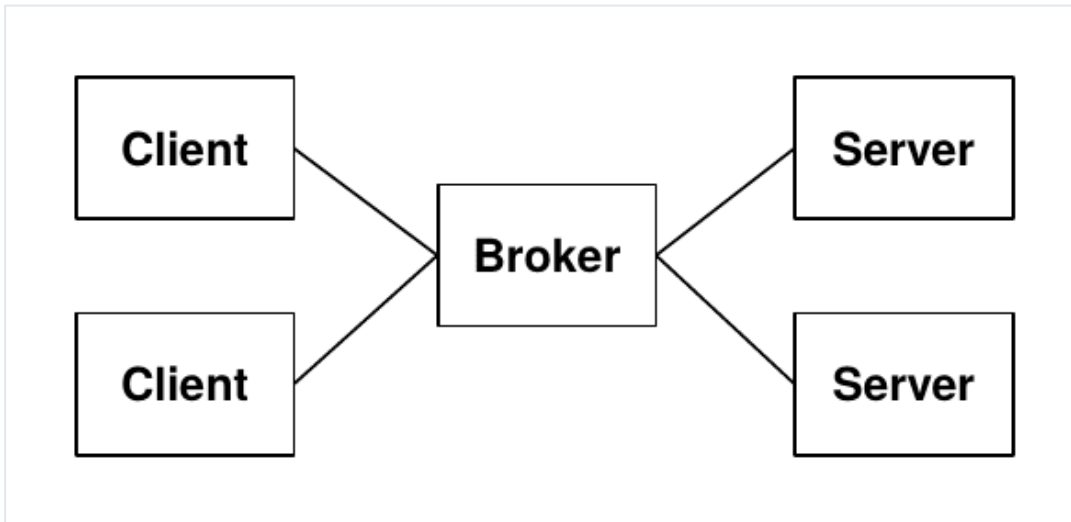
Dispatcher Style 개요

■ 정의

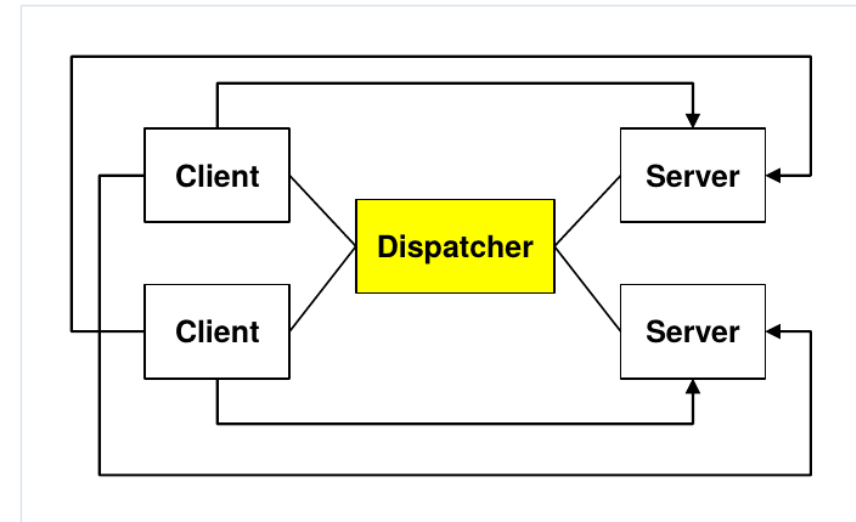
- Client 와 server 사이에 중간 계층인 dispatcher component 를 둔다
- Direct Communication Broker 변형의 경량(lightweight) 버전이다

■ 동작 방식

- Client 는 dispatcher 에 의존해 특정 server 를 찾고 그 server 와 통신 링크를 맺는다
- 링크가 만들어진 뒤에는 client 와 server 가 직접 통신한다 — 모든 메시지가 broker 를 거치는 Broker 스타일과의 차이



Broker: 모든 통신이 Broker 를 경유



Dispatcher: 연결만 맺어 주고 이후는 직접 통신

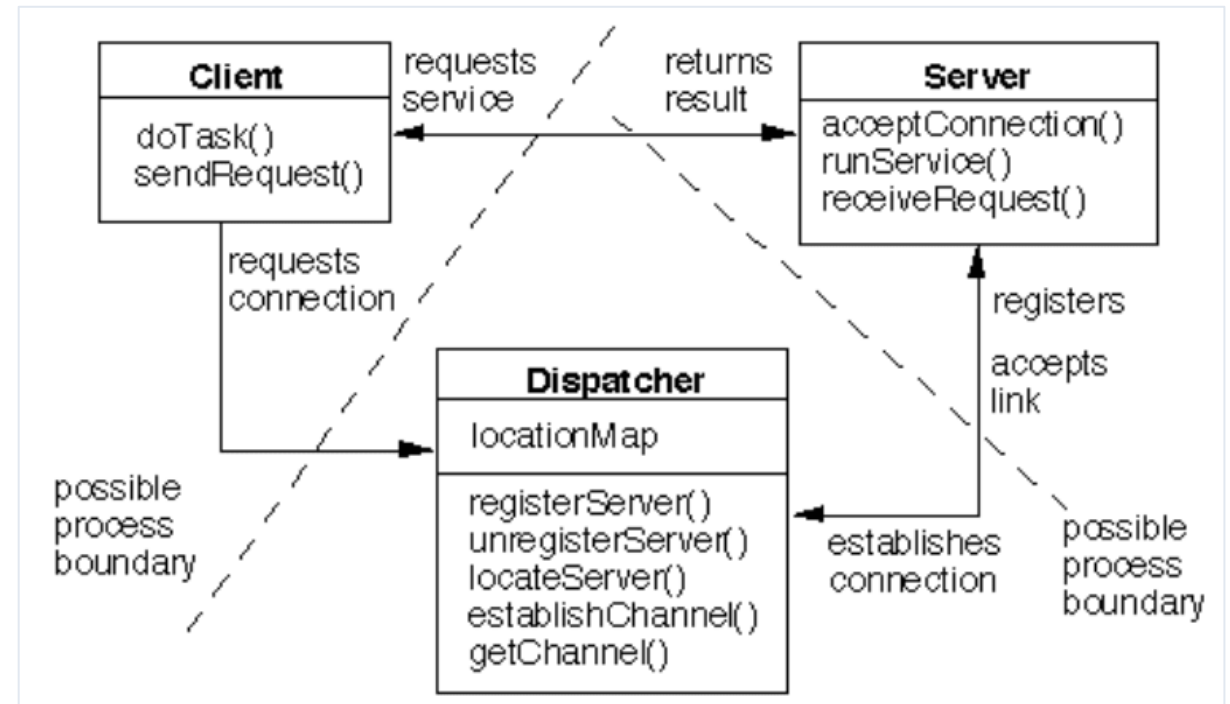
Dispatcher Style — 구성 요소

■ Elements

- Client: dispatcher 에 server 연결을 요청하고, server 의 서비스를 호출
- Server: client 에 서비스를 제공하고, dispatcher 에 자신을 등록
- Dispatcher: 통신 채널 수립, server 찾기, server 등록, server 위치 map 관리

■ Relations

- Client → Dispatcher: requests connection
- Server → Dispatcher: registers
- Dispatcher → Server: establishes connection
- Client ↔ Server: requests service / returns result

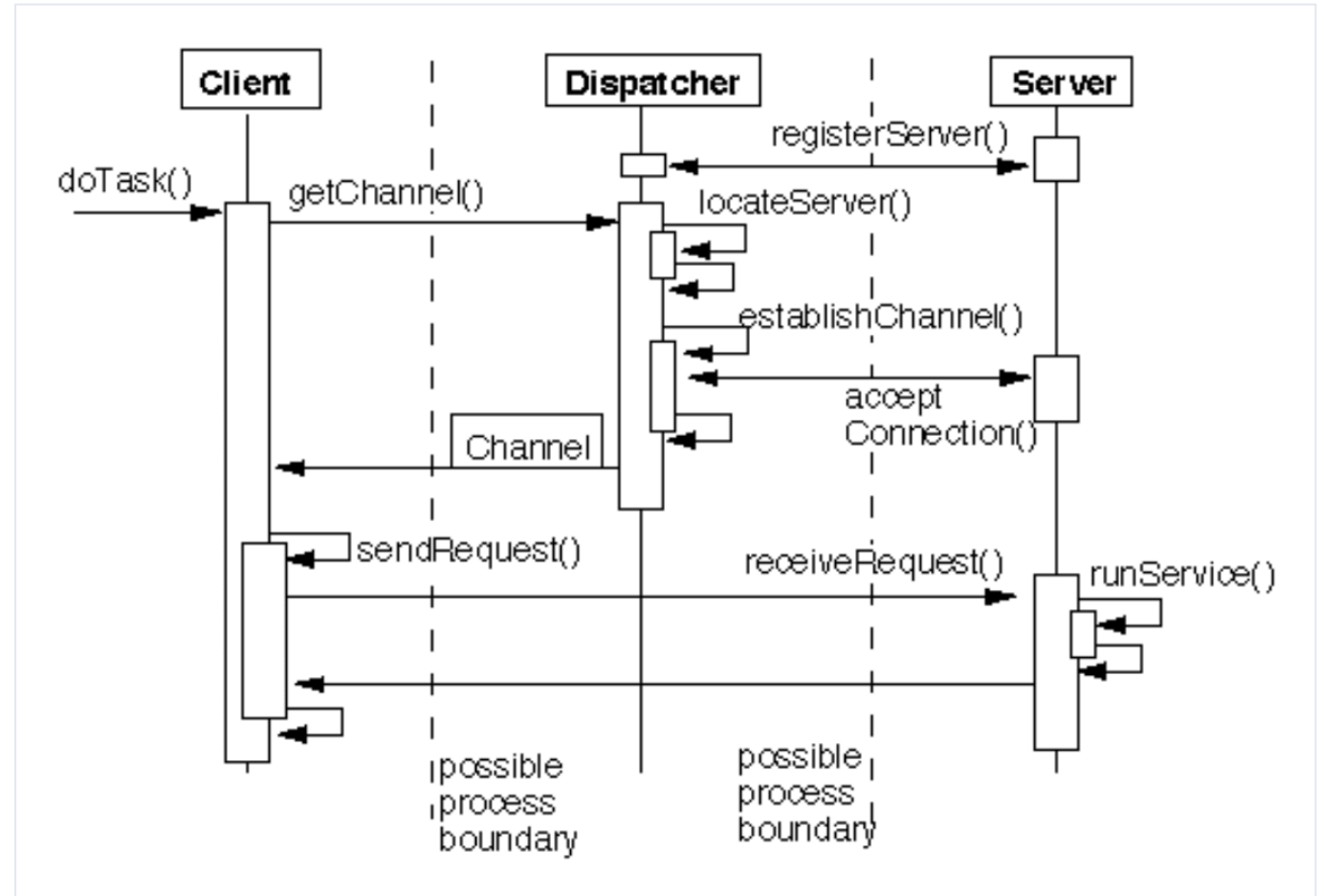


Client · Dispatcher(locationMap) · Server 의 관계

Dispatcher Style — Dynamics

■ 동작 순서

- Server 가 registerServer() 로 dispatcher 에 등록
- Client 가 doTask() 를 시작하며 getChannel() 로 채널 요청
- Dispatcher 가 locateServer() 로 server 를 찾고 establishChannel() → server 가 acceptConnection()
- Channel 이 만들어지면 client 가 sendRequest(), server 가 receiveRequest() 후 runService() 로 처리

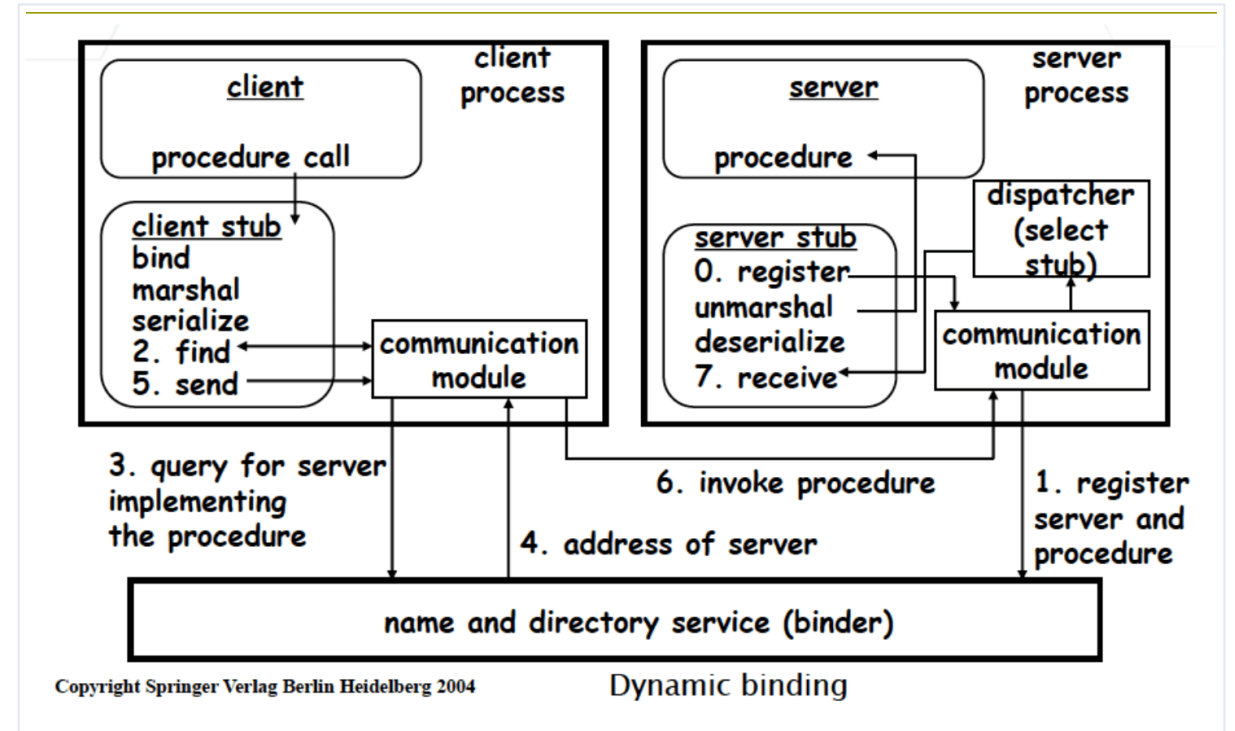


Client – Dispatcher – Server sequence

Dispatcher 예제 — RPC (Dynamic binding)

■ Remote Procedure Call

- Server 는 name and directory service(binder) 에 server 와 procedure 를 등록(1. register)
- Client stub 은 binder 에 procedure 를 구현한 server 를 질의(3. query)하고 server 주소를 받는다 (4. address)
- 이후 client stub 이 server stub 에 직접 procedure 를 호출(6. invoke) — binder 가 dispatcher 역할
- Client stub 은 bind / marshal / serialize / find / send, server stub 은 register / unmarshal / deserialize / receive 를 담당

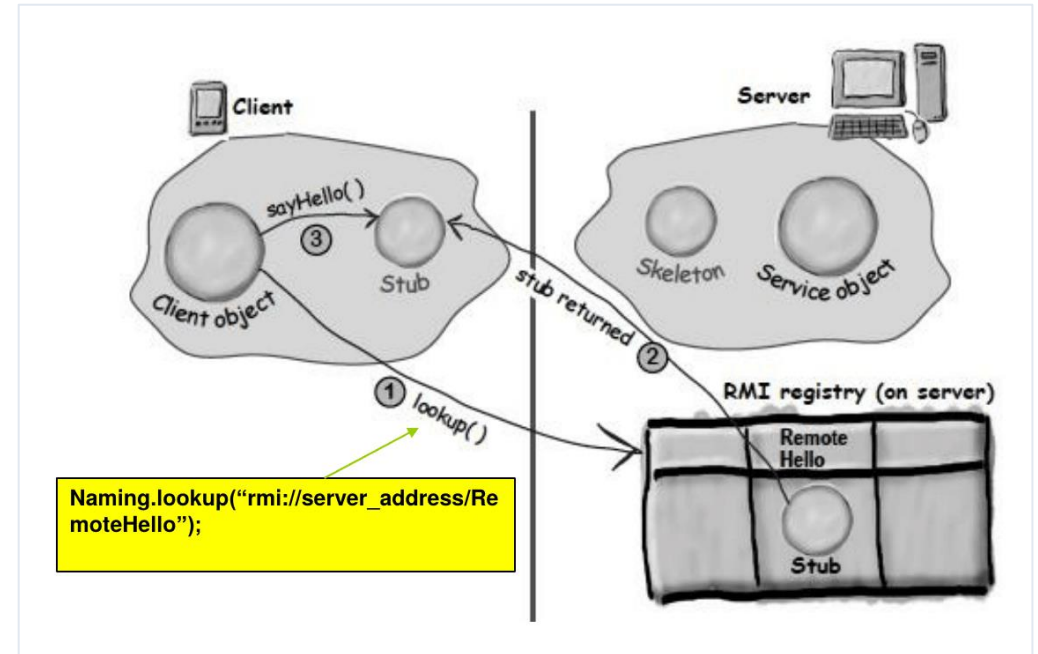


RPC 의 dynamic binding: binder 가 server 를 찾아 준다

Dispatcher 예제 — Java RMI

■ Remote Method Invocation

- Server 의 Service object 는 RMI registry 에 Remote Hello 라는 이름으로 등록된다
- Client 는 `Naming.lookup("rmi://server_address/RemoteHello")` 로 registry 에서 찾는다(1. lookup)
- Registry 가 stub 을 돌려주면(2. stub returned) client 는 stub 을 통해 `sayHello()` 를 호출(3.)
- Stub ↔ Skeleton 이 네트워크 통신을 대신 처리한다 — RMI registry 가 dispatcher

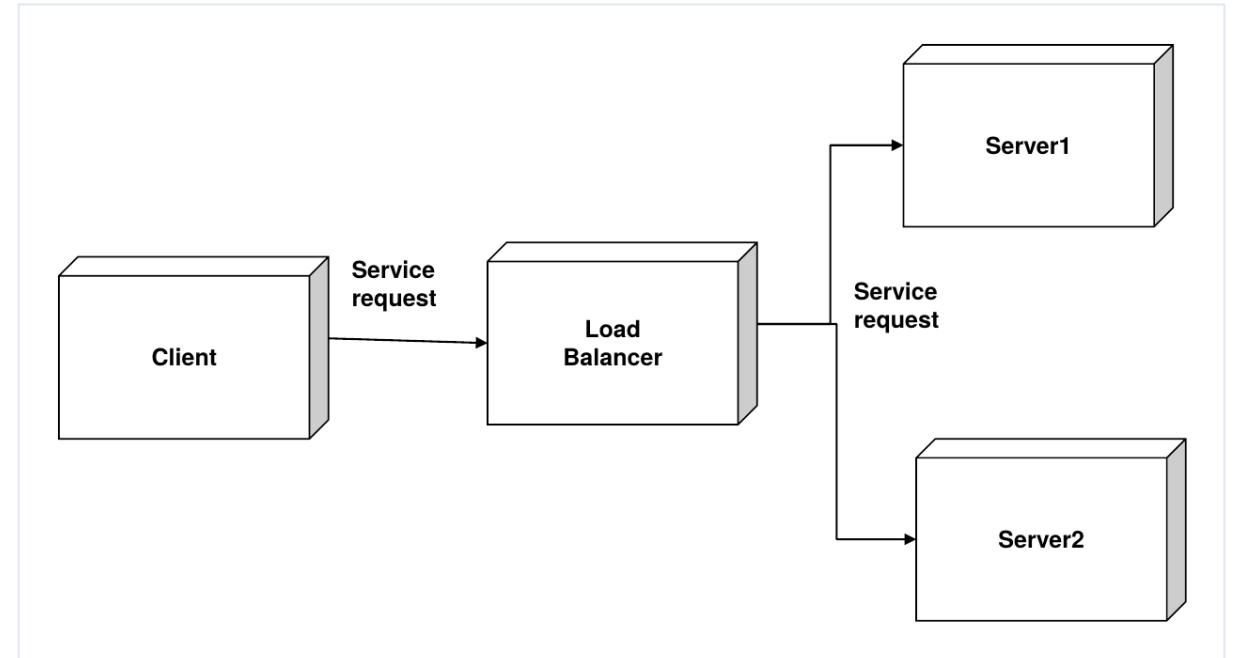


Java RMI: registry 를 통한 stub 획득과 원격 호출

Dispatcher 예제 — Load Balancer

■ Load Balancer

- Client 의 service request 를 Load Balancer 가 받아 Server1 또는 Server2 로 배분한다
- Client 는 어느 server 가 처리하는지 알 필요가 없다
- 여러 server 에 부하를 나누어 확장성과 가용성을 높인다



Client → Load Balancer → Server1 / Server2

Dispatcher Consequences: 장점과 약점

■ Benefits (장점)

- Server 교체 용이성(Exchangeability)
 - dispatcher 나 client 를 고치지 않고 server 를 추가·수정할 수 있다
- 위치·이동 투명성(Location and migration transparency)
 - client 가 연결 중이 아니라면 server 를 다른 장비로 동적으로 옮길 수 있다
- 재구성(Re-configuration)
 - 어떤 server 를 어디에 둘지 시작 시점이나 그 이후에 결정할 수 있다
- Fault tolerance
 - 네트워크나 server 장애 시 다른 장비에서 새 server 를 활성화할 수 있다

■ Weaknesses (약점)

- Dispatcher overhead
- Dispatcher 의 interface 변경에 민감하다

Service-Oriented Architecture (SOA) 개요

■ Service 란?

- 다음 조건을 만족하는 business 기능
 - 잘 정의되어 있다(well-defined)
 - 자기 완결적이다(self-contained)
 - 다른 서비스와 독립적이다
 - 표준 프로그래밍 interface 로 공개(published)되어 사용할 수 있다

■ Service-Oriented Architecture

- Application component 가 네트워크 상의 통신 프로토콜을 통해 다른 component 에 서비스를 제공하는 소프트웨어 설계 스타일
- 그 원칙은 특정 vendor 나 기술에 독립적이다

핵심 SOA 의 핵심 단위는 '표준 interface 로 공개된, 독립적인 business 기능' 인 service 이다